

Билеты по дисциплине “Сети ЭВМ и телекоммуникации”

1 - Методы коммутации. Коммутация каналов и коммутация пакетов. Достоинства и недостатки. Принципы реализации.	3
2 - Иерархическая организация связи. Декомпозиция, уровни взаимодействия. Эталонная модель ISO/OSI. стек протоколов.	7
3 - Иерархическая организация связи. Инкапсуляция, мультиплексирование/демультиплексирование.	11
4 - Характеристики каналов и линий связи.	14
5 - Канальный уровень. Функции уровня. Принципы определения границ кадра. Методы обнаружения ошибок. MTU	17
6 - Канальный уровень. Подуровни канального уровня. Технология Ethernet. Формат кадров.	24
7 - Канальный уровень. Коллизии. Метод CSMA/CD. Автосогласование.	29
8 - Мосты, коммутаторы. Принципы работы. Достоинства и проблемы использования.	32
9 - Мосты, коммутаторы. Алгоритм покрывающего дерева. 802.1Q. VLAN.	35
10 - Беспроводные сети. История развития беспроводных сетей. Характеристики беспроводных сетей различных стандартов.	39
11 - Беспроводные сети. DSSS. MIMO. Фазовая манипуляция. Соотношение скорости соединения и размера кадров.	43
12 - Функции сетевого уровня. Составная сеть. Адресация, маршрутизация, маршрутизаторы. Типовая таблица маршрутизации.	46
13 - Адресация в сетях TCP/IP. Классы сетей. Структуризация IP-сетей с помощью масок. Бесклассовая адресация.	50
14 - Протокол IP. Основные функции. Формат IP-пакета. Фрагментация.	53
15 - Взаимное отображение физических адресов и IP-адресов.	55
16 - Протокол ICMP.	58
17 - IPv6. Формат заголовка. Расширенные заголовки. Сравнение IPv4 и IPv6.	60
18 - Адресация в IPv6. Типы и структура адресов.	64
19 - ICMPv6 и NDP.	69
20 - DHCP для IPv4, DHCPv6.	73
21 - NAT, принципы работы, преимущества и недостатки.	76
22 - Маршрутизация, статическая и динамическая, правило наидлиннейшего префикса. Link-State vs Distance-Vector.	79
23 - OSPF. Алгоритм работы. Зоны, формат и типы пакетов.	82
24 - BGP. Автономная система. Типы провайдеров. Состояния BGP.	

25 - Функции транспортного уровня стека TCP/IP. Порты. UDP.	89
26 - Протокол TCP. Методы обеспечения надежной доставки.	95
27 - Протокол TCP. Автомат состояния TCP. Ориентация на соединение. Установление и закрытие TCP-соединения.	105
28 - Протокол TCP. Управление потоком. Скользящее окно.	106
29 - QUIC: Особенности, преимущества и недостатки.	110
30 - QUIC: Заголовки.	112
31 - Протоколы прикладного уровня. Модель “клиент-сервер”, Модель P2P. Примеры протоколов прикладного уровня.	114
32 - Протокол HTTP. История от 0.9 до 3.	117
33 - Протокол HTTP. Структура, методы, коды состояний.	121
34 - Протоколы Telnet (SSH) и FTP.	125
35 - Служба DNS, Пространство символьных имен.	128
36 - Протоколы электронной почты, программы Ping, Traceroute/Tracert.	133
37 - Технологии работы с трафиком. Отличие VPN от proxy.	137
38 - История развития VPN.	140
39 - Принципы работы TOR браузеров, преимущества и недостатки.	142
40 - Гонки процессов (состязания). Критические секции.	145
41 - Средства синхронизации процессов. Семафоры, мьютексы.	146
42 - Средства синхронизации процессов. События, таймеры, сигналы.	147
43 - Средства передачи информации между процессами. Каналы, сокет.	148
44 - Средства передачи информации между процессами. Атомы, сообщения, почтовые ящики.	149
45 - Средства передачи информации между процессами. Брокер RabbitMQ.	150
46 - Средства передачи информации между процессами. Брокер Apache Kafka.	153

1 - Методы коммутации. Коммутация каналов и коммутация пакетов. Достоинства и недостатки. Принципы реализации.

Определения

Коммутация – это соединение двух различных узлов/абонентов.

Коммутация – это переключение (замыкание/размыкание) различных участков линий связи/цепи.

Коммутация – одна из важнейших функций передачи данных от одного абонента к другому.

Задачи коммутации

1. Выявление информационных потоков, которые требуют маршрутизации (сетевых потоков пакетов, которыми нужно управлять).
2. Маршрутизация потоков и их передача.
3. Продвижение потоков – распознавание и локальная коммутация на углах коммутации.
4. Мультиплексирование и демультиплексирование потоков.

Выделяют *два типа коммутации*: динамическая и статическая (постоянная).

Динамическая	Статическая
В процессе соединения устанавливается под конкретного пользователя	Устанавливает оператор
Существует только на время сеанса (когда данные передали – пакеты скомутированы, когда ничего не передается – коммутации не происходит)	Устанавливается на длительный срок
Обеспечивает возможность соединения с любым абонентом	Устанавливается между конкретной парой абонентов
Длительность соединения: от нескольких секунд до нескольких часов	Длительность соединения: длительная (месяцы, годы и более)

Структура сети с коммутацией

- Не полносвязная топология, потому что если бы каждый абонент был связан с каждым, то и коммутировать было бы нечего.
- Адресация: чтобы понимать, как скомутировать пакет, нужно понимать, куда ему надо, т.е. надо уметь идентифицировать абонента и понимать, как проложить к нему маршрут.
- Есть транзитные узлы коммутации, т.е. есть конечные узлы, которые общаются, и ряд промежуточных.
- Существуют линии связи (ЛС):
 - индивидуальные абонентские (АЛ) – личная витая пара, телефонная линия, оптический кабель, который непосредственно входит к нам как к абоненту,
 - совместно используемые,
 - ЛС может включать N каналов, т.е. подразделяться на каналы.
- В промежуточных узлах хранится маршрут в виде набора таблиц. В каждом узле есть таблица маршрутизации, которая позволяет осуществить передачу, т.е. коммутацию пакетов.

Типы трафика и методы коммутации

- *Традиционные сети:* радио, телефонные и ТВ сети.
Относительно равномерный поток данных: данные идут с одинаковой скоростью и потоком, т.е. регулярный трафик. Изменение скорости потока происходит в каких-то известных пределах, поэтому чаще всего используется *коммутация каналов*: по верхней и нижней границе скорости можно оценить, какой требуется канал.
- Компьютерные сети.
Неравномерный поток данных, трафик нерегулярный. Применяется *коммутация пакетов*.

Коммутация каналов

Коммутация каналов — образование непрерывного составного физического канала из последовательно соединенных отдельных канальных участков (звеньев).

Эффективна для регулярного трафика.

- Есть процедура установления соединения.
- Адрес важен только на этапе установления соединения, дальше все сообщения переходят по готовому каналу.

- Возможен отказ в соединении, линии бывают “заняты”.
- Очень много соединений — большинство соединений не используются.
- Задержка фиксированная и маленькая.
- Пропускная способность фиксированная.
- Порядок следования данных сохраняется.

Коммутация пакетов

Коммутация пакетов — способ передачи данных в компьютерных сетях, при котором вся информация разбивается на небольшие порции — пакеты.

Эффективна для передачи данных при пульсирующем трафике.

Если скорость поступления данных больше скорости передачи, то накапливается очередь. Чем больше очередь, тем выше задержка. При переполнении очереди происходит потеря пакетов.

Сообщение – это то, что мы хотим передать/принять. Исходный объем информации мы делим на части – *пакеты*, т.к. не можем передать его за один раз из-за технических характеристик сетей.

К каждому пакету добавляется заголовок: в нем содержится служебная информация (адрес, сервис, приоритет и т.д.). Чем больше пакетов, тем больше накладных расходов возникает из-за заголовков, тем больше необходимо передать данных. В разных протоколах заголовки отличаются по длине, но всегда ненулевой длины.

- Каждый пакет обрабатывается независимо.
- Для хранения пакетов в устройствах используется память (буфер).

Недостатки коммутации пакетов:

- передаем лишние данные,
- происходят задержки в источнике и в промежуточных устройствах.

Преимущества КП:

- есть дейтаграммный режим, который позволяет независимо маршрутизировать пакеты, т.е. каждый из передаваемых пакетов

рассматривается как полноценная единица информации, и мы его обрабатываем независимо от других;

- есть коммутаторы пакетов с запоминающими устройствами: можем соединять абонентов с разными скоростями входными и выходными.

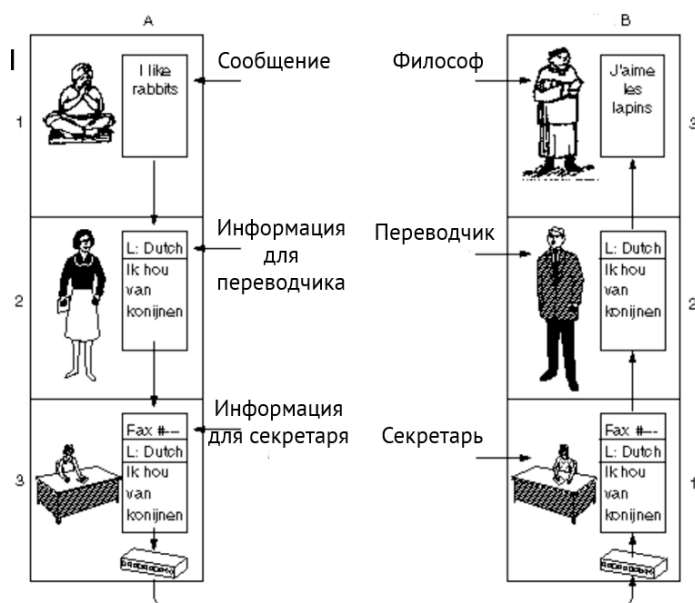
Время ожидания в коммутации **переменное**. Мы не знаем, насколько нагружен коммутатор 😞

2 - Иерархическая организация связи. Декомпозиция, уровни взаимодействия. Эталонная модель ISO/OSI. Стек протоколов.

Иерархическая организация сети

Общий подход:

- разбиение задачи на более простые независимые или частично зависимые подзадачи,
- каждая функция четко определена,
- для каждой подзадачи определены входные и выходные интерфейсы,
- организована координация между подзадачами.



Иерархический подход:

- строгая последовательность: задачи организованы в строгой иерархической структуре, они выполняются последовательно,
- связь только со смежными уровнями.

Эталонная модель ISO/OSI. Стек протоколов

ISO (International Standards Organization) разработала стандартную модель взаимодействия открытых систем — **OSI (Open System Interconnection)**.

Модель OSI (Open Systems Interconnection) — это эталонная модель, разработанная для описания функций телекоммуникационных или вычислительных систем, необходимых для сетевого взаимодействия. Она разделяет процесс сетевого взаимодействия на семь взаимосвязанных уровней. Каждый уровень выполняет специфические функции и взаимодействует с уровнями непосредственно выше и ниже.

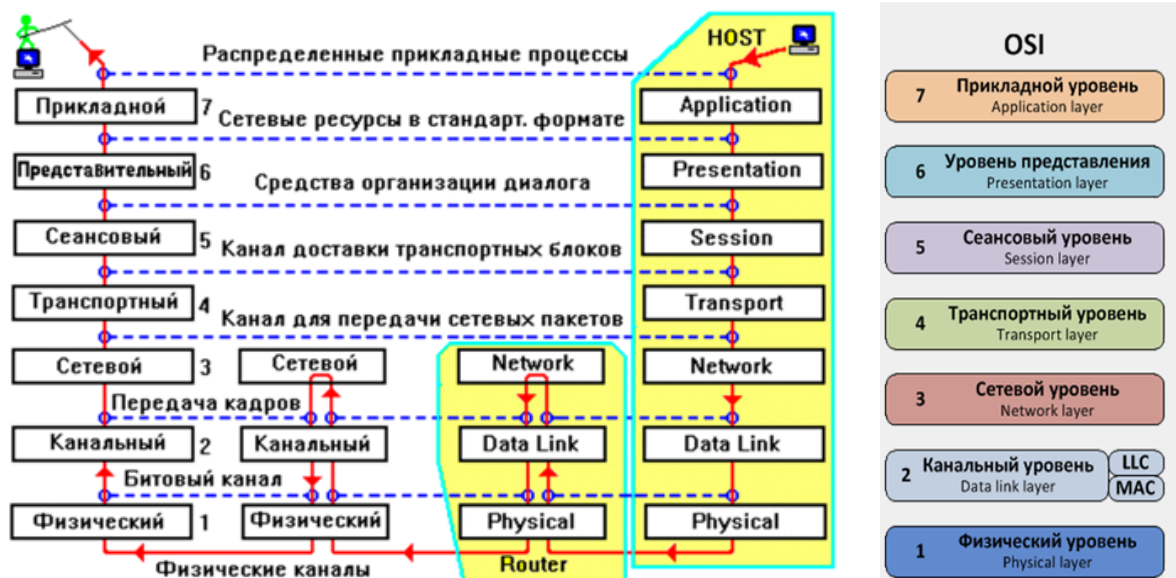
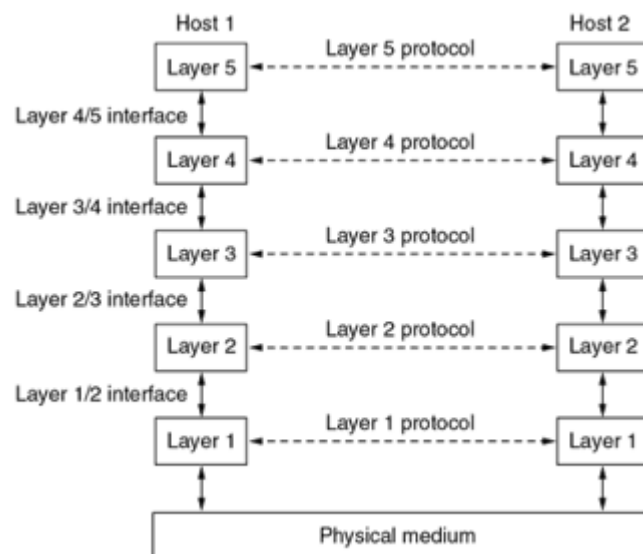
Важно различать *модель OSI* и *стек протоколов OSI*. В то время как модель OSI является *абстрактной концепцией*, стек OSI, построенный на основе модели OSI, представляет собой *набор спецификаций конкретных протоколов*, а также многочисленные программные реализации этих протоколов. В отличие от других стеков протоколов, стек OSI полностью соответствует модели OSI, включая спецификации

протоколов для всех семи уровней взаимодействия, определенных в этой модели.

Функциональные уровни — уровни взаимодействия.

Протоколы — правила взаимодействия уровней (горизонтально).

Интерфейс — правила взаимодействия смежных уровней (вертикально).



Все 7 уровней есть только на хостах. Остальные виды оборудования используют далеко не все уровни.

Физический уровень

- есть во всех устройствах сети, отвечает за физическую реализацию связи между устройствами,
- физическая среда, дальность, скорость, кодирование, синхронизация, уровни сигналов, разъемы: уровень управляет физическими аспектами передачи данных;
- аппаратная реализация (сетевые адаптеры);
- описывается физическими стандартами, например, 10BaseT.

Канальный уровень

Уровень решает проблему того, как устройства получают доступ к общей физической среде, например, локальной сети. Протокол MAC (Media Access Control) определяет правила доступа к среде, чтобы избежать конфликтов при передаче данных.

- управление доступом к среде (MAC),
- обработка ошибок:
 - проверка целостности данных: данные группируются в кадры, проверяется контрольная сумма, производится повторная передача),
- адресация: каждое устройство в сети имеет уникальный адрес (MAC-адрес), который используется для идентификации отправителя и получателя данных,
- управление потоком: контроль скорости передачи данных,
- устройства: хосты (сетевые карты + драйверы), мосты, коммутаторы, маршрутизаторы,
- LAN: Ethernet, Token Ring, FDDI;
- WAN: PPP, LAP-B, "точка-точка".

Сетевой уровень

- для работы составных сетей (объединяет сети):
 - компоненты составной сети – локальные и глобальные сети.
Термин "сеть" на сетевом уровне имеет специфическое значение. Обычно сеть – это типовая топология, стандартный канальный протокол, один администратор;
- внутри сети – канальный уровень, между сетями – сетевой;
- основная задача – маршрутизация;
- адресация: адрес отвечает на вопросы "что?" и "где?" и используется для передачи данных между устройствами, находящимися в разных сетях, поэтому нужно знать номер сети, в котором находится устройство;
- согласование технологий — допустим, одна сеть Ethernet, а другая Token Ring, несмотря на то, что это разные канальные уровни, на сетевом уровне будет произведено согласование технологий и устройства смогут обмениваться данными;
- реализуется на программном уровне ОС или через ПО маршрутизатора,
- уровень одного протокола — IP, остальные (ICMP, ARP, IGMP) – служебные.

Транспортный уровень

- протоколы END-to-END,
- качество передачи,
- управление передачей,
- мультиплексирование,
- реализуется через ПО хоста,
- пример: TCP, UDP, SPX.

Сеансовый уровень

- осуществляет управление диалогом,
- осуществляет установление/разъединение соединения,
- синхронизация (точки отката) – если соединение оборвалось, то сможем подключиться обратно,
- реализуется программно через ПО хоста,
- обычно совмещается с транспортным/прикладным уровнем.

Представительский уровень

- унификация формы представления информации,
- шифрация,
- обычно совмещается с прикладным уровнем,
- пример: SSL.

Прикладной уровень

- доступ к распределенным ресурсам,
- большое число потоков,
- модель “клиент-сервер”,
- HTTP, FTP, NFS, Telnet, DNS...

3 - Иерархическая организация связи. Инкапсуляция, мультиплексирование/демультиплексирование.

Иерархическая организация сети

Сетевое взаимодействие — это сложная задача. Нужно физически передать сигнал, проверить ошибки, выбрать маршрут, гарантировать доставку, открыть приложение и т.д. Если писать одну огромную программу для всего, в ней никто не разберется.

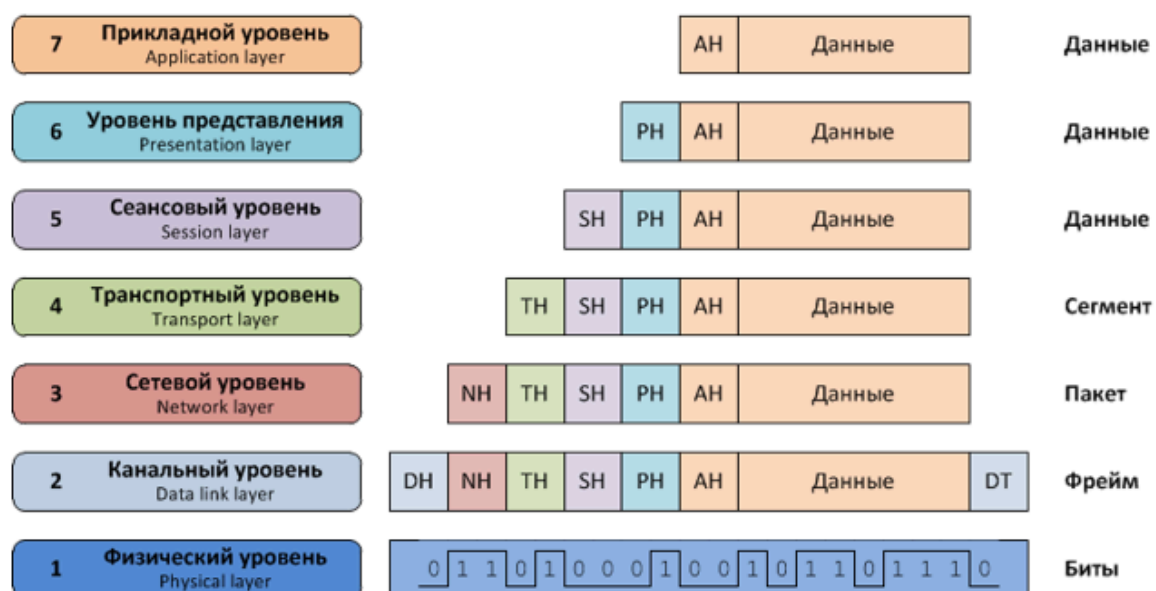
Общий подход:

- разбиение задачи на более простые независимые или частично зависимые подзадачи,
- каждая функция четко определена,
- для каждой подзадачи определены входные и выходные интерфейсы,
- организована координация между подзадачами.

Иерархический подход:

- строгая последовательность: задачи организованы в строгой иерархической структуре, они выполняются последовательно,
- связь только со смежными уровнями.

Инкапсуляция (EnCapsulation – “помещение в капсулу”) — процесс преобразования данных с верхнего уровня в сигналы.



Когда приложение (уровень 7) передает данные вниз, каждый следующий уровень добавляет свой заголовок (*header*).

Каждый заголовок содержит служебную информацию, нужную именно этому уровню.

Пример: имеем некоторые **Данные** (допустим, текст письма);
транспортный уровень добавляет заголовок TH (Transport Header), теперь это **Сегмент**;
сетевой уровень добавляет заголовок NH (Network Header), теперь это Пакет;
канальный уровень добавляет заголовок DH (Data Link Header) и “хвост” DT (Data Trailer), теперь это **Фрейм**;
физический уровень превращает все это в **Биты** для отправки по проводу.

PDU (Protocol Data Unit) — данные (вместе с заголовками), которые переходят с уровня на уровень. Единица данных протокола: сегмент, пакет, фрейм.

PDU уровня N состоит из *Заголовка уровня N + PDU уровня $N-1$* .

Декапсуляция — процесс преобразования сигналов в данные.

Когда данные приходят к получателю (движение снизу вверх по картинке), происходит обратное:

- физический уровень собирает биты во фрейм,
- канальный уровень читает заголовок DH, проверяет ошибки и отбрасывает его, передавая пакет на уровень выше,
- и т.д., пока до приложения не дойдут “чистые” данные.

В реальности существуют нарушения следования уровней.

Мультиплексирование — сбор данных от нескольких источников в один поток для передачи.

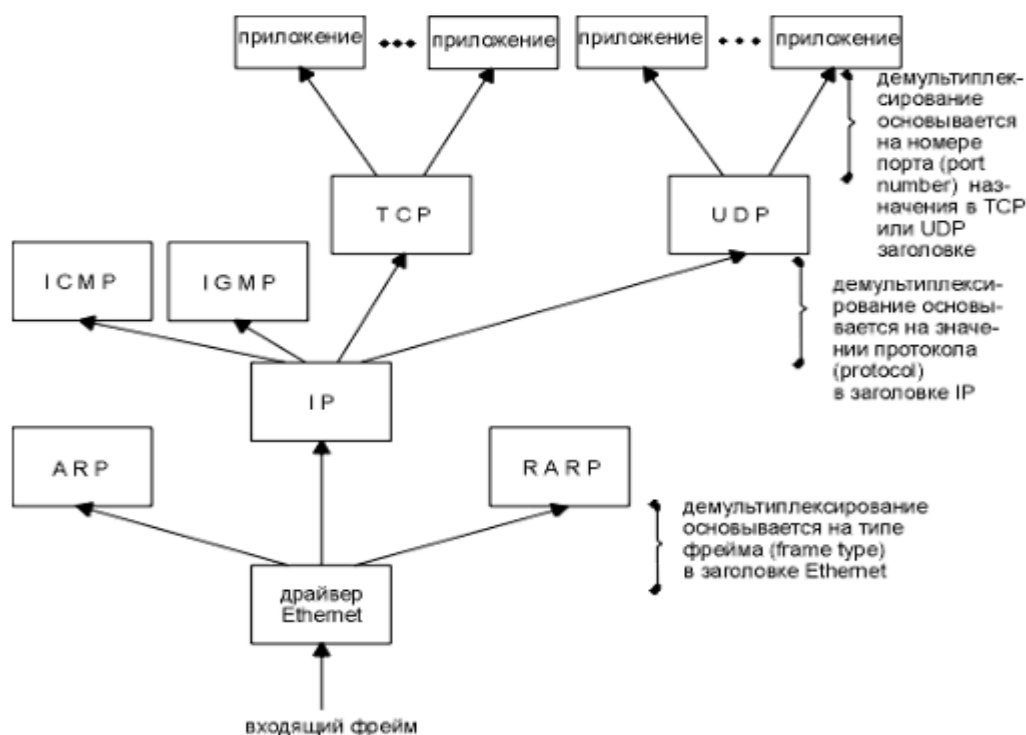
Демультиплексирование — разделение входящего потока данных и доставка их нужным получателям.

Далее по картинке ниже *примеры*:

1. Демультиплексирование на уровне приложений (TCP/UDP)

Допустим на устройстве открыты браузер и Яндекс.Музыка. Чтобы различать поступающие пакеты, в заголовке TCP/UDP есть поле **Port Number**:

- например, браузер слушает порт 50000, а ЯМ слушает порт 50001;
- когда ОС получает данные, она смотрит на номер порта и отдает данные нужной программе.



2. Демультиплексирование на уровне IP (Сетевой)

Поступил какой-то IP-пакет. Внутри может быть надежный поток TCP (веб-сайт), быстрый поток UDP (видеозвонок) или служебное сообщение ICMP. В заголовке IP есть поле **Protocol**:

- если там написано “6”, то внутри TCP,
- если “17”, то UDP,
- если “1” — ICMP.

3. Демультиплексирование на уровне Ethernet (Канальный)

Сетевая карта получила электрический сигнал и собрала из него фрейм. Чтобы понять, что внутри, в заголовке фрейма Ethernet есть поле **Type**:

- код 0x0800 — внутри IP,
- 0x0806 — внутри ARP.

4 - Характеристики каналов и линий связи.

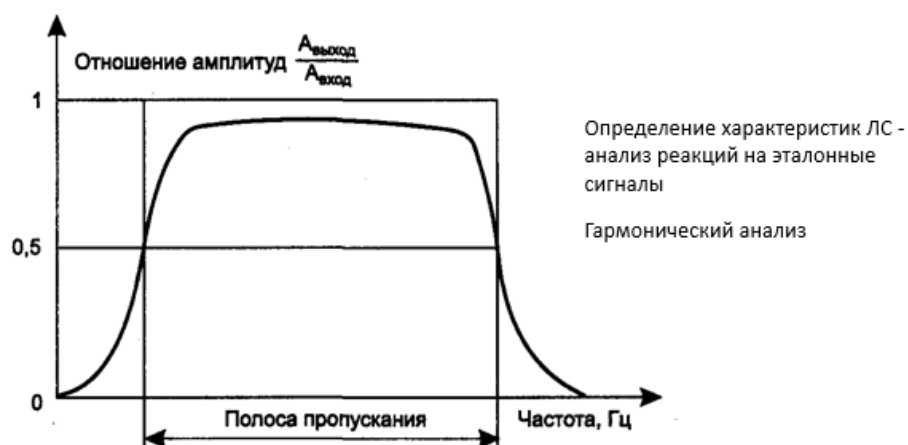
Определение характеристик линий связи

Анализ реакций на эталонные сигналы — основной метод исследования каналов.

В зависимости от используемой кодировки полоса пропускания определяет максимальное количество бит, которое можно передать за единицу времени.

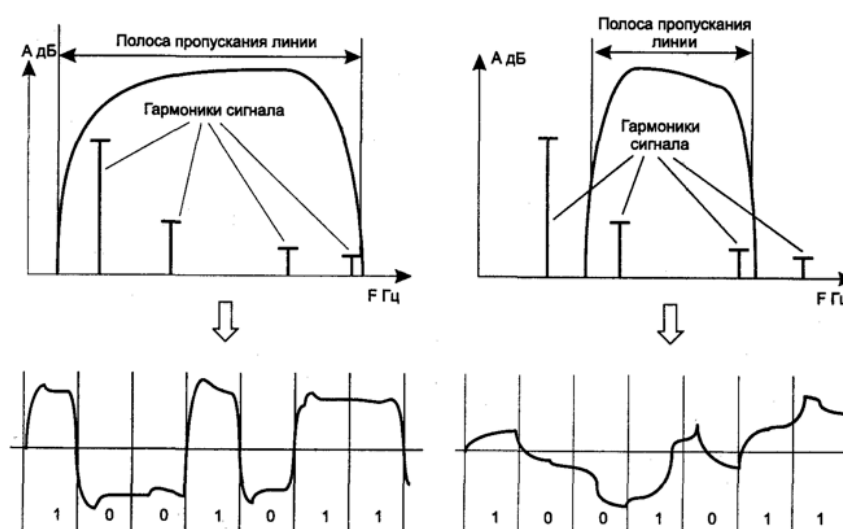
1-й график: АЧХ (амплитудно-частотная характеристика).

Полоса пропускания — диапазон частот, где сигнал проходит с минимальными искажениями.



2-й график: влияние на форму сигнала

- слева: гармоники сигнала попадают в полосу пропускания => сигнал искажается мало;
- справа: часть гармоник обрезаается => сигнал сильно искажается.



Цифровой сигнал состоит из множества гармоник. Если полоса пропускания узкая, высшие гармоники обрезаются, и прямоугольник превращается в плавную кривую.

Полоса пропускания и кодировка

Формула Найквиста:

$$C = 2B \times \log_2(M)$$

где:

- C – максимальная скорость (бит/с),
- B – полоса пропускания (Гц),
- M – количество уровней сигнала.

На практике из-за шумов используется формула Шеннона:

$$C = B \times \log_2(1 + S/N)$$

где S – средняя мощность сигнала в канале, N – средняя мощность аддитивного гауссовского шума в канале.

Задержка

Задержка (latency) — время для передачи пакета до приемника:

- обработка пакета,
- передача по линиям связи – скорость передачи и время формирования пакета в линии связи,
- ожидание в очереди.

Общая задержка = T_{обработки} + T_{передачи} + T_{распространения} + T_{очереди}

1. Обработка пакета:
 - a. Проверка контрольной суммы
 - b. Поиск в таблице маршрутизации
 - c. Принятие решения о следующем прыжке
2. Время передачи: Размер_пакета/Скорость_канала
3. Распространение сигнала: Расстояние/Скорость_сигнала
4. Очередь:
 - a. Зависит от загруженности канала
 - b. Худший случай: буфер переполнен => потеря пакетов.

Пропускная способность

Пропускная способность (Throughput) – фактически достигнутая скорость передачи за единицу времени. В зависимости от размеров пакетов может варьироваться, например, для 10Base-T от 5,48 до 9,76 Мбит/с.

Теоретическая полоса (Bandwidth) – максимальная скорость, заявленная производителем.

Throughput < Bandwidth, так как:

1. Накладные расходы протоколов
Пример: из 1500 байт Ethernet-кадра только ~1460 байт – полезные для TCP.
2. Коллизии и повторные передачи.
3. Управление потоком.
4. АСК-пакеты.

Пример для 10Base-T:

- Теоретически: 10 Мбит/с
- Фактически: 5,48-9,76 Мбит/с
- Зависит от размера пакетов:
 - маленькие пакеты (64 байта) => больше накладных расходов => 5,48 Мбит/с;
 - большие пакеты (1500 байт) => меньше накладных расходов => 9,76 Мбит/с.

Bandwidth-Delay Product

Bandwidth-Delay Product (BDP) – произведение пропускной способности канала на задержку передачи. Результат (обычно в битах или байтах) представляет собой максимальный объем данных, который может одновременно находиться “в пути”.

$$BDP = \text{Bandwidth} * RTT$$

- Для TCP — размер окна/RTT:
 - для 100 Мбит/с и RTT=40 мс: $BDP = 4 * 10^6$ бит ~ 0,5 МБ (размер окна приемника/передатчика);
 - для 10 Гбит/с и RTT=80 мс: $BDP \sim 100$ МБ;
 - при росте буфера растет задержка.

Методы компенсации:

- управление перегрузкой TCP,
- несколько потоков;
- приоритезация трафика для чувствительных приложений.

5 - Канальный уровень. Функции уровня. Принципы определения границ кадра. Методы обнаружения ошибок. MTU

Канальный уровень (англ. **Data Link layer, DLL**), также уровень передачи данных, уровень звена данных — *второй уровень* сетевой модели OSI, предназначенный для передачи данных узлам, находящимся в том же сегменте локальной сети.

Кадр — блок данных, передаваемый между логическими объектами уровня звена данных.

Функции канального уровня

1. Интерфейс с сетевым и физическим уровнем.

Канальный уровень находится между ними; задача канального уровня — взять данные с сетевого уровня и передать на физический.

2. Адресация.

Куда именно доставить, к какому узлу/хосту.

3. Преобразование “Биты $\Leftrightarrow$ Кадры”.

От физического к прикладному; определить, где в потоке бит найти исходные данные.

4. Синхронизация кадров.

Как определить границы последовательности.

5. Обработка ошибок.

- Обнаружение искажений данных.
- Коррекция ошибок (не во всех протоколах).

6. Управление потоком.

- Регулирование скорости передачи.
- Предотвращение переполнения приемного буфера.

7. Управление доступом к среде.

- Для разделяемых сред (Ethernet, Token Ring).
- Методы случайного и детерминированного доступа.

Последние 2 пункта связаны: канальный уровень должен следить, чтобы не было наложений если среда общая, при этом обеспечивать максимально качественную передачу.

В рамках всей сети уровень реализуется через компьютеры, коммутаторы, маршрутизаторы, мосты.

В рамках компьютера (хоста) реализуется через драйвер+сетевую карту.

Особенности реализации

В локальных сетях (LAN):

- доставка кадра между любыми узлами сети,
- топология без петель,
- единый формат адресации,
- подуровни LLC и MAC.

В глобальных сетях (WAN):

- доставка кадра только между соседними узлами,
- связь “точка-точка”,
- адресация не требуется,
- возможны искажения сигналов;
- активно используются методы обработки ошибок;
- протоколы PPP, HDLC.

Протоколы канального уровня обеспечивают синхронизацию битов, байтов, кадров; выявляют, где в потоке информации начинаются полезные данные.

Протоколы характеризуются 2 величинами:

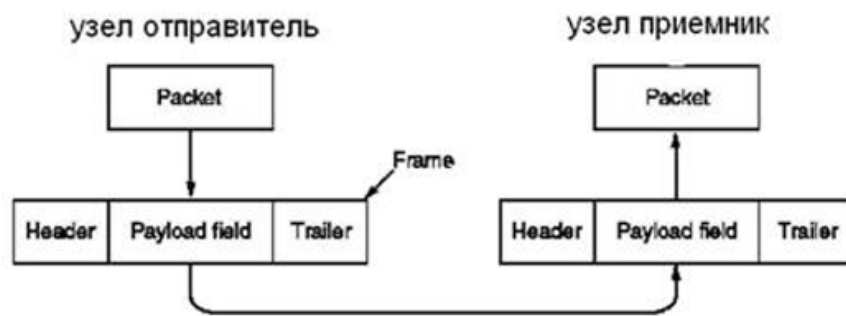
- минимальная длина одного кадра,
- максимальная длина одного кадра.

Бит-ориентированные синхронные протоколы:

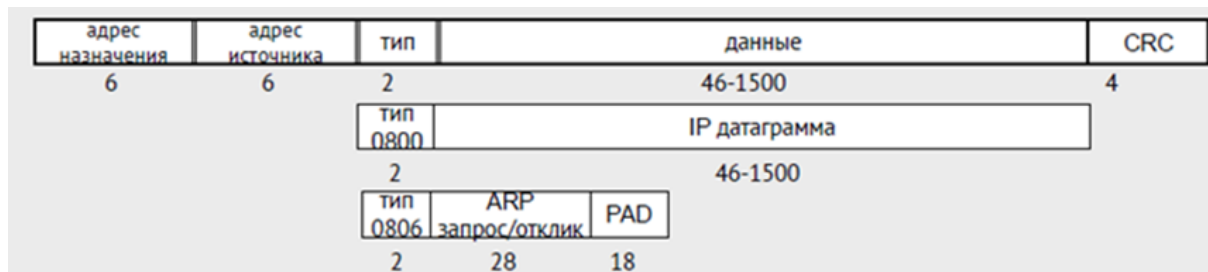
- все биты кадра в непрерывном синхронном потоке, т.е. передаются с одинаковой постоянной частотой;
- скорость, самосинхронизирующиеся коды;
- различное определение границ кадра (в разных протоколах – разные границы кадра).

Принцип работы канального уровня:

1. Пакет от отправителя сетевого уровня попадает на канальный уровень, из пакета превращается в кадр.
Отличительной особенностью Ethernet является то, что помимо заголовка есть еще и хвостовая часть (данные записываются и спереди, и сзади).
2. При поступлении кадра на канальный уровень другого узла откидываются заголовочные и хвостовые части, и канальный уровень передает вверх в сетевой уровень информацию.



В Ethernet: заголовок – адрес назначения (6 байт), адрес источника (6 байт), тип (2 байта), затем идут данные, хвостовая часть – контрольная сумма CRC (4 байта). Перед заголовком также идет синхро-последовательность.



Старшие 3 байта адреса – ID производителя. Индивидуальный (старший бит старшего байта = 0), групповой (старший бит старшего байта = 1), широковещательный (все единицы: 0xffffffff; каждый получатель будет воспринимать пакет как свой).

Заголовок начинается с адреса назначения, потому что Ethernet используется в локальных сетях, и пакет, отправленный от какого-нибудь компьютера, придет каждому в этой локальной сети. Каждый компьютер через XOR сравнит адрес назначения со своим адресом и при совпадении запишет пакет себе в память, иначе проигнорирует и не будет забивать себе память.

Основные 2 типа в Ethernet: 0x0800 (IP пакет), 0x0806 (ARP запрос/отклик).

Поле данных: для IP пакета — все поле от 46 до 1500 байт; в ARP намного меньше (см. картинку выше). Поле данных должно быть минимально 46 байт, поэтому PAD в ARP это мусор для добивания до 46 байт.

MAC-адрес конкретного компьютера существует до маршрутизатора данной локальной сети, при передаче информации дальше — маршрутизатор подставляет свой MAC-адрес.

Принципы определения границ кадра

Проблема: в бит-ориентированных синхронных протоколах все биты кадра передаются непрерывным потоком. Необходимо четко определить, где начинается и заканчивается каждый кадр.

1. Использование флага FLAG с бит-стаффингом.

Принцип: специальная битовая последовательность (флаг) обозначает начало и конец кадра. Стандартный флаг 01111110 (0x7E).

Что делать, если такая последовательность встречается в данных?

Решением служит *бит-стаффинг*: после каждой последовательности из **пяти единиц** вставляется **0**.

Пример:

Исходные данные: 1100011111110011111011101010

После стаффинга: 11000111101110011110011101010

[Флаг 01111110] [Данные со стаффингом] [Флаг 01111110]

На приемной стороне удаляется каждый 0, следующий после пяти 1 и восстанавливаются исходные данные.

Преимущества:

- не требуется поле длины кадра,
- надежное определение границ.

Недостатки:

- увеличение объема передаваемых данных.

2. Использование поля “Длина кадра”

Принцип: в заголовке кадра указывается количество байт/бит в кадре (один стартовый флаг + поле длины).

Структура кадра Ethernet:

[Преамбула] [SoF] [Address Dest] [Address] [Length] [Данные]

7 байт 1 байт 6 байт 6 байт 2 байт 46-1500

Преамбула: 10101010...10 (для синхронизации)

SoF (Start of Frame): 10101011 (маркер начала кадра)

Преимущества:

- простота реализации,
- нет увеличения объема данных.

Недостатки:

- ошибки в поле длины приводят к потере синхронизации,
- необходима проверка корректности длины.

3. Использование запрещенных сигналов

Принцип: используются специальные символы, которые не могут появиться в данных.

Пример:

В Манчестерском коде каждый бит представляется переходом. Определенные последовательности переходов объявляются недопустимыми — эти последовательности используются как маркеры начала/конца кадра.

Преимущества:

- не требует дополнительных битов в данных.

Недостатки:

- применим только к определенным типам кодирования,
- требует специального физического кодирования.

Методы обнаружения ошибок

Основная идея: передача избыточной информации вместе с данными. Приемник проверяет соответствие данных избыточной информации и фиксирует ошибку при несоответствии.

FCS (Frame Check Sequence) — поле в конце кадра с контрольной суммой, которое вычисляется по содержимому кадра.

CRC (Cyclic Redundancy Check) – циклический избыточный код.

Математическая основа:

- двоичное деление полиномов по модулю 2,
- CRC — остаток от деления кадра на образующий многочлен $G(x)$.

Образующий многочлен $G(x)$ — заранее согласованный полином, степень которого $<$ длины кадра.

Стандартные многочлены:

- CRC-16: $G(x) = x^{16} + x^{15} + x^2 + 1$

- CRC-32 (Ethernet): $G(x) = x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x + x^7 + x + x^4 + x^2 + x + 1$

Пример расчета:

$G(x) = x^4 + x + 1 = 10011$ (в двоичном виде)

На передающей стороне:

1. 1101011011 — исходные данные.
2. Добавляем 4 нуля (степень многочлена): 11010110110000
3. Делим на $G(x) = 10011$
4. Остаток 1110 — это CRC
5. Передаваемый кадр — данные + CRC: 11010110111110

На приемной стороне:

1. 11010110111110 — принятый кадр.
2. Делим на тот же многочлен $G(x) = 10011$.
3. Если остаток = 0 => ошибок нет, в противном случае обнаружена ошибка.

MTU (Maximum Transmission Unit)

MTU — максимальный размер поля данных (полезной нагрузки), который может быть передан в одном кадре канального уровня.

Поле данных в кадре может иметь разную длину: от минимальной (46 байт для Ethernet) до максимальной (MTU).

Типовые значения MTU:

- Ethernet — 1500 байт
- PPPoE — 1492 байт
- Token Ring — 4464 байт
- FDDI — 4352 байт
- ATM — 9180 байт
- Jumbo Frames — 9000 байт

Структура кадра Ethernet:

[Заголовок 14 байт] [Данные 46-1500 байт] [CRC 4 байта]

минимальный размер кадра: 64 байта (14 + 46 + 4)

максимальный размер кадра: 1518 байт (14 + 1500 + 4)

Path MTU — минимальный MTU на пути следования пакета. В сети с разнородными технологиями разные сегменты имеют разный MTU.

Фрагментация — разделение на части. Если пакет сетевого уровня больше MTU канального уровня, то выполняется фрагментация на сетевом уровне.

Пример:

IP-пакет: 2000 байт

MTU Ethernet: 1500 байт

Решение:

- Фрагмент 1: 1500 байт

- Фрагмент 2: 500 байт + заголовки

6 - Канальный уровень. Подуровни канального уровня. Технология Ethernet. Формат кадров.

Подуровни канального уровня

Согласно стандартам IEEE 802, канальный уровень в локальных сетях делится на два подуровня: *LLC (Logical Link Control)* и *MAC (Medium Access Control)*.

LLC — управление логическим уровнем.

Функции:

- взаимодействие с сетевым уровнем,
- мультиплексирование протоколов сетевого уровня,
- *опционально*: управление потоком, контроль ошибок.

Типы LLC

Тип	Описание	Применение
LLC1	Без установления соединения и подтверждений	TCP/IP
LLC2	С установлением соединения и подтверждением	Hewlett-Packard, IBM
LLC3	Без установления соединения, но с подтверждением	Промышленные сети

MAC — управление доступом к среде.

Функции:

- управление доступом к разделяемой среде передачи,
- формирование кадров (фреймов),
- физическая адресация (MAC-адреса),
- обнаружение ошибок (CRC).

Методы доступа к среде: случайный доступ (Ethernet CSMA/CD), детерминированный доступ (Token Ring, FDDI).

Технология Ethernet (IEEE 802.3)

Разные типы кадров Ethernet:

Ethernet IEEE 802.3, Ethernet DIX[1] (10 мбит/с), fast ethernet (100 мбит/с), gigabit ethernet (1 гбит/с).

На рис. а) DIX; б) IEEE 802.3

Bytes	8	6	6	2	0-1500	0-46	4	
(a)	Preamble	Destination address	Source address	Type	Data	Pad	Check-sum	
(b)	Preamble	SO F	Destination address	Source address	Length	Data	Pad	Check-sum

Определение типа кадра Ethernet:

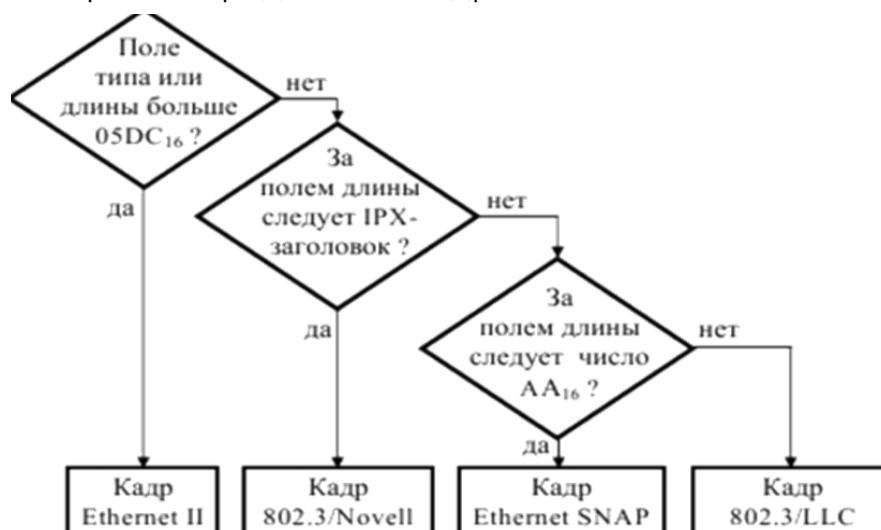
Кадр 802.3/LLC							
6	6	2	1	1	1(2)	46-1497 (1496)	4
DA	SA	L	DSAP	SSAP	Управляющее поле	Данные	FCS
					Заголовок LLC		

Кадр Raw 802.3/Novell 802.3							
6	6	2	46-1500				4
DA	SA	L	Данные				FCS

Кадр Ethernet DIX (II)							
6	6	2	46-1500				4
DA	SA	T	Данные				FCS

Кадр Ethernet SNAP									
6	6	2	1	1	1	3	2	46-1492	4
DA	SA	L	DSAP	SSAP	Управляющее поле	OUI	T	Данные	FCS
			AA	AA	03	000000			
Заголовок LLC						Заголовок SNAP			

Алгоритм определения кадра Ethernet:



Форматы кадров Ethernet

Общая структура кадра:

[Преамбула] [SoF] [Dst MAC] [Src MAC] [Length/Type] [Данные] [CRC]
7 байт 1Б 6Б 6Б 2Б 46-1500Б 4Б

Поля кадра

Поле	Размер	Описание
Преамбула	7 байт	10101010... для синхронизации
SoF	1 байт	10101011 – маркер начала кадра
Destination MAC	6 байт	Адрес получателя
Source MAC	6 байт	Адрес отправителя
Length/Type	2 байта	Длина данных ИЛИ тип протокола
Data (Payload)	46-1500 байт	Полезная нагрузка
CRC (FCS)	4 байта	Контрольная сумма

Типы кадров Ethernet

1. Ethernet II (DIX)

[Dst MAC] [Src MAC] [Type] [Данные] [CRC]
6Б 6Б 2Б 46-1500Б 4Б

Типе указывает протокол верхнего уровня (0x0800 – IP, 0x0806 – ARP, 0x86SS – IPv6).

2. IEEE 802.3 (с LLC)

[Dst MAC] [Src MAC] [Length] [LLC] [Данные] [CRC]
6Б 6Б 2Б 3-8Б 43-1497Б 4Б

Здесь LLC – это заголовок подуровня LLC.

3. IEEE 802.3 SNAP

[Dst MAC] [Src MAC] [Length] [LLC+SNAP] [Данные] [CRC]
6Б 6Б 2Б 8Б 43-1492Б 4Б

SNAP (SubNetwork Access Protocol) для совместимости с Ethernet II.

4. IEEE 802.1Q (VLAN-tagged)

[Dst MAC]	[Src MAC]	[802.1Q Tag]	[Type/Length]	[Данные]	[CRC]
6Б	6Б	4Б	2Б	46-1500Б	4Б

Tag содержит VLAN ID (12 бит) и приоритет (3 бита). Максимальный размер кадра 1522 байта.

Алгоритм определения типа кадра:

Если поле Type/Length ≥ 1536 (0x0600):

 это Ethernet II (Type указывает протокол)

Иначе:

 это IEEE 802.3 (Length указывает размер данных)

здрасьте
добры й вечер
я кто?
гиена
окак, интересно
ну хотя бы не Радионова Полина
были бы друзьями, если б была радионовой полиной, потому что вы
полина радионова
интересное наблюдение
я устала
надо отдохнуть
как раз собираюсь
правильное решение
меня пожурили за малюсенький запрос клодику сегодня, потому что
он добил бедолагу, и теперь нужно ждать, а тот факт, что остальные
95% токенов использовала не я никого не смущает
беспредел да и только
я сегодня на бесплатных запросах в квене живу
я тоже решила что билеты не стоят токенов, но отчетик надо было
сделать
ох чтож делать чтож делать
напиваться конечно
скорее бы бар какойнибудь
даааа хочется очень хочется
Автосогласование – последние 3 строчки этого диалога.
поздравляю мы расписали билет
ураааа можно отдыхать
пойду допишу пару билетов по бд, раз уж никто ими не занимается
а мне бы тут билетик дописать
но тут ужас уже 27 страниц и я только на 8 билете вообще кошмар,
мрак и катастрофа
я полистала и мне страшно стало, не хочу это учить
я вот написала и ниче не выучила адреса какие то айпишники угрозы
угрозы спокойствию душевному явно
100%
мне кажется мы с ума сходим потихоньку, как в dont starve together
когда темно
ааахаххахаха я думаю эт нормально в нашей ситуации
кто-то еще не сошел с ума?

7 - Канальный уровень. Коллизии. Метод CSMA/CD. Автосогласование.

Коллизия — одновременная передача данных двумя или более узлами в одной разделяемой среде передачи.

Почему коллизии возникают?

В Ethernet все узлы подключены к одной физической или логической шине. Нет централизованного регулятора, поэтому два компьютера могут начать передачу одновременно, не зная о намерениях друг друга.

Чтобы отправитель успел зафиксировать коллизию до окончания передачи своего кадра, должно выполняться условие:

1. $T_{min} \geq RTT$
2. $T_{min} \geq PDV$

где **Tmin** – время передачи минимального кадра, **RTT/PDV** – время прохождения сигнала от отправителя до самого удаленного узла и обратно.

Если кадр слишком короткий, отправитель завершит передачу раньше, чем вернется сигнал коллизии, и ошибка останется необнаруженной.

Коллизионный домен — часть сети, где любые два узла могут создавать коллизию. Границы домена определяются коммутаторами/мостами (они не пропускают коллизии дальше).

Метод CSMA/CD

CSMA/CD — Carrier Sense Multiple Access with Collision Detection — Множественный доступ с прослушиванием несущей и обнаружением коллизий.

Алгоритм работы

1. Carrier Sense (прослушивание несущей)
Узел слушает линию. Если линия свободна, то переход к шагу 3. Если занята, то ожидание освобождения.
2. Межкадровый интервал (IPG – Inter Packet Gap)
После обнаружения свободы линии узел ждет 9.6 мкс (для 10 Мбит/с) или 0.96 мкс (для 100 Мбит/с) для восстановления схемы приёмника и синхронизации.
3. Передача кадра

Узел начинает передачу, во время передачи он продолжает слушать линию.

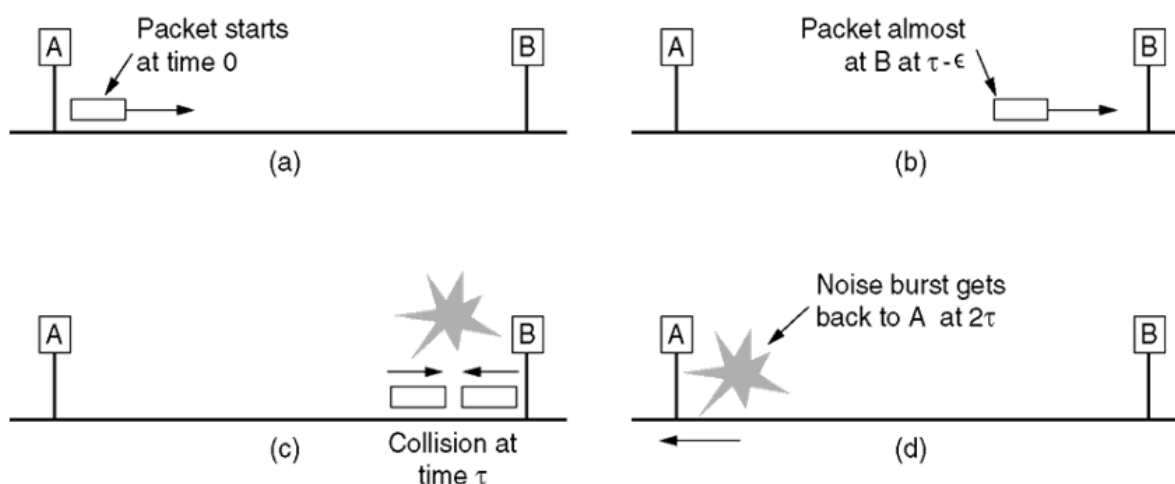
4. Collision Detection (Обнаружение коллизии)

Если напряжение в линии не равняется передаваемому сигналу, то фиксируется коллизия.

5. Действия при коллизии:

- a. Остановка передачи текущего кадра.
- b. Передача ЖАМ-последовательности (32 бита искусственного шума), чтобы гарантировать, что все узлы в домене заметят коллизию.
- c. Случайная задержка (backoff): $L * 512$ битовых интервалов.
 - L – случайное число из диапазона $0 \dots 2^{(N-1)}$
 - $N = \min(\text{номер попытки}, 10)$ (максимум 1024 интервала)
 - после 16 неудачных попыток кадр отбрасывается, ошибка сообщается на уровень выше.
- d. Повторная попытка после задержки.

Какая-то картиночка:



Автосогласование

Автоматический выбор *наивысшего общего режима* работы между двумя устройствами на концах канала.

Механизм работы

Устройства обмениваются Fast Link Pulse (FLP) – специальными импульсами, содержащими битовую карту поддерживаемых режимов. Каждое устройство отправляет свой список возможностей. Алгоритм выбирает режим с наивысшим приоритетом, присутствующий у обоих.

Таблица приоритетов

Приоритет	Режим (старый стандарт)	Приоритет (2018+, IEEE 802.3) 
1	100BASE-TX full-duplex	40GBASE-T full-duplex
2	100BASE-T4	25GBASE-T full-duplex
3	100BASE-TX half-duplex	10GBASE-T full-duplex
4	10BASE-T full-duplex	5GBASE-T / 2.5GBASE-T full-duplex
5	10BASE-T half-duplex	1000BASE-T full/half, 100BASE-TX full/half, 10BASE-T full/half

Автосогласование должно быть включено на обоих концах. Если один порт настроен вручную (например, 100/full), а второй в auto, происходит *duplex mismatch*:

- Полудуплексная сторона ждёт CSMA/CD, полнодуплексная передаёт постоянно.
- Результат: поздние коллизии, ошибки CRC, падение скорости до 1–2 Мбит/с при nominal 100 Мбит/с.

8 - Мосты, коммутаторы. Принципы работы. Достоинства и проблемы использования.

Логическая структуризация сети – разбиение общей разделяемой среды на логические сегменты – самостоятельные сегменты с меньшим числом узлов (мосты, коммутаторы).

Мост — сетевое устройство канального уровня модели OSI, предназначенное для соединения нескольких сегментов локальной сети и фильтрации кадров между ними.

Мост работает на **канальном уровне** (2 уровень OSI) и использует таблицу MAC-адресов.

Мост анализирует MAC-адреса устройств и принимает решение, передавать кадр в другой сегмент сети или нет.

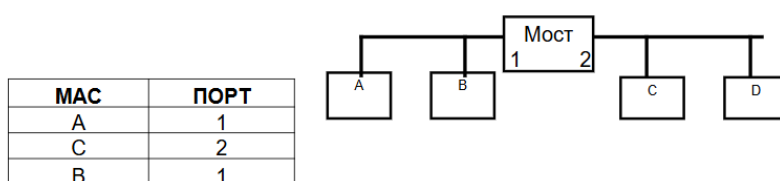
Основная задача моста — уменьшение количества лишнего трафика в сети и разделение сети на отдельные сегменты.

Принцип работы:

1. Мост получает кадр.
2. Запоминает MAC-адрес отправителя и порт, с которого пришел кадр.
3. Анализирует MAC-адрес получателя.
4. Если адрес находится в той же сети, кадр не передается дальше.
5. Если адрес находится в другом сегменте — кадр пересылается.
6. Если адрес неизвестен — кадр рассылается во все сегменты.

Происходит автоматическое (постоянное) заполнение таблицы коммутации.

- Кадры FF-FF-FF-FF-FF-FF передаются на все порты.
- Кадр никогда не передается на порт, от которого этот кадр получен.



Коммутатор — многопортовый мост, предназначенный для соединения устройств в локальной сети.

Принцип работы такой же, как у моста.

Достоинства мостов и коммутаторов

1. Снижение объёма трафика
Кадры передаются только в тот сегмент сети, где находится получатель.
2. Повышение производительности сети
Уменьшается количество ненужных передач и снижается нагрузка на сеть.
3. Уменьшение числа коллизий
Коммутаторы создают отдельный домен коллизий для каждого порта, что повышает эффективность передачи данных.
4. Простота подключения и расширения сети
Позволяют легко добавлять новые устройства без изменения структуры сети.
5. Прозрачность для пользователей и приложений
Не требуют изменения программного обеспечения или настроек конечных устройств.

Проблемы использования мостов и коммутаторов

1. Различие форматов кадров
При соединении разных типов сетей требуется переформатирование кадров, что вызывает дополнительные задержки и усложняет обработку данных.
2. Различие скоростей передачи
При работе сегментов на разных скоростях используется буферизация. При перегрузке возможна потеря кадров из-за переполнения буферов.
3. Проблемы защиты информации
Разные технологии сетей обеспечивают различный уровень безопасности и поддержки шифрования.
4. Проблемы QoS
Не все сети поддерживают механизмы приоритизации трафика, из-за чего возможны задержки при передаче голоса и видео.

Ограничения мостов и коммутаторов

1. Поддержка только топологий без петель
При наличии петель возможна бесконечная циркуляция кадров и перегрузка сети.
2. Слабая изоляция сети
Широковещательный трафик распространяется по всей сети, снижая её производительность.
3. Ограничения по MTU
Корректная работа возможна в основном между сетями с одинаковым размером MTU.
4. Привязка к MAC-адресам
Мосты и коммутаторы работают только на канальном уровне и поддерживают близкие сетевые технологии.
5. Плохая масштабируемость крупных сетей
В больших сетях требуется либо усложнение канального уровня, либо использование сетевого уровня и маршрутизаторов.

9 - Мосты, коммутаторы. Алгоритм покрывающего дерева. 802.1Q. VLAN.

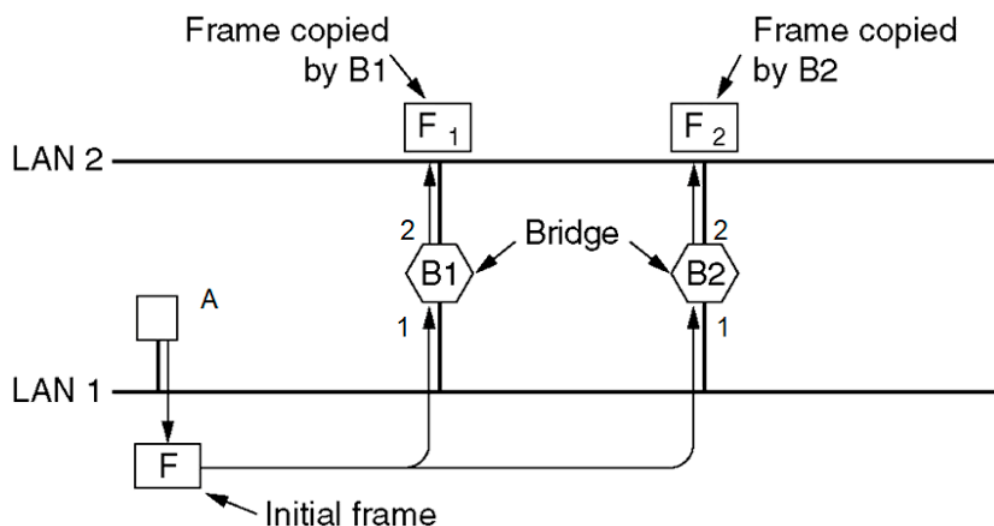
Мост называется *прозрачным*, если он работает незаметно для других устройств в сети, таких как компьютеры и серверы. То есть:

- устройства не знают о его существовании,
- он не изменяет MAC-адреса или данные кадров,

Он просто пересылает кадры между сегментами сети, как будто его нет. Это отличается, например, от маршрутизатора, который явно участвует в коммуникации и изменяет IP-пакеты.

Петли в сети

Два параллельных прозрачных моста:



В сети, где две подсети соединены двумя коммутаторами (B1 и B2), возникает бесконечный цикл перезаписи MAC-таблиц при передаче пакета между подсетями. Происходит следующее:

1. Устройство A (подсеть 1) отправляет кадр в другую подсеть. Пакет попадает в коммутатор B1. B1 записывает в свою таблицу: MAC A → порт 1. Кадр пересылается в коммутатор B2. B2 записывает: MAC A → порт 1.
2. B2 не знает MAC-адреса получателя (так как он (получатель) в другой подсети) → флудит кадр (передает на все порты, кроме порта 1). Кадр возвращается в B1 через другой порт → B2 обновляет запись: MAC A → порт 2.
3. B1 снова получает кадр и обновляет свою таблицу: MAC A → порт 2. И снова посылает кадр в B2 - цикл повторяется.

Spanning Tree Protocol (STP) — алгоритм покрывающего дерева

STP — это сетевой протокол канального уровня, разработанный для предотвращения петель. Основная задача STP — *автоматически* создавать логическую древовидную структуру сети, исключающую циклы, которые могут привести к бесконечной пересылке одних и тех же кадров и, как следствие, к перегрузке и отказу сети.

Принцип работы протокола основан на выборе одного из коммутаторов в сети в качестве «корневого» (Root Bridge). Этот выбор происходит на основе уникального идентификатора, который включает приоритет устройства и его MAC-адрес. После определения корневого коммутатора все остальные устройства вычисляют кратчайшие пути к нему, оценивая стоимость каждого канала передачи данных. На основании этих вычислений для каждого коммутатора выбирается «корневой порт» — порт с наименьшей стоимостью пути к корню. Кроме того, для каждого сегмента сети определяется «назначенный порт», который отвечает за передачу трафика в этом сегменте.

Все остальные порты, которые не входят в оптимальную древовидную структуру, переводятся в заблокированное состояние, чтобы исключить появление петель. При этом заблокированные порты остаются в резерве и могут быть активированы автоматически, если основной путь перестанет работать, что обеспечивает отказоустойчивость сети.

Коммутаторы, поддерживающие STP, регулярно обмениваются специальными сообщениями — BPDU (Bridge Protocol Data Units). Эти сообщения содержат информацию о состоянии портов, идентификаторах коммутаторов и стоимости путей, что позволяет устройствам синхронизировать данные о топологии и своевременно реагировать на её изменения. При обнаружении изменений в сети (например, при подключении или отключении устройства) коммутаторы отправляют уведомления о смене топологии, что запускает процесс перестройки дерева и корректировки блокировок портов.

Кратко:

1. Выбор корневого моста (Root Bridge)
 - Все мосты обмениваются BPDU (Bridge Protocol Data Unit)
 - Выбирается мост с наименьшим Bridge ID
 - Bridge ID = Priority (2 байта) + MAC-адрес (6 байт)

2. Определение корневых портов
 - Каждый мост (кроме корневого) выбирает один корневой порт
 - Это порт с наименьшей стоимостью до корневого моста
3. Выбор назначенных портов (Designated Ports)
 - На каждом сегменте выбирается один назначенный порт
 - Это порт моста с наименьшей стоимостью до корня
4. Блокировка остальных портов
 - Все порты, не ставшие корневыми или назначенными блокируются
 - Состояние: Blocking (не пересылают кадры, но слушают BPDU)

Виртуальные локальные сети (VLAN)

VLAN (Virtual LAN) — логическая подсеть в рамках физической инфраструктуры коммутаторов.

VLAN является решением проблемы *физических сегментов*: логическая группировка пользователей = физической группировке, => чтобы переместить пользователя в другой отдел, нужно переподключить кабель.

VLAN — логическая группировка независимо от физического расположения.

Типы VLAN:

1. Port-based VLAN — группировка по порту
Пример: порты 1-8 => VLAN 10, порты 9-16 => VLAN 20, ...
2. MAC-based VLAN — группировка по MAC-адресам,
3. Protocol-based VLAN — по типу протокола.
4. Subnet-based VLAN — по IP-подсети.

VLAN на *одном* коммутаторе

- все хосты в одной VLAN могут общаться напрямую,
- хосты в разных VLAN не могут общаться без маршрутизатора.

VLAN на *нескольких* коммутаторах

Проблема: как передать трафик конкретной VLAN между коммутаторами?

Вариант 1-й: выделить порт на каждую VLAN => нужно много кабелей



Вариант 2-й: Trunk-порт с тегированием (802.1Q) => один кабель для всех VLAN 😊

Стандарт IEEE 802.1Q

IEEE 802.1Q — стандарт тегирования кадров для передачи VLAN-информации между коммутаторами.

Формат кадра 802.1Q (Tagged Frame):

[Dst MAC 6Б] [Src MAC 6Б] [802.1Q Tag 4Б] [Type 2Б] [Data] [CRC 4Б]

Поле 802.1Q Tag содержит следующее:

- биты 0-15 — TPID (Tag Protocol Identifier): маркер 0x8100
- биты 16-18 — Priority (PCP): приоритет 0-7
- бит 19 — CFI (Canonical Format Indicator): обычно 0
- биты 20-31 — VID (VLAN ID): идентификатор VLAN 0-4095

VID = 0 — приоритетный тег (не VLAN),

VID = 1 — VLAN по умолчанию,

VID = 4095 зарезервирован.

Итого доступно только 4094 (2-4094).

Типы портов в 802.1Q

1. Access Port:

- подключение ключевых устройств (ПК, серверы),
- отправляет тег VLAN при отправке,
- удаляет тег при получении,
- принадлежит одной VLAN.

2. Trunk Port:

- коммутатор-коммутатор или коммутатор-маршрутизатор
- соединение,
- передает данные с тегом.

10 - Беспроводные сети. История развития беспроводных сетей. Характеристики беспроводных сетей различных стандартов.

Беспроводные локальные сети (WLAN — Wireless LAN) — технология передачи данных между устройствами без использования кабельной инфраструктуры на основе радиоволн или инфракрасного излучения.

История развития стандартов IEEE 802.11

Первый базовый стандарт 802.11 (1997)

Базовый 802.11 поддерживал скорость передачи данных 1 или 2 Мбит/с в диапазоне частот 2,4 ГГц и предусматривал два альтернативных метода физического кодирования: метод прямой последовательности с расширением спектра (DSSS — Direct Sequence Spread Spectrum) и метод скачкообразной перестройки частоты (FHSS — Frequency Hopping Spread Spectrum). Оба метода были направлены на повышение помехоустойчивости передачи за счет «размазывания» сигнала по более широкой полосе частот, что позволяло системе сохранять работоспособность даже при наличии узкополосных помех.

Для управления доступом к разделяемой беспроводной среде в стандарте 802.11 был определён метод **CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance)** — множественный доступ с прослушиванием несущей и предотвращением коллизий. В отличие от проводного CSMA/CD, где коллизии обнаруживаются в процессе передачи, в беспроводных сетях устройство не может одновременно передавать и принимать сигнал на одной частоте, поэтому коллизии не обнаруживаются, а предотвращаются. Для этого перед началом передачи узел прослушивает эфир, и если среда свободна, он выжидает случайный интервал времени, после чего отправляет кадр. Дополнительно может использоваться механизм **RTS/CTS (Request to Send / Clear to Send)**, позволяющий решить проблему скрытых узлов, когда два передатчика не «слышат» друг друга, но могут создавать помехи одному и тому же приемнику.

Стандарт 802.11b (1999)

Первым массовым стандартом стал 802.11b. Он работал в диапазоне 2,4–2,4835 ГГц и использовал только метод модуляции DSSS, что упростило реализацию оборудования и снизило его стоимость. Полоса пропускания 22 МГц делилась на 11 каналов с шагом 5 МГц, из которых только три (каналы 1, 6 и 11) не перекрывались по спектру и могли использоваться одновременно в соседних точках доступа без

взаимных помех. Стандарт поддерживал четыре скорости передачи: 1, 2, 5,5 и 11 Мбит/с, причем переключение между ними могло происходить автоматически в зависимости от качества сигнала. Безопасность обеспечивалась протоколом WEP (Wired Equivalent Privacy), который впоследствии был признан криптографически нестойким. Дальность связи в помещении составляла порядка 30–50 метров, на открытом пространстве — до 100 метров. Главным преимуществом 802.11b стала доступность, которая способствовала быстрому распространению беспроводных сетей в домашних и офисных условиях.

Стандарт 802.11a (1999)

Одновременно с 802.11b был утверждён стандарт 802.11a, работавший в диапазоне 5 ГГц. Ключевым отличием этого стандарта стало применение метода ортогонального частотного мультиплексирования (OFDM — Orthogonal Frequency Division Multiplexing), который позволил достичь скорости передачи до 54 Мбит/с. Принцип OFDM заключается в разделении широкого канала на множество узких ортогональных поднесущих, по которым данные передаются параллельно. Это обеспечивает высокую устойчивость к многолучевому распространению сигнала и узкополосным помехам. Диапазон 5 ГГц на момент появления стандарта был значительно менее загружен, чем 2,4 ГГц, что снижало уровень интерференции. Однако 802.11a не получил широкого распространения из-за двух существенных недостатков: несовместимости с оборудованием 802.11b (разные частотные диапазоны) и меньшей дальности действия, поскольку сигналы на более высоких частотах хуже проникают через стены и другие препятствия.

Стандарт 802.11g (2003)

Стандарт 802.11g объединил преимущества предыдущих версий. Он работал в диапазоне 2,4 ГГц, обеспечивая совместимость с массовым парком устройств 802.11b, но при этом использовал модуляцию OFDM для достижения скорости до 54 Мбит/с. Поддерживались как режимы обратной совместимости (когда в одной сети могут работать устройства стандартов b и g), так и «чистый» режим 802.11g, обеспечивающий максимальную производительность. В смешанном режиме производительность всей сети снижалась из-за необходимости использования защитных механизмов, предотвращающих коллизии между устройствами с разными физическими уровнями. Для защиты данных в 802.11g уже мог

применяться протокол WPA (Wi-Fi Protected Access), пришедший на смену небезопасному WEP.

Стандарт 802.11n (2009) и технология MIMO

Стандарт 802.11n стал качественным скачком в развитии беспроводных сетей. Он впервые ввёл поддержку работы в двух диапазонах — 2,4 и 5 ГГц — и возможность использования каналов шириной 40 МГц вместо стандартных 20 МГц, что удвоило потенциальную пропускную способность. Однако главным нововведением стала технология **MIMO (Multiple Input Multiple Output)**, предполагающая использование нескольких передающих и приемных антенн для одновременной передачи нескольких пространственных потоков данных. В конфигурации 4×4 MIMO (четыре передающие и четыре приемные антенны) теоретическая скорость могла достигать 600 Мбит/с (150 Мбит/с на поток). Помимо пространственного мультиплексирования, 802.11n внедрил механизмы агрегации кадров (объединение нескольких кадров канального уровня в один физический кадр) и блочного подтверждения (Block Acknowledgement), что позволило существенно снизить накладные расходы на служебные заголовки и подтверждения.

Стандарт 802.11ac (2014) и технология MU-MIMO

Стандарт 802.11ac развивает идеи 802.11n, но работает исключительно в диапазоне 5 ГГц. Ключевыми улучшениями стали поддержка каналов шириной до 160 МГц, применение модуляции 256-QAM (которая кодирует 8 бит данных на один символ, что на 33% эффективнее 64-QAM) и внедрение технологии MU-MIMO (Multi-User MIMO). Если в обычном MIMO точка доступа в каждый момент времени передаёт данные только одному клиенту, используя все доступные антенны, то MU-MIMO позволяет формировать независимые пространственные потоки для нескольких устройств одновременно. Теоретическая максимальная скорость 802.11ac достигает 6,77 Гбит/с при использовании восьми пространственных потоков и канала 160 МГц.

Безопасность в беспроводных сетях

Вопросы безопасности в беспроводных сетях имеют критическое значение, так как радиосигнал доступен для перехвата любому устройству в зоне покрытия. Первый протокол защиты WEP, предусмотренный в базовом стандарте 802.11, использовал статические ключи шифрования и алгоритм RC4, что делало его уязвимым для атак перебора. Протокол WPA, появившийся как

временное решение, внедрил динамическую смену ключей и алгоритм TKIP, но сохранил совместимость со старым оборудованием. Современный стандарт WPA2, обязательный для сертификации устройств с 2006 года, использует стойкий алгоритм AES-CCMP и обеспечивает надежную защиту при условии использования сложного пароля. В 2018 году был утверждён протокол WPA3, который добавляет защиту от атак перебора методом грубой силы и реализует механизм одновременной аутентификации равных (SAE) для повышения безопасности в открытых сетях.

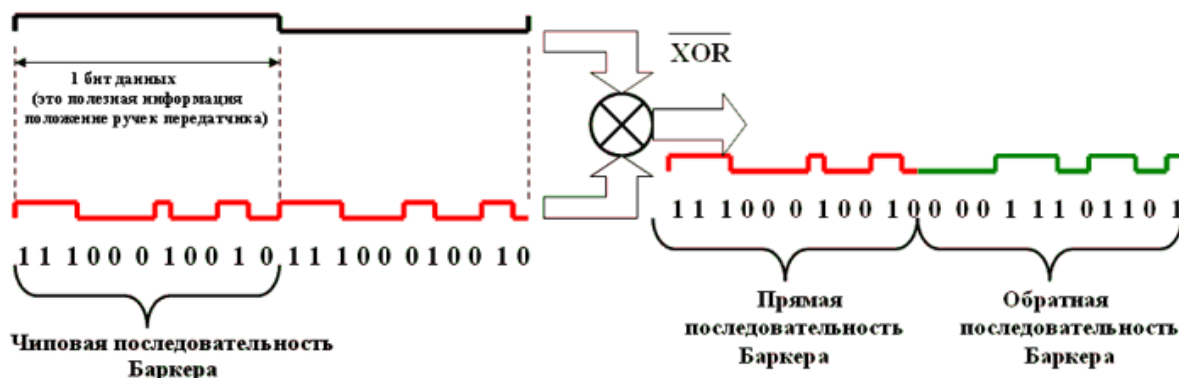
11 - Беспроводные сети. DSSS. MIMO. Фазовая манипуляция. Соотношение скорости соединения и размера кадров.

DSSS (Direct Sequence Spread Spectrum)

Метод прямой последовательности с расширением спектра (DSSS) был заложен в стандарте 802.11 и стал основой для стандарта 802.11b.

Принцип работы DSSS заключается в искусственном расширении занимаемой полосы частот: рабочий диапазон 22 МГц условно разделяется на 11 подканалов, а каждый передаваемый информационный бит преобразуется в 11-битовую чиповую последовательность, которая передается параллельно. На стороне приемника выполняется обратное преобразование путем умножения принятого сигнала на ту же чиповую последовательность, что эквивалентно вычислению автокорреляционной функции.

Такой подход позволяет «размазать» энергию сигнала по широкому диапазону, значительно повышая помехоустойчивость системы и обеспечивая корректное восстановление данных даже при наличии узкополосных помех или многолучевого распространения.



Фазовая манипуляция

Фазовая манипуляция применяется для непосредственного кодирования цифровой информации в радиосигнал в беспроводных сетях. Базовой формой фазовой манипуляции является квадратурная фазовая манипуляция (QPSK).

В отличие от бинарной схемы, передающей один бит за такт, QPSK использует две ортогональные несущие, что позволяет кодировать два бита информации в одном символьном интервале за счет выбора одного из четырех фазовых состояний. По мере развития стандартов фазовая манипуляция эволюционировала в комбинированные

фазо-амплитудные схемы (QAM), позволяющие передавать 4, 6, 8 и более бит на символ.

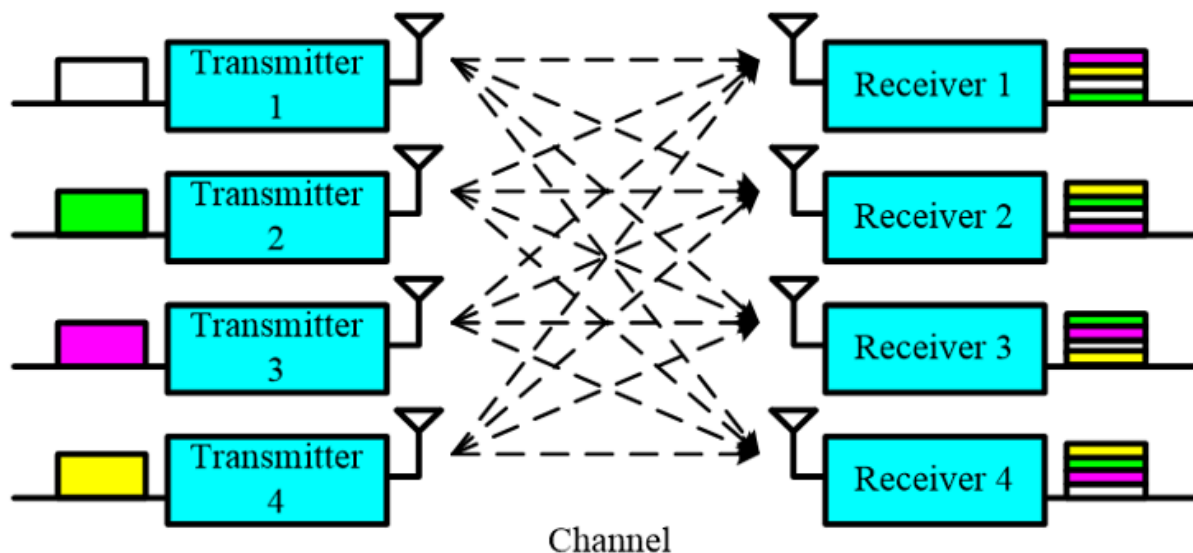
Рост порядка модуляции напрямую увеличивает физическую скорость канала, однако требует пропорционального улучшения отношения сигнал/шум. При ухудшении условий связи беспроводные адаптеры автоматически переключаются на более устойчивые, но менее ёмкие схемы модуляции, обеспечивая адаптивность канала к текущим условиям радиосреды.

Технология MIMO

Суть MIMO заключается в использовании нескольких передающих и приемных антенн для одновременной работы с несколькими пространственными потоками данных на одной частоте.

В конфигурации с четырьмя потоками каждый канал может передавать до 150 Мбит/с, что в сумме обеспечивает теоретическую скорость до 600 Мбит/с.

В стандарте 802.11ac технология была расширена до многопользовательского режима (MU-MIMO) с поддержкой до восьми потоков, что обеспечило скорости до 6,77 Гбит/с в диапазоне 5 ГГц при использовании каналов шириной до 160 МГц.



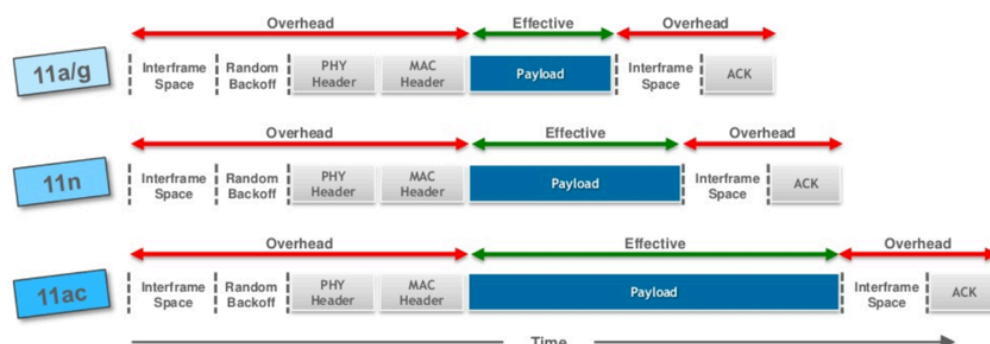
Соотношение скорости соединения и размера кадров

Каждый кадр 802.11 содержит служебные заголовки, состоящие из преамбулы, физического заголовка и полей модуляции, а также требует передачи управляющих пакетов для настройки соединения и

контроля качества связи. При передаче коротких кадров накладные расходы занимают значительную долю времени эфира, что снижает полезную пропускную способность независимо от физической скорости модуляции.

Для решения проблемы в стандартах 802.11n и 802.11ac внедрен механизм агрегации данных, позволяющий объединять несколько кадров канального уровня в один крупный физический блок размером от 8 КБ до 1 МБ.

Дополнительно эффективность повышается за счет блочного подтверждения (Block Acknowledgement), реализованного по стандарту 802.11e, при котором один служебный кадр подтверждает доставку целой группы агрегированных кадров, устраняя необходимость отправки отдельного подтверждения после каждого фрагмента. Таким образом, увеличение размера пакетов улучшает соотношение полезных данных к служебным заголовкам, позволяя сети приближаться к теоретическим показателям скорости соединения.



PHY	Aggregation	Max Bytes (Layer 2)	Max Bytes (Layer 1)
11b	N (fragmentation often used)	2,304	~2,336
11a/g	N	2,304	~2,336
11n	Y	7,935	65,535
11ac	Y	11,454	1,048,575

Увеличение размера пакетов улучшает соотношения полезных данных к «накладным расходам»

12 - Функции сетевого уровня. Составная сеть. Адресация, маршрутизация, маршрутизаторы. Типовая таблица маршрутизации.

Сетевой уровень — третий уровень модели OSI, предназначенный для передачи данных между различными сетями.

Основные функции:

Функция	Описание
Маршрутизация	Определение оптимального пути передачи пакета от источника к получателю через составную сеть
Логическая адресация	Присвоение уникальных IP-адресов интерфейсам узлов (IPv4/IPv6)
Фрагментация	Разделение больших пакетов на фрагменты при превышении MTU канала
Перенаправление пакетов	Инкапсуляция/декапсуляция пакетов в кадры канального уровня разных технологий
Контроль перегрузок	Управление трафиком для избежания заторов в сети

Составная сеть — это совокупность подсетей (S1, S2, S3...), соединенных маршрутизаторами (R1, R2...):

- Подсети могут использовать разные технологии канального уровня (Ethernet, Token Ring, Frame Relay)
- Сетевая адресация единая для всей составной сети
- MAC-адреса меняются в каждой подсети, IP-адреса остаются неизменными

Адресация:

- Каждый порт маршрутизатора имеет **свой IP-адрес** в каждой подсети
- IP-адрес однозначно идентифицирует интерфейс в составной сети
- При конфликте IP-адресов возникает нестабильность сети и потеря пакетов

Маршрутизация:

- Маршрутизатор анализирует **IP-адрес назначения** в пакете
- По таблице маршрутизации определяет порт и следующий маршрутизатор
- Решение принимается на основе **метрики** (критерия оптимальности)

Маршрутизатор — устройство сетевого уровня, соединяющее как минимум две сети.

Характеристика	Описание
Порты	Минимум 2, каждый в своей подсети
Адреса	Каждый порт имеет IP-адрес и MAC-адрес своей подсети
Функция	Декапсулирует пакет → анализирует IP → пересылает на нужный порт
Протоколы	RIP, OSPF, BGP для обмена маршрутами

Таблица маршрутизации — база данных на маршрутизаторе, описывающая соответствие между адресами назначения и интерфейсами.

Поле	Описание	Пример
Сеть назначения	Адрес сети + маска	192.168.1.0/24
Следующий хоп (Next Hop)	IP-адрес следующего маршрутизатора	10.0.0.2
Интерфейс	Порт маршрутизатора для отправки	eth0

Метрика	Расстояние/приоритет маршрута	10
Административное расстояние	Степень доверия к источнику маршрута	120 (RIP)

Порядок просмотра таблиц маршрутизации:

1. Извлечение IP-адреса (IPd) из пакета
2. Поиск специфического маршрута
3. Просмотр каждой строки таблицы: Маска (из строки) накладывается на адрес (IPd & M) . Результат сравнивается с полем строки «Адрес назначения».
 - Если совпадение — строка помечается
 - Если нет — просмотр следующей строки - и так до конца таблицы
4. Действие с пакетом:
 - Нет маршрута - отброс пакета
 - Есть совпадение - отправка пакета
 - Несколько совпадений (чаще всего, если маршрут 0.0.0.0) - отправка пакета по нескольким маршрутам

Пример таблицы маршрутизации с масками:

Таблица маршрутизатора R2 в сети с масками одинаковой длины

Адрес назначения	Маска	Адрес следующего маршрутизатора	Адрес порта	Расстояние
129.44.0.0	255.255.192.0	129.44.0.1	129.44.0.1	Подключена
129.44.64.0	255.255.192.0	129.44.64.7	129.44.64.7	Подключена
129.44.128.0	255.255.192.0	129.44.128.5	129.44.128.5	Подключена
129.44.192.0	255.255.192.0	129.44.192.1	129.44.192.1	Подключена
0.0.0.0	0.0.0.0	129.44.192.2	129.44.192.1	—
129.44.128.15	255.255.255.255	129.44.64.8	129.44.64.7	—

Пример: IPd = 129.44.78.200 -
 1) 129.44.128.15 - спец. маршрут
 2) 1 строка - нет
 3) 2 строка - есть маршрут

Расстояние: подключена 👍

Ключевые протоколы сетевого уровня

- **IP (IPv4/IPv6)** — основной протокол адресации и маршрутизации
- **ARP** — разрешение IP в MAC-адрес (я так понимаю сюда же RARP – это то же самое, но в другую сторону)
- **ICMP** — диагностика (ping, tracer) (тут еще ICMPv6 для IPv6)
- **OSPF, RIP, BGP** — протоколы маршрутизации

Реальный пример (из презы) – команда route print

IPv4 таблица маршрута

```
=====
Активные маршруты:
Сетевой адрес      Маска сети      Адрес шлюза      Интерфейс      Метрика
0.0.0.0            0.0.0.0         195.208.112.30   195.208.112.25 25
127.0.0.0          255.0.0.0       On-link          127.0.0.1      306
127.0.0.1          255.255.255.255 On-link          127.0.0.1      306
127.255.255.255    255.255.255.255 On-link          127.0.0.1      306
195.208.112.0      255.255.255.224 On-link          195.208.112.25 276
195.208.112.25     255.255.255.255 On-link          195.208.112.25 276
195.208.112.31     255.255.255.255 On-link          195.208.112.25 276
224.0.0.0          240.0.0.0       On-link          127.0.0.1      306
224.0.0.0          240.0.0.0       On-link          195.208.112.25 276
255.255.255.255    255.255.255.255 On-link          127.0.0.1      306
255.255.255.255    255.255.255.255 On-link          195.208.112.25 276
=====
Постоянные маршруты:
Сетевой адрес      Маска      Адрес шлюза      Метрика
0.0.0.0            0.0.0.0      195.208.112.30   5
=====
```

Постоянные маршруты сохраняются после перезагрузки (а так эта таблица динамически строится, если мы используем OSPF, например).

А так вообще эту таблицу можно заполнять вручную, при динамической маршрутизации маршрутизаторы обмениваются своими таблицами по протоколам (RIP, OSPF, BGP) и обновляют таблицу. Второе используется чаще в больших сетях с часто меняющимися топологиями.

13 - Адресация в сетях TCP/IP. Классы сетей. Структуризация IP-сетей с помощью масок. Бесклассовая адресация.

В стеке TCP/IP используются **три типа адресов**:

Тип адреса	Где используется	Пример
Локальный (аппаратный/MAC)	Канальный уровень	00:1A:2B:3C:4D:5E
IP-адрес	Сетевой уровень	192.168.1.10
Символьное доменное имя	Прикладной уровень	google.com

IP -> MAC (ARP), MAC -> IP (RARP), domain -> IP (DNS).

IP-адрес — 32-битное число (IPv4), состоящее из адреса сети и адреса хоста. Записывается в десятичном формате с точками: 192.168.1.10.

Фан-факт, который зачем-то мулюха поставил в презу: в целом можно разделять IP на группы по 3, 2 или вовсе не разделять. То есть 100.500 это IP в принципе (и это тоже 1677722100).

Классы сетей (классовая адресация)

Изначально IP-адреса делились на **5 классов** по первым битам адреса:

Класс	Первые биты	Диапазон первого байта	Маска по умолчанию	Число сетей	Число хостов в сети	Назначение
A	0	1–127	255.0.0.0 (/8)	128	16 777 216	Крупные организации
B	10	128–191	255.255.0.0 (/16)	16 384	65 536	Средние организации

C	110	192–223	255.255.255.0 (/24)	2 097 152	254	Малые сети
D	1110	224–239	—	—	—	Мультивещание (multicast)
E	1111	240–255	—	—	—	Зарезервировано

Если что по сути в этой системе маски не использовались, как я понял, они здесь для наглядности.

То же самое картинкой:



Структуризация IP-сетей с помощью масок

Маска подсети — 32-битное число, показывающее, какие биты относятся к **сети**, а какие к **хосту**:

Маска (десятичная)	Маска (бинарная)	Префикс	Объяснение
--------------------	------------------	---------	------------

255.0.0.0	11111111.00000000.00000000 0.00000000	/8	8 бит сеть, 24 бита хост
255.255.0.0	11111111.11111111.00000000 0.00000000	/16	16 бит сеть, 16 бит хост
255.255.255.0	11111111.11111111.11111111 1.00000000	/24	24 бита сеть, 8 бит хост

Короче, чтобы выделить адрес сети просто делаем AND с маской. Если хотим адрес хоста – AND с $\sim mask$.

И еще: в любой сети будет на 2 адреса меньше из-за самого адреса сети и широковещательного адреса. Поэтому самая большая маска /30 – будет 2 адреса на хосты. Минимальная маска /8 – потому что для меньшей маски будет слишком много хостов (>16млн.), что не дает эффективно маршрутизировать такие сети.

Также есть выделенные адреса: 127.0.0.1 (наш любимый localhost) – это loopback адрес, то есть по нему вы обращаетесь к своему же ПК; 255.255.255.255 – это широковещательный адрес для всех устройств сети (но за сеть такой запрос не выходит).

Проблема: идея оказалась херней, и неэффективным распределением подсетей (из-за жестких размеров сетей по классам, а также из-за того, что часть битов уходило на определение класса). В итоге адреса начали заканчиваться. Как мы потом поймем они все равно закончились, но тогда решением стали маски вместо классов.

CIDR (Classless Inter-Domain Routing) — метод IP-адресации, отменивший жёсткие классы.

Преимущества CIDR:

1. **Гибкое управление пространством адресов** — не нужно выбирать между классом А (16 млн хостов) или С (254 хоста)
2. **VLSM (Variable Length Subnet Mask)** — можно создавать подсети с разной длиной маски
3. **Агрегация маршрутов** — уменьшение таблиц маршрутизации в интернете

Число после слэша = количество единичных бит в маске.

14 - Протокол IP. Основные функции. Формат IP-пакета. Фрагментация.

IP — протокол сетевого уровня (L3) стека TCP/IP, обеспечивающий бесконтактную доставку дейтаграмм между хостами через соединенные сети.

Основные функции (первые 3 указаны в стандарте **RFC-791**):

Функция	Описание
Адресация хостов	Универсальная адресация хостов в Internet — каждый хост имеет уникальный IP-адрес
Маршрутизация дейтаграмм	Одна из основных задач — маршрутизация дейтаграмм по множеству соединенных сетей
Фрагментация	Разбивает большие пакеты на фрагменты при превышении MTU канала
Безустановочная передача	IP не устанавливает соединение — каждый пакет передается независимо
Ненадежная доставка	Не гарантирует доставку — не отправляет подтверждения, не восстанавливает потерянные пакеты

Важно: Надёжность обеспечивает **TCP** (транспортный уровень), а не IP.

IP-пакет называется **дейтаграммой** и состоит из заголовка (20 - 60 байт) и данных.

А и еще, протокол IP не гарантирует вообще ничего: у вас могут приходить сообщения в другом порядке, в другом количестве, не приходить, могут приходить но не целыми. Решение всех этих проблем обычно возлагают на плечи TCP.

Фрагментация IP-пакетов

Фрагментация — процесс разбиения больших IP-пакетов на более мелкие фрагменты для прохождения через сети с различными MTU.

Фрагментация происходит, если IP-пакет превышает **MTU** сетевого сегмента (MTU — максимальный размер пакета, который может быть передан без разбиения) (*просто невероятное определение*)).

Маршрутизатор дробит сообщение на максимально допустимые размеры, получатель потом это собирает (на основе заголовков).

Если хоть один фрагмент будет потерян, придется заново передавать все фрагменты.

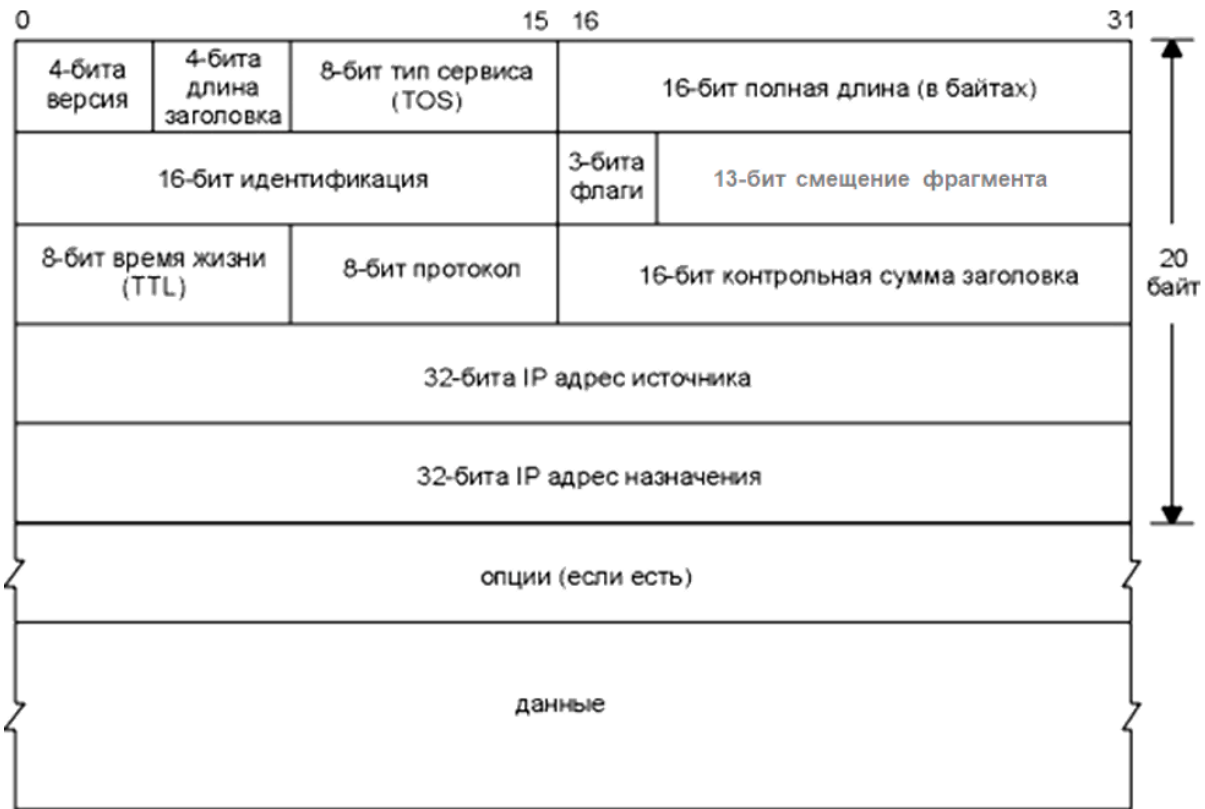
Original IP Datagram

Sequence	Identifier	Total Length	DF May / Don't	MF Last / More	Fragment Offset
0	345	5140	0	0	0

IP Fragments (Ethernet)

Sequence	Identifier	Total Length	DF May / Don't	MF Last / More	Fragment Offset
0-0	345	1500	0	1	0
0-1	345	1500	0	1	185
0-2	345	1500	0	1	370
0-3	345	700	0	0	555

Структура IPv4 пакета



Поле протокол указывает какому протоколу верхнего уровня принадлежат данные IP-пакета (TCP, UDP, ICMP, OSFP). Технически максимальный размер пакета IPv4 **65 535 байт**, однако на практике такие размеры не используются, так как на канальном уровне размер кадра ограничен MTU (для Ethernet это 1500 байт), а фрагментация сильно снижает производительность.

15 - Взаимное отображение физических адресов и IP-адресов.

Направление	Протокол	Описание
IP → MAC (прямое)	ARP (Address Resolution Protocol)	Определение физического адреса по IP-адресу
MAC → IP (обратное)	RARP (Reverse ARP)	Определение IP-адреса по известному MAC-адресу

ARP – Address Resolution Protocol (IP → MAC)

ARP – базовый протокол сетевого уровня для динамического преобразования IP-адресов в уникальные физические адреса хостов локальных сетей.

Узел X хочет связаться с узлом Y в той же LAN:

1. X рассылает ARP-запрос всем узлам сети (broadcast)
→ В запросе: IP и MAC узла X, IP узла Y
2. Узел Y узнаёт свой IP-адрес → формирует ARP-ответ
→ В ответе: IP и MAC узла Y
3. X получает ответ → помещает запись в кэш ARP
4. Теперь X может отправлять пакеты напрямую Y

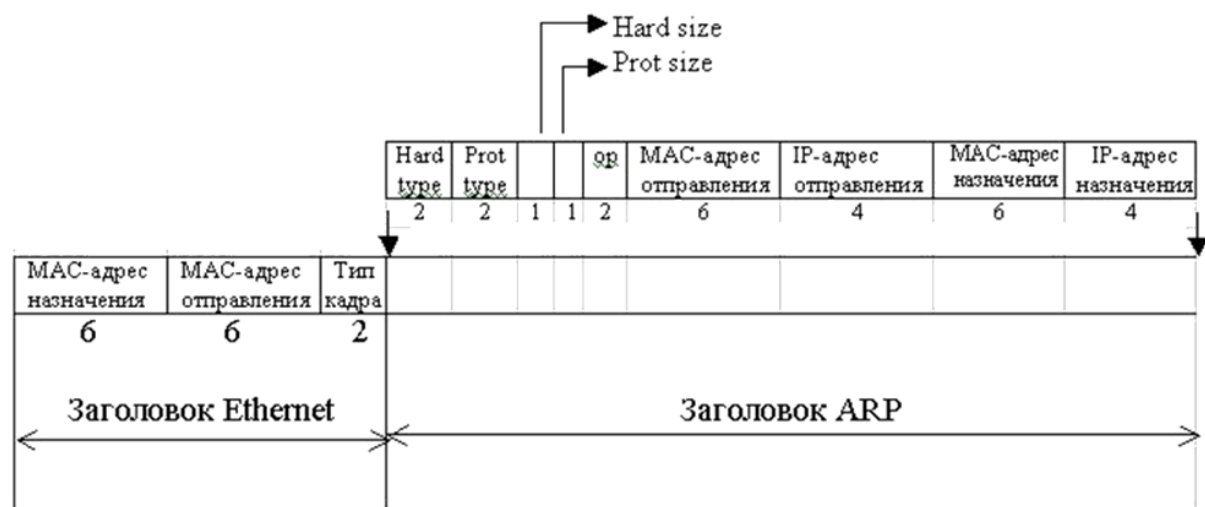
Кэш ARP — таблица преобразований, обновляемая автоматически:

- Полные записи хранятся **20 минут**
- Неполные записи хранятся **3 минуты**
- После удаления записи из кэша требуется новый ARP-запрос

ARP работает по-разному в зависимости от протокола канального уровня:

Тип сети	Протокол канального уровня	Работа ARP
Локальная сеть (LAN)	Ethernet, Token Ring	Прямое ARP-запрос/ответ
Глобальная сеть (WAN)	x.25, Frame Relay	Специфичная для технологии

Пакет ARP / RARP (Размер пакета ARP/RARP **28 байт** (для Ethernet IPv4)):



Hard size - размер аппаратного адреса
Prot size - размер адреса протокола
Hard type – тип аппаратного адреса
Prot type – тип адреса протокола
op – код операции (1- запрос, 2 – ответ)
op – 3 и 4 (я хз как там вообще 4 может быть) это запрос/ответ для RARP

Поле	Значение в ARP-запросе	Значение в ARP-ответе
HTYPE	1 (Ethernet)	1 (Ethernet)
PTYPE	0x0800 (IPv4)	0x0800 (IPv4)
HLEN	6	6
PLEN	4	4
OP	1	2
SHA	MAC-адрес отправителя	MAC-адрес целевого хоста, чей IP-адрес искали
SPA	IP-адрес отправителя	IP-адрес целевого хоста
THA	Неизвестно — заполняется нулями (00:00:00:00:00:00), так	MAC-адрес отправителя запроса того, кто спрашивал

	как не знаем MAC-адрес получателя	
ТРА	Искомый IP-адрес — IP целевого хоста	IP-адрес отправителя запроса

Топ-5 фактов (ака ключевые особенности):

Характеристика	Описание
Автоматизм	Таблицы ARP обслуживаются ядром ОС, пользователи не имеют прямого доступа
Динамичность	Записи обновляются автоматически при получении ARP-запросов и ответов
Кэширование	Избегаются повторные запросы для ранее известных адресов
Локальность	ARP работает только в пределах локальной сети (LAN)

16 - Протокол ICMP.

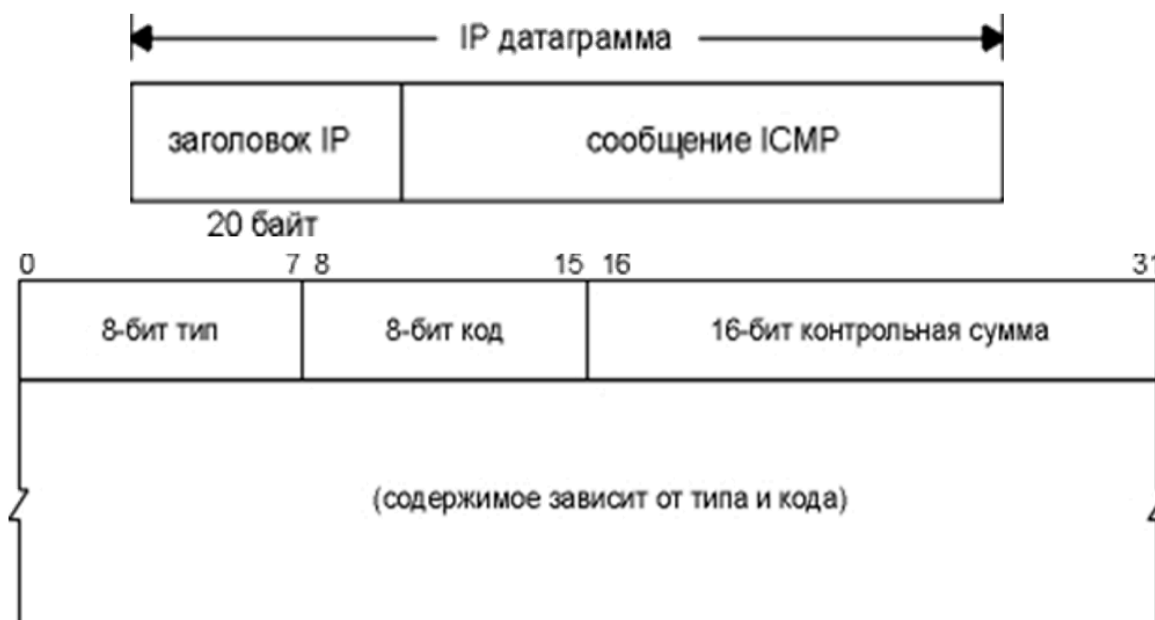
ICMP (Internet Control Message Protocol) – протокол сетевого уровня (L3 модели OSI), используемый для передачи диагностических и информационных сообщений между устройствами в сети TCP/IP.

ICMP не передает пользовательские данные – он помогает устройствам сообщать о проблемах и передавать информацию о статусе соединений:

Характеристика	Описание
Работает поверх IP	Сообщения ICMP инкапсулируются в IP-пакеты
Не для данных пользователя	Передаёт только служебную информацию
Сигнализация об ошибках	Уведомляет отправителя о проблемах доставки пакетов
Диагностика	Используется для проверки доступности узлов и маршрутов

Используется в утилитах *ping* и *tracert/traceroute*.

Структура ICMP-сообщения



Ну вроде не сложно

Украл из презы (типы и коды сообщений ICMP)

Тип	Код	Описание	Запрос	Ошибка
0	0	Эхо ответ	X	
3	0	Сеть недоступна		X
	1	Хост недоступен		X
	2	Протокол недоступен		X
	3	Порт недоступен		X
	4	Необходима фрагментация, которая запрещена		X
4	0	Останов источника (устарел)		
5	X	<u>Перенаправление</u>		
8	0	Эхо запрос		
9	0	Объявление маршрутизатора		
10	0	Запрос маршрутизатора		
11	0	Время истекло (		

Время истекло это $TTL = 0$, это используем в *tracert*.

Угрозы (вставил только из-за смурфа)

Атака	Описание
ICMP Flood (DoS)	Огромное количество ping-запросов с разных источников на хост назначения
Smurf Attack	Использование широковещательных ICMP-запросов для DDoS-атаки

17 - IPv6. Формат заголовка. Расширенные заголовки. Сравнение IPv4 и IPv6.

погнал из лекций:

IPv6

Протокол IPv6 — это новая версия протокола IP, разработанная в качестве преемника IPv4, которая призвана решить проблему исчерпания адресного пространства. Основными отличиями протокола IPv6 от IPv4 являются:

- Больше адресное пространство
- Иерархическое назначение индивидуальных адресов
- Расширенная поддержка групповых адресов
- Автоконфигурация
- Новый формат дейтаграммы
- Поддержка качества обслуживания (QoS).
- Поддержка безопасности

Формат заголовка

Фиксированный заголовок IPv6 имеет строго определенную структуру и всегда состоит из 40 байт. Он включает в себя следующие поля:

- **Версия (Version):** Занимает 4 бита, для IPv6 значение этого поля всегда равно 6.
- **Класс трафика (Traffic Class):** Занимает 8 бит и представляет собой поле приоритета пакета.
- **Метка потока (Flow Label):** Занимает 20 бит. Используется отправителем для обозначения последовательности пакетов, которые должны быть подвергнуты определенной обработке маршрутизаторами.
- **Размер поля данных (Payload Length):** Занимает 16 бит. Это число, указывающее длину поля данных, идущего за заголовком пакета, причем с учетом всех расширенных заголовков.
- **Следующий заголовок (Next Header):** Занимает 8 бит. Задаёт тип расширенного заголовка IPv6, который следует сразу за фиксированным заголовком.
- **Предельное число шагов (Hop Limit):** Занимает 8 бит. Значение этого поля уменьшается на 1 каждым маршрутизатором, через

который передается пакет. При достижении значения, равного 0, пакет отбрасывается.

- **Адрес источника (Source Address):** 128-битный адрес отправителя пакета.
- **Адрес назначения (Destination Address):** 128-битный адрес получателя пакета.

Версия (4 бита)	Класс трафика (8 бит)	Метка потока (20 бит)	
Размер поля данных (16 бит)		Следующий заголовок (8 бит)	Предельное число шагов (8 бит)
Адрес источника (128 бит)			
Адрес назначения (128 бит)			

Расширенные заголовки

Расширенные заголовки используются для поддержки механизмов безопасности, фрагментации и сетевого управления.

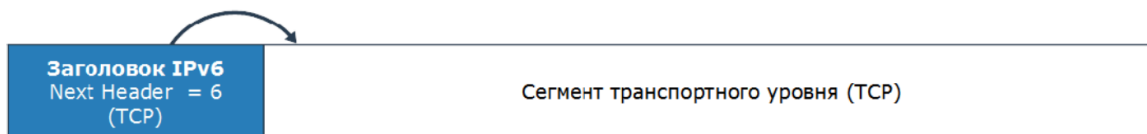
- Они расположены между фиксированным заголовком и заголовком протокола более высокого уровня
- Пакет IPv6 может вообще не содержать расширенных заголовков, а может содержать 1 или сразу несколько
- Каждый из расширенных заголовков идентифицируется значением поля «Next Header» предшествующего ему заголовка

Основные типы расширенных заголовков:

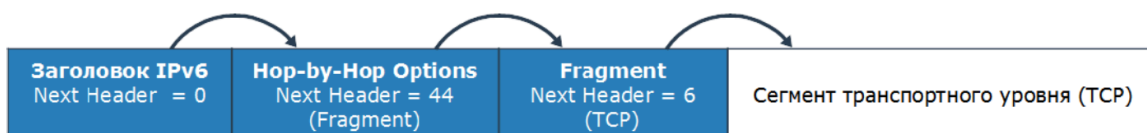
Расширенный заголовок	Тип	Описание
Hop-by-Hop Options	0	Параметры которые должны быть обработаны каждым транзитным узлом
Routing	43	Позволяет отправителю определять список узлов, которые пакет должен пройти
Fragment	44	Содержит информацию по фрагментации пакета
Encapsulating Security Payload (ESP)	50	Обеспечивает шифрование данных с помощью IPSec
Destination Options	60	Определяет произвольный набор опций, которые должны быть обработаны получателем пакета
Authentication Header (AH)	51	Содержит информацию для проверки подлинности зашифрованных данных при использовании IPSec
TCP	6	Заголовок вышележащего уровня
UDP	17	
ICMPv6	58	

Fe31:8800)
:0b)

IPv6-пакет без расширенных заголовков



IPv6-пакет с расширенными заголовками



Сравнение IPv4 и IPv6

Заголовок IPv4 (20 байт)

Версия (4 бита)	Длина заголовка (4 бита)	Тип сервиса (8 бит)	Общая длина (16 бит)	
Идентификатор пакета (16 бит)			Флаги (3 бита)	Смещение фрагмента (13 бит)
Время жизни (8 бит)		Протокол (8 бит)	Контрольная сумма (16 бит)	
Адрес источника (32 бита)				
Адрес назначения (32 бита)				

Заголовок IPv6 (40 байт)

Версия (4 бита)	Класс трафика (8 бит)	Метка потока (20 бит)	
Размер поля данных (16 бит)		Следующий заголовок (8 бит)	Предельное число шагов (8 бит)
Адрес источника (128 бит)			
Адрес назначения (128 бит)			

Размер: Базовый заголовок IPv4 имеет длину 20 байт, в то время как фиксированный заголовок IPv6 — 40 байт.

Упрощение структуры: В IPv4 присутствуют такие поля, как *Длина заголовка*, *Тип сервиса*, *Идентификатор пакета*, *Флаги*, *Смещение фрагмента*, *Контрольная сумма*. В IPv6 фиксированный заголовок избавлен от этих полей (например, данные для фрагментации вынесены в отдельный расширенный заголовок Fragment).

Адресация: Адреса источника и назначения в IPv4 занимают по 32 бита каждый, а в IPv6 они значительно расширены и занимают по 128 бит каждый.

18 - Адресация в IPv6. Типы и структура адресов.

ШИШМАЧЕНКО ПОЧИНИ ВПН ИНАЧЕ Я НАЧНУ УЧИТЬСЯ 😡😡😡

Представление адреса

- Адрес IPv6 имеет длину 128 бит и записывается как восемь групп по четыре шестнадцатеричные цифры, которые разделены двоеточием.
- Пример полной записи адреса:
`2001:0DB8:AC10:FE01:0018:8BFF:FED8:E3E0.`

Для удобства предусмотрены способы сокращения записи IPv6-адреса:

- Нули в начале группы можно удалить или заменить одним нулем, если вся группа состоит только из нулей.
- Наибольшая идущая подряд группа, состоящая из более чем одного нуля, может быть заменена двойным двоеточием «::».
- При этом конечные нули в группе обязательно должны присутствовать (например, `1000` нельзя сократить до `1`).

`2001:0DB8:0000:0000:0000:ABCD:0000:0001`

`2001:0DB8:0:0:0:ABCD:0:1`

`2001:0DB8::ABCD:0:1`

`2001:0DB8:1000:0000:0000:ABCD:0000:0001`

`2001:0DB8:1000::ABCD:0:1`

Структура

IPv6-адрес всегда состоит из двух логических частей:

- **Префикс (Prefix):** Часть адреса, отведенная под идентификатор сети/подсети. Длина префикса определяет количество битов,

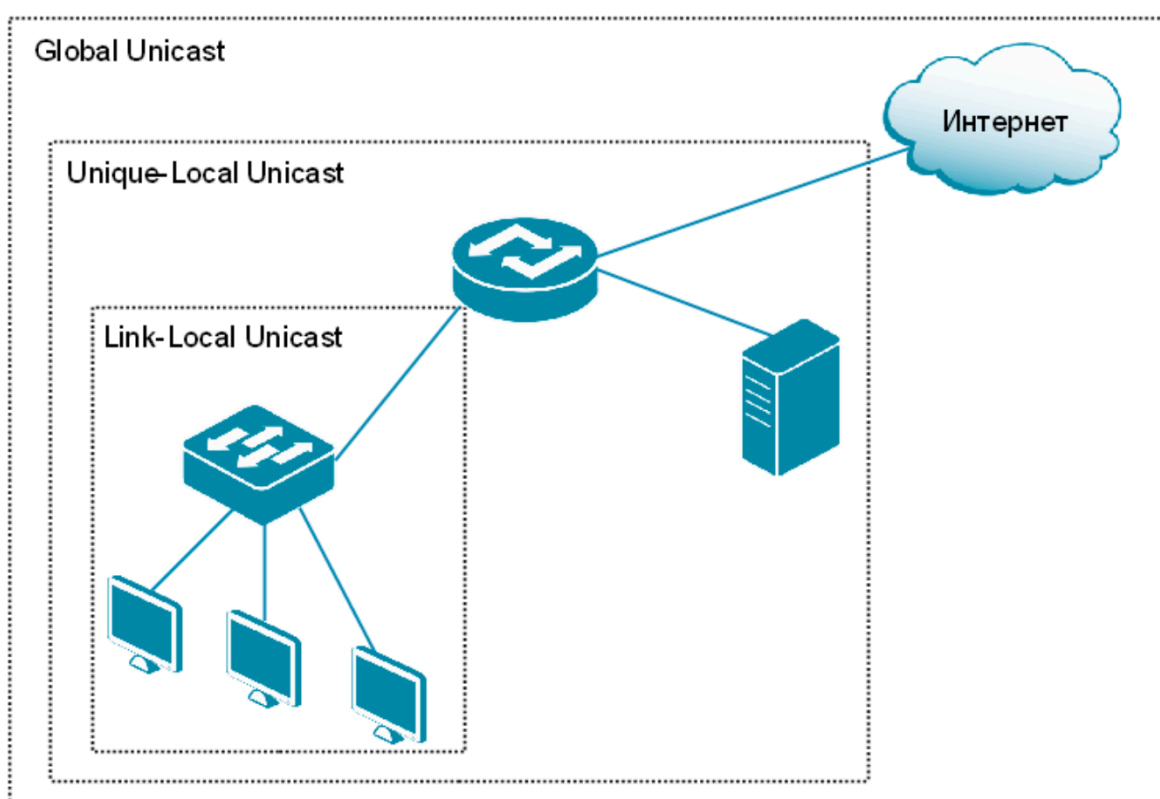
выделенных под этот идентификатор. Формат записи префикса аналогичен нотации CIDR в IPv4: **адрес IPv6/длина префикса** (например, **12AB:0:0:CD30::/60**).

- **Идентификатор интерфейса (Interface ID):** Часть адреса, идентифицирующая конкретный интерфейс узла. Этот идентификатор должен быть строго уникальным внутри своей сети или подсети.

Типы адресов

Адресное пространство протокола IPv6 разделено на три основных типа, при этом широковещательные адреса (broadcast) полностью исключены для снижения сетевого трафика и нагрузки на системы — их функции теперь выполняют групповые адреса.

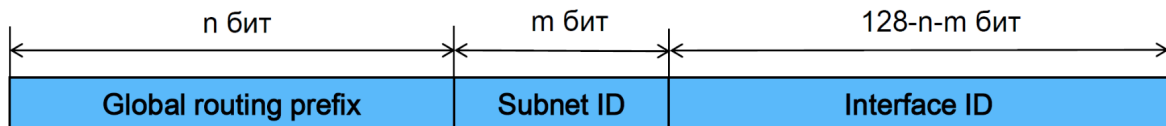
1. Индивидуальные адреса (Unicast)



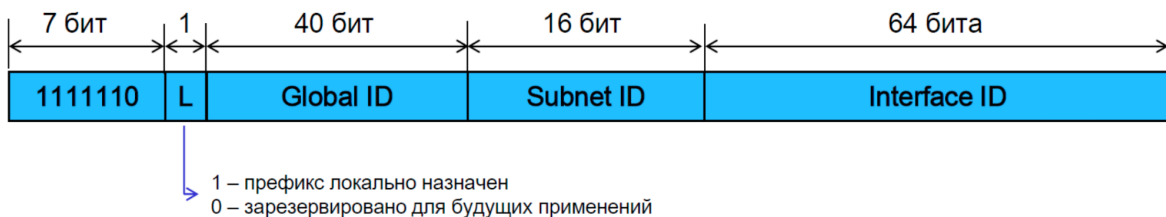
Идентифицируют ровно один интерфейс устройства. Пакеты, отправленные на индивидуальный адрес, доставляются только этому конкретному интерфейсу. Важно отметить, что интерфейс всегда имеет сразу несколько индивидуальных адресов разных типов.

Существует три подтипа индивидуальных адресов:

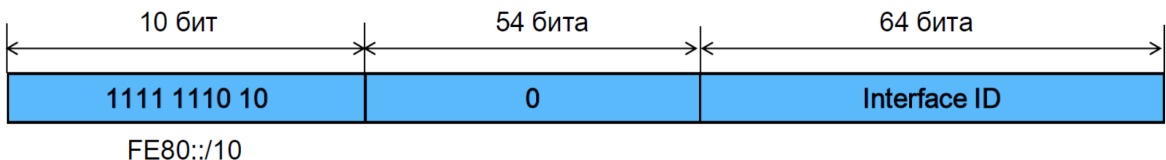
- **Global Unicast (Глобальные):** Являются аналогом публичных IPv4-адресов и используются для идентификации устройств в глобальной сети Интернет. Они выдаются IANA и в настоящее время назначаются с префикса **2000::/3**. Формат включает в себя глобальный префикс маршрутизации, идентификатор подсети и идентификатор интерфейса.



- **Unique-Local Unicast (Локально-используемые уникальные):** Являются глобально уникальными, но предназначены исключительно для адресации узлов внутри локальной сети (эквивалент частных IPv4-адресов). Они начинаются с префикса **FC00::/7**.

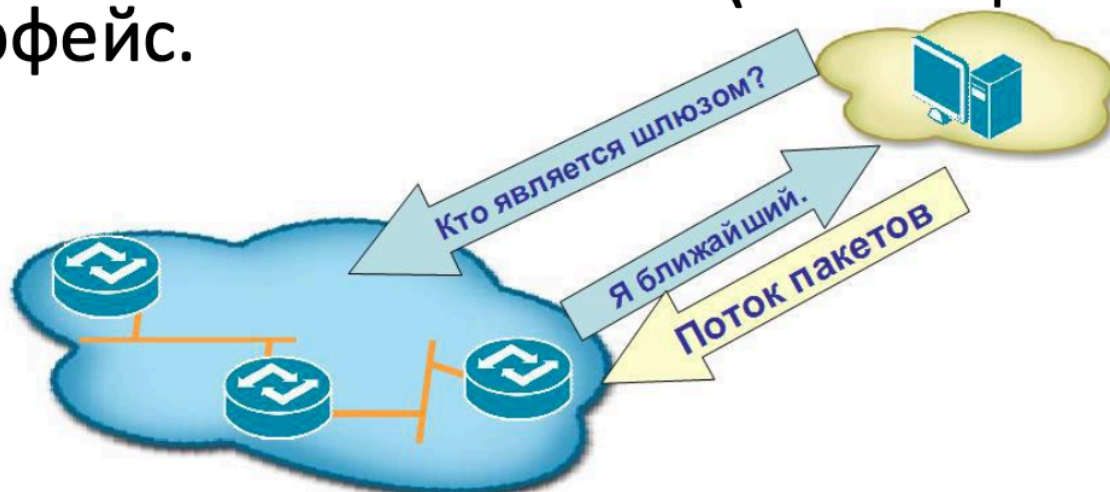


- **Link-Local Unicast (Локально-используемые индивидуальные):** Предназначены только для взаимодействия внутри одного сегмента сети или по прямому каналу связи «точка-точка». Маршрутизаторы никогда не пересылают Link-Local пакеты в другие сети. Узлы автоматически назначают себе такие адреса независимо от наличия маршрутизатора или DHCPv6-сервера, используя префикс **FE80::/10**.



2. Альтернативные адреса (Anycast)

Адреса «ближайший» (альтернативный) интерфейс.



Данный тип адресов позволяет адресовать целую группу интерфейсов, которые обычно принадлежат разным узлам сети.

- Пакеты, отправленные на альтернативный адрес, доставляются только на *один* интерфейс из группы — обычно это «ближайший» интерфейс с точки зрения метрик маршрутизации.
- Альтернативный адрес входит в адресное пространство индивидуальных адресов, но не может быть использован в качестве адреса отправителя пакета.
- Такие адреса назначаются только маршрутизаторам.

3. Групповые адреса (Multicast)



Идентифицируют группу интерфейсов, которые участвуют в получении одного и того же контента. Пакеты, посланные на такой адрес, доставляются всем участникам данной группы.

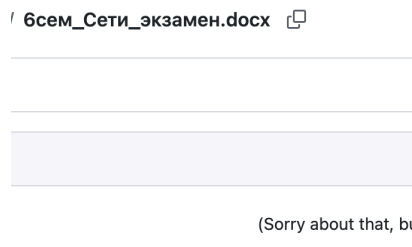
- Групповые адреса всегда начинаются с префикса **FF00::/8**.
- В структуре адреса присутствует специальное поле «Scope», которое определяет область действия адреса (то есть, как далеко друг от друга могут находиться члены данной группы: локальный узел, канал, организация или глобально).

Так как в IPv6 нет broadcast-адресов, их роль выполняют специальные (предопределенные) групповые адреса:

- **FF01::1** — группа, включающая все IPv6-узлы в диапазоне Interface-Local.
- **FF02::1** — группа, включающая все IPv6-узлы в диапазоне Link-Local.
- **FF01::2** — группа всех IPv6-маршрутизаторов в диапазоне Interface-Local.
- **FF02::2** — группа всех IPv6-маршрутизаторов в диапазоне Link-Local.
- **FF05::2** — группа всех IPv6-маршрутизаторов в диапазоне Site-Local.

19 - ICMPv6 и NDP.

try qwen studio ->



Sorry Bout That

Трек – Yeat · 2021 г. ⋮



—->

- ICMPv6 используется узлами IPv6 для сообщений об ошибках при обработке пакетов, и для выполнения других функций уровня Интернет, таких как диагностика (ICMPv6 «ping») и сообщение об участии в мультикастинг группах.
- Протокол ICMPv6 является интегрированной частью IPv6 и должен реализовываться каждым узлом, поддерживающим IPv6.
- ICMPv6 имеет код следующего заголовка 58 (это из таблички 17 билет)

Формат ICMPv6



- Поле **тип** указывает на тип сообщения. Его значение определяет формат последующих данных.
- Поле **код** зависит от типа сообщения. Оно используется для создания дополнительного уровня структуризации сообщения.
- Поле **контрольная сумма** используется для обнаружения повреждений сообщения ICMPv6 и заголовка IPv6.

Типы сообщений

Сообщения ICMPv6 образуют 2 класса:

- 1) сообщения об ошибках

идентифицируются по нулю в старшем бите поля тип. Таким образом, сообщения об ошибках могут иметь код поля тип от 0 до 127;

Сообщения об ошибках ICMPv6:

- 1 destination unreachable (место назначения недоступно)
- 2 packet too big (пакет слишком велик)
- 3 time exceeded (время превышено)
- 4 parameter problem (проблема с параметрами)

2) информационные сообщения
имеют коды поля тип от 128 до 255.

Информационные сообщения ICMPv6:

- 128 echo request (Запрос эхо)
- 129 echo reply (Эхо-отклик)
- 130 group membership query (запрос участия в группе)
- 131 group membership report (отчет об участии в группе)
- 132 group membership reduction (сокращение числа участников в группе)

Обработка сообщений

- Если получено сообщение о неизвестной ошибке ICMPv6, оно должно быть передано верхнему уровню.
- Если получено информационное сообщение ICMPv6 неизвестного типа, оно должно быть выброшено.
- Каждое ICMPv6 сообщение об ошибке (тип < 128) включает в себя целиком или частично IPv6 пакет, вызвавший ошибку, при условии, что сообщение об ошибке не превысит 576 октетов.
- В тех случаях, когда протокол интернет-уровня нуждается в передаче сообщения об ошибке ICMPv6 вышерасположенному уровню, тип протокола верхнего уровня извлекается из исходного пакета (содержащегося в теле сообщения об ошибке ICMPv6) и используется для выбора соответствующего протокола верхнего уровня при последующей обработке сообщения об ошибке.

Если исходный пакет имеет необычно большое число заголовков расширения, возможно, что тип протокола верхнего уровня может отсутствовать в сообщении ICMPv6, из-за укорочения исходного пакета до уровня 576 октетов. В этом случае сообщение об ошибке отбрасывается после обработки уровня IPv6.

Сообщение об ошибке ICMPv6 не должно посылаться в качестве результата получения: да мне похуй честно

- 1) сообщения об ошибке ICMPv6,
- 2) пакета, направленного по IPv6 мультикаст-адресу (существует два исключения из этого правила:
 - сообщения packet too big - пакет слишком велик) - чтобы позволить скорректировать MTU прохода,
 - сообщения parameter problem (проблема с параметрами), Код 2, оповещающий о нераспознанной опции),
- 3) мультикастинг-пакета канального уровня, (исключения пункта 2 применимы и здесь),
- 4) широковещательного пакета канального уровня, (исключения пункта 2 применимы и здесь),
- 5) пакета, чей адрес отправителя не однозначно определяет какой-то узел, например, не специфицированный адрес IPv6, мультикаст-адрес IPv6 или эникаст-адрес.

Протокол обнаружения соседей (NDP)

NDP ответственен за:

- автонастройку адреса конечных и промежуточных точек сети
- обнаружение других узлов на линии
- определение адреса других узлов канального уровня
- обнаружение конфликта адресов
- поиск доступных маршрутизаторов и DNS-серверов
- определение префикса адреса и поддержки доступности информации о путях к другим активным соседним узлам

Функционал NDP (в основе ICMPv6)

- **Запрос маршрутизатора** (тип 133, код 0) – хосты запрашивают с помощью сообщений запроса маршрутизатора информацию о местонахождении маршрутизаторов на подключенном канале связи. Маршрутизаторы, которые пересылают пакеты, не адресованные им, генерируют объявления маршрутизатора сразу после получения этого сообщения, а не в следующее запланированное время.
- **Объявление маршрутизатора** (тип 134, код 0) – маршрутизаторы объявляют о своем присутствии вместе с различными параметрами канала связи и Интернета либо периодически, либо в ответ на сообщение запроса маршрутизатора.

- **Запрос соседа** (тип 135 , код 0) – запросы соседа используются узлами для определения адреса канального уровня соседа или для проверки того, что сосед по-прежнему доступен через кэшированный адрес канального уровня.
- **Объявление соседа** (тип 136 , код 0) – используются узлами для ответа на сообщение Neighbor Solicitation или, без запроса, для быстрого предоставления новой информации.
- **Перенаправление** (тип 137 , код 0) – маршрутизаторы могут сообщать хостам о более подходящем маршрутизаторе первого перехода для пункта назначения

20 - DHCP для IPv4, DHCPv6.

DHCP (Dynamic Host Configuration Protocol) для IPv4

DHCP – протокол динамической конфигурации хостов, который автоматически назначает IP-адреса и другие сетевые параметры клиентам IPv4. Работает поверх **UDP**.

Функция	Описание
Назначение IP-адресов	Автоматическая выдача IP-адресов из пула
Маска подсети	Передача маски подсети клиенту
Шлюз по умолчанию	IP-адрес маршрутизатора (шлюза)
DNS-серверы	IP-адреса серверов доменных имен
Время аренды	Срок действия выданного адреса (lease time)

Процесс работы DHCP (DORA):

1. **DISCOVER** → Клиент: "Есть ли кто-нибудь, кто может дать мне IP?" (broadcast)
2. **OFFER** → Сервер: "Я могу дать тебе IP 192.168.1.100" (unicast на mac клиента)
3. **REQUEST** → Клиент: "Хорошо, беру 192.168.1.100" (broadcast)
4. **ACK** → Сервер: "Подтверждаю, IP 192.168.1.100 твой" (unicast на mac клиента)

Тут важно уточнить, что клиент отдает широковещательный запрос DISCOVER и его могут увидеть несколько DHCP-серверов (если их несколько в сети), в таком случае каждый отправляет OFFER с предлагаемым IP и клиент уже выбирает один из них, отправляя с REQUEST. Только сервер, который предложил клиенту этот IP отправит ACK, остальные проигнорируют.

Вообще есть *Broadcast flag* в пакете DHCP, который делает все эти запросы широковещательными, но его используют редко (когда клиент не может принимать *unicast*).

DHCPv6 — эквивалент DHCP для IPv6, который конфигурирует IPv6-хосты адресами, префиксами и другими параметрами.

Режим	Описание
Stateful (с состоянием)	Аналогичен DHCP IPv4 — сервер выдаёт полный IP-адрес и все параметры
Stateless (без состояния)	Сервер предоставляет только дополнительную информацию (DNS), а адреса хосты генерируют сами через SLAAC

SLAAC — механизм автоматической генерации IPv6-адреса хостом без сервера. Например:

Префикс сети (из Router Advertisement) + MAC-адрес (EUI-64) =
IPv6-адрес

Каждый DHCPv6-клиент и DHCPv6-сервер имеют свой DUID:

DUID (DHCP Unique Identifier) — уникальный идентификатор DHCP, который используется в **DHCPv6** для идентификации клиента и сервера. DUID состоит из 2-байтового поля типа и поля идентификатора переменной длины до 128 байт.

Тип	Название	Как генерируется	Описание
1	DUID-LLT	Адрес канального уровня + время	Для Ethernet используется MAC-адрес + временная метка
2	DUID-EN	Производитель оборудования	Присваивается производителем (на основе номера компании)
3	DUID-LL	Адрес канального уровня	Основывается только на MAC-адресе интерфейса
4	DUID-UUID	UUID (универсальный уникальный идентификатор)	Глобально уникальный, генерируется с использованием UUID

Сервер использует DUID для **отслеживания аренды** и применения специфических конфигураций.

Не думаю, что мы это запомним, но пусть будет **(заголовок DHCPv4)**:

Dynamic Host Configuration Protocol				
Bit Offset	0–15		16–31	
0	OpCode	Hardware Type	Hardware Length	Hops
32	Transaction ID			
64	Seconds Elapsed		Flags	
96	Client IP Address			
128	Your IP Address			
160	Server IP Address			
196	Gateway IP Address			
228+	Client Hardware Address (16 bytes)			
	Server Host Name (64 bytes)			
	Boot File (128 bytes)			
	Options			

21 - NAT, принципы работы, преимущества и недостатки.

NAT (Network Address Translation) – технология трансляции сетевых адресов, которая позволяет устройствам из частной сети с «серыми» IP-адресами получать доступ в интернет через один или несколько публичных IP-адресов.

Принцип работы:

1. Устройства внутри локальной сети используют приватные IP-адреса (192.168.x.x, 10.x.x.x, 172.16.x.x–172.31.x.x)
2. При выходе в интернет NAT-роутер заменяет локальный IP на публичный IP-адрес
3. NAT хранит таблицу трансляций (какое внутреннее устройство → какой внешний порт)
4. Приходящий ответ NAT преобразует обратно в локальный адрес и отправляет нужному устройству

Тип NAT	Принцип работы	Применение
Статический NAT	Локальные и глобальные адреса транслируются 1:1	Веб-серверы, устройства с постоянной доступностью извне
Динамический NAT	Набору «внутреннему» соответствует группа «внешних» адресов	Когда нужно несколько публичных адресов для группы клиентов
PAT (Port Address Translation)	Многие к одному – несколько приватных адресов на один публичный, идентификация через TCP/UDP-порты	Домашние роутеры (самая распространённая форма NAT)

Для последнего (самого распространенного) типа NAT используется комбинация IP+port для трансляции адреса:

Внутренний IP	Внутренний порт	Внешний IP	Внешний порт
192.168.1.2	50300	184.86.48.128	49127
192.168.1.3	52001	184.86.48.128	49128
192.168.1.2	49238	184.86.48.128	49129

Преимущества и недостатки NAT:

Преимущество	Описание
Консервация публичных IP-адресов	Количество публичных адресов ограничено, NAT позволяет использовать один адрес для множества устройств
Повышение безопасности	Внутренние адреса хостов неизвестны за NAT-транслятором – внешние атакующие не видят реальную топологию сети
Гибкость адресации	Изменения схемы адресации публичных адресов не влияет на адресацию хостов во внутреннем сегменте

Недостаток	Описание
Снижение производительности	NAT увеличивает задержки коммутации, поскольку преобразование каждого IPv4-адреса в заголовках пакетов требует времени (особенно для протоколов реального времени, таких как VoIP)
Потеря сквозной адресации	Многие интернет-протоколы и приложения зависят от сквозной адресации от источника до узла назначения. Некоторые приложения не совместимы с NAT (например, электронные подписи)
Усложнение трассировки	Утрачивается возможность трассировки сквозного соединения IPv4 – очень сильно усложняется трассировка пакетов, подвергающихся многочисленным изменениям адреса

Проблемы с туннелированием	NAT усложняет протоколы туннелирования вроде IPsec, поскольку изменяет значения в заголовках, что мешает проверкам целостности
Проблемы с P2P	Работа служб, требующих инициализации соединений TCP из внешней сети или использующих UDP, может быть нарушена. Некоторые протоколы (например, FTP в пассивном режиме) не работают, если обе системы отделены NAT

22 - Маршрутизация, статическая и динамическая, правило наидлиннейшего префикса. Link-State vs Distance-Vector.

Маршрутизация – процесс определения оптимального пути передачи пакета от источника к получателю через составную сеть.

Характеристика	Статическая маршрутизация	Динамическая маршрутизация
Настройка	Администратор вручную прописывает маршруты в таблицу маршрутизации	Маршрутизаторы автоматически обмениваются информацией с помощью протоколов
Обновление	Не обновляется автоматически — требует ручного вмешательства	Демоны непрерывно получают информацию от других демонов, постоянно обновляя таблицы
Преимущества	Простота, нет нагрузки на сеть, точный контроль маршрутов	Автоматическая адаптация к изменениям, масштабируемость
Недостатки	Не масштабируется, не адаптируется к изменениям топологии	Нагрузка на сеть, использование CPU, сложнее настройка
Применение	Малые сети, шлюз по умолчанию, критичные маршруты	Большие сети, интернет, часто меняющиеся топологии

Протоколы динамической маршрутизации: RIP, OSPF, BGP.

Правило наидлиннейшего префикса (Longest Prefix Match)

Когда в таблице маршрутизации есть несколько маршрутов, подходящих к адресу назначения, маршрутизатор выбирает маршрут с наименьшим адресом сети (наиболее специфичным), то есть с наибольшим размером маски.

Пример:

Есть адреса в таблице маршрутизации:

192.168.1.0/24 и 192.168.0.0/16

Нам нужно отправить пакет в 192.168.1.50 – подходят оба адреса, но выбираем мы с большей маской – /24 (он более специфичный).

Distance Vector и Link State – два основных алгоритма динамической маршрутизации (*то есть как выбрать маршрут*).

Distance Vector Routing (маршрутизация на основе вектора расстояния):

Характеристика	Описание
Как работает	Маршрутизатор вычисляет расстояния до всех возможных целей только на основе непосредственных соседей
Что передаёт	Таблица маршрутизации передаётся прямо соединённым соседям через регулярные интервалы
Алгоритм	Bellman-Ford
Протоколы	RIP, IGRP
"Routing by Rumor"	Маршрутизатор знает только о соседях — назван так, потому что полагается на информацию от соседей, не зная полной картины сети

Link State Routing (маршрутизация на основе состояния каналов):

Характеристика	Описание
Как работает	Каждый маршрутизатор сохраняет информацию о всей сети, а не только о соседях
Что передаёт	Маршрутизаторы заливают (flood) информацию о состоянии канала по всей сети, чтобы все маршрутизаторы имели одинаковое представление о топологии
Алгоритм	Dijkstra (Shortest Path First, SPF)
Протоколы	OSPF, IS-IS
Карта сети	Каждый маршрутизатор строит карту связности всей сети, затем независимо вычисляет оптимальные маршруты

Сравнение:

Характеристика	Distance Vector	Link State
Знание сети	Только о непосредственных соседях	О всей сети
Алгоритм	Bellman-Ford	Dijkstra (SPF)
Обновления	Регулярные, большие, только соседям	При изменении, маленькие, всем маршрутизаторам
Метрика	Hop count (количество прыжков)	Cost (затраты, пропускная способность)
Скорость реакции	Медленнее на изменения	Быстрее на изменения
Сложность	Проще, меньше CPU	Сложнее, больше CPU
Циклы	Возможны (Count to Infinity)	Нет постоянных циклов

23 - OSPF. Алгоритм работы. Зоны, формат и типы пакетов.

OSPF – динамический протокол маршрутизации, протокол состояния каналов (Link-State).

Самый известный протокол из семейства IGP (Interior Gateway Protocol), разработанный в середине 1980-х годов рабочей группой IETF (то есть это стандартный протокол).

Алгоритм (Дейкстры):

// $D(v)$ равно сумме весов связей для данного пути, $c(i,j)$ – вес связи между.

1. Устанавливаем множество узлов $N = \{1\}$
2. Для каждого узла v не из множества N устанавливаем $D(v) = c(1,v)$.
3. Для каждого шага находим узел w не из множества N , для которого $D(w)$ минимально, и добавляем узел w в множество N
4. Актуализируем $D(v)$ для всех узлов не из множества N
5. $D(v) = \min\{D(v), D(v) + c(w,v)\}$.
6. Повторяем шаги 2-4, пока все узлы не окажутся в множестве N .

На отдельной странице позже есть картинка, если хотите разобраться подробнее.

Зона (Area) — совокупность сетей и маршрутизаторов, имеющих один и тот же идентификатор зоны (домен).

Разделение на зоны позволяет уменьшить нагрузку на ЦПУ маршрутизаторов (меньше SPF-перерасчетов), размер таблиц маршрутизации, количество пакетов обновлений состояния канала.

Магистральная зона (нулевая зона или зона 0.0.0.0) формирует ядро сети OSPF. Все остальные зоны соединены с ней, и межзональная маршрутизация происходит через маршрутизатор, соединенный с магистральной зоной.

Пограничный маршрутизатор (area border router, ABR) – соединяет одну или больше зон с магистральной зоной и выполняет функции шлюза для межзонального трафика.

Пограничный маршрутизатор автономной системы (AS boundary router, ASBR) – обменивается информацией с маршрутизаторами, принадлежащими другим автономным системам или не-OSPF маршрутизаторами.

Алгоритм работы OSPF:

1. Инициализация. Запуск OSPF, отправка Hello-пакетов
2. Обнаружение соседей. Настройка соседств (Neighbor)

Соседи – это роутеры с интерфейсами с общей сетью.

3. Установление смежности. Обмен базами данных (Full state)
4. Построение базы данных (LSDB). Все маршрутизаторы в зоне имеют одинаковую LSDB
5. Вычисление кратчайших путей → Алгоритм Dijkstra (SPF)
6. Построение таблицы маршрутизации → Вставка маршрутов в таблицу

Каждый роутер периодически отправляет какое-нибудь LSA (link-state advertisement). LSA также отправляются в том случае, когда изменяется состояние какого-нибудь роутера.

LSA включает в себя информацию о смежностях роутера. При сравнении установленных смежностей с состоянием канала быстро обнаруживаются отказавшие роутеры, и топология сети изменяется соответствующим образом.

Из топологической базы данных, генерируемых LSA, каждый роутер рассчитывает дерево наикратчайшего пути, корнем которого является он сам. В свою очередь дерево наикратчайшего пути выдает маршрутную таблицу.

Формат пакета OSPF

Октет	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0—3	Version							Type							Packet length																	
4—7	Router ID																															
8—11	Area ID																															
12—15	Checksum															Authentication type																
16—19	Authentication																															
20—23																																

Version — номер версии протокола OSPF, текущая версия OSPF для сетей IPv4 – 2, для IPv6 – 3;

Type — тип OSPF-пакета;

Packet length — длина пакета, включая заголовок;

Router ID — идентификатор маршрутизатора, уникальное 32-битное число, идентифицирующее маршрутизатор в пределах автономной системы;

Area ID — 32-битный идентификатор зоны (например, 0.0.0.0);

Authentication type — тип используемой схемы аутентификации, возможные значения:

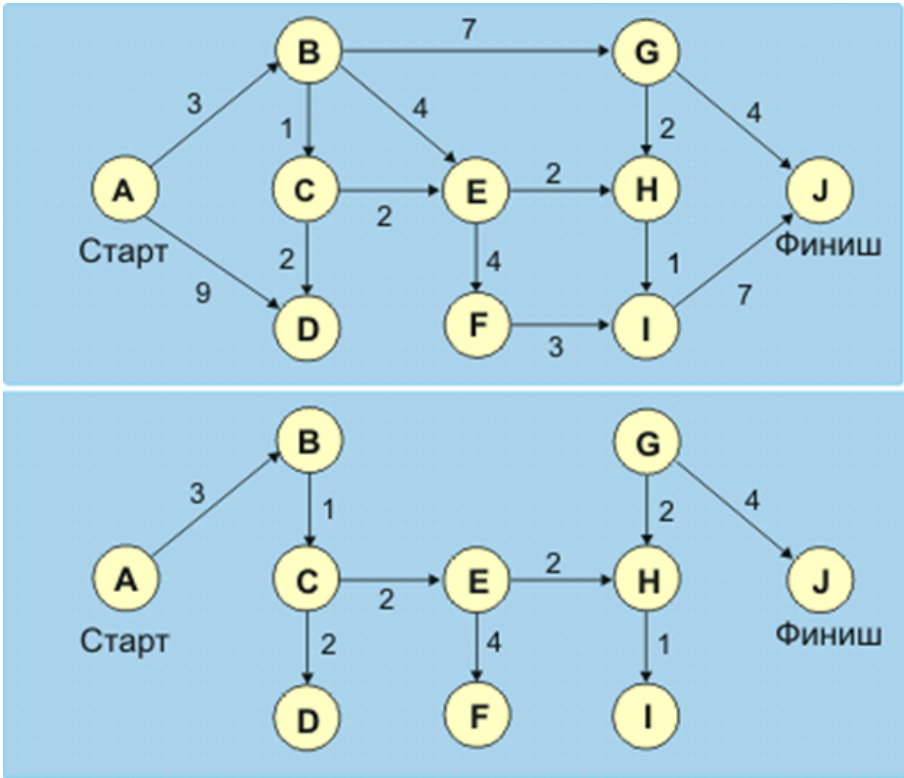
- 0 — аутентификация не используется
- 1 — аутентификация открытым текстом
- 2 — MD5-аутентификация

Authentication — поле данных аутентификации.

Типы пакетов OSPF (это все):

Тип	Название	Описание	Назначение
1	Hello	Пакет приветствия	Установление и поддержание отношений смежности с соседями
2	DBD (Database Description)	Пакет описания базы данных	Содержит сокращённый список LSDB отправителя; сверка с локальной базой
3	LSR (Link-State Request)	Пакет запроса состояния канала	Запрос дополнительных данных о записях в DBD
4	LSU (Link-State Update)	Пакет обновления состояния канала	Отклик на LSR; объявление новых данных
5	LSAck (Link-State Acknowledgment)	Пакет подтверждения состояния канала	Подтверждение приёма LSU; поле данных пустое

Картиночки для алгоритма Дейкстры (в целом можно скипать)



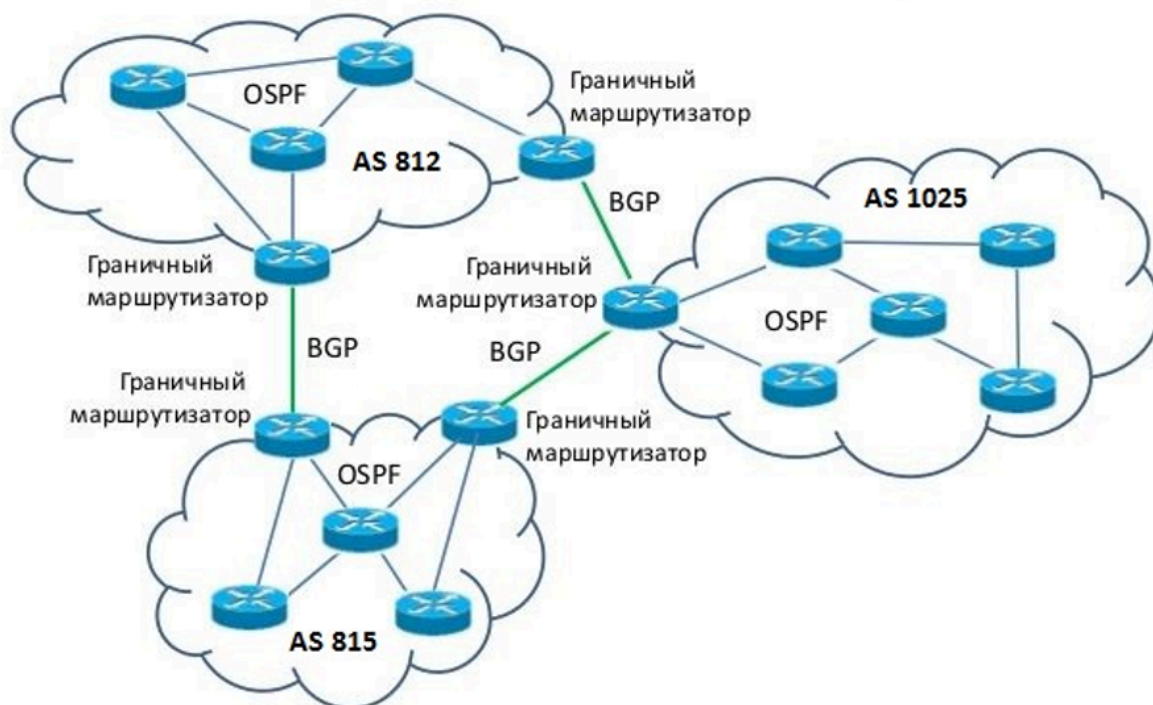
Множество	Метрика связи узла а с узлами									
Шаг	N	B	C	D	E	F	G	H	I	J
0	{A}	3	-	9	-	-	-	-	-	-
1	{A,B}	(3)	4	9	7	-	10	-	-	-
2	{A,B,C}	3	(4)	6	6	10	10	8	-	14
3	{A,B,C,D}	3	4	(6)	6	10	10	8	9	14
4	{A,B,C,D,E}	3	4	6	(6)	10	10	8	9	14
5	{A,B,C,D,E,H}	3	4	6	6	10	10	(8)	9	14
6	{A,B,C,D,E,H,I}	3	4	6	6	10	10	8	(9)	14
7	{A,B,C,D,E,H,I,F}	3	4	6	6	(10)	10	8	9	14
8	{A,B,C,D,E,H,I,F,G}	3	4	6	6	10	(10)	8	9	14
9	{A,B,C,D,E,H,I,F,G,J}	3	4	6	6	10	10	8	9	(14)

24 - BGP. Автономная система. Типы провайдеров. Состояния BGP.

BGP – протокол граничного шлюза, который используется для обмена информацией о маршрутизации и доступности между автономными системами в Интернете.

Работает на основе **TCP** (нужна надежность), таким образом BGP-соседи (peer) могут быть подключены не напрямую.

Среди прочих протоколов маршрутизации BGP выделяют отдельно, т.к. в отличие от RIP, OSPF и прочих IGP (Interior Gateway Protocols), он используется для взаимодействия **между** автономными системами. Таким образом, BGP – единственный представитель «семейства» EGP (Exterior Gateway Protocols).



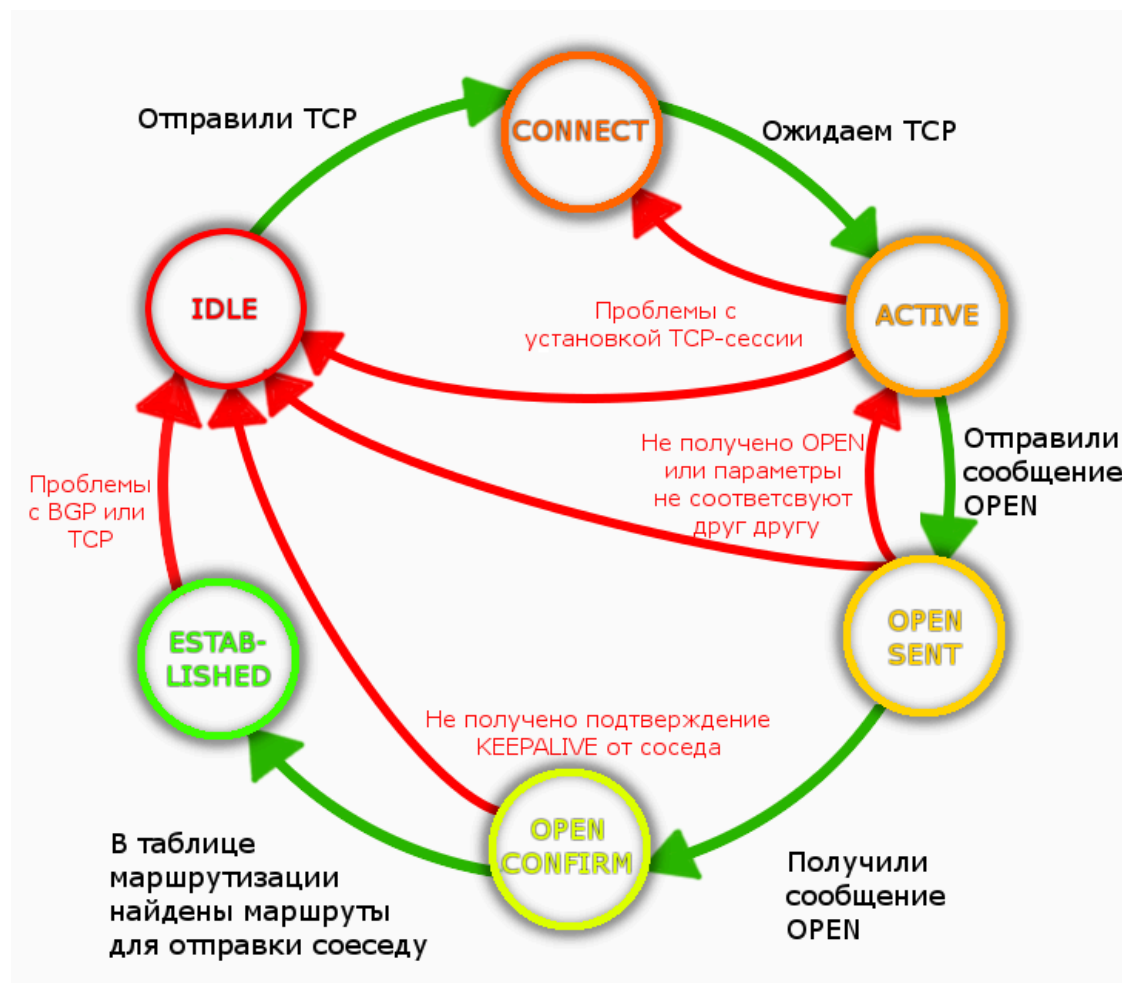
Установление соединения:

Состояние	Описание	Переход
IDLE	Начальное состояние — BGP-спикер не ищет соседей	При наличии маршрута до соседа → CONNECT
CONNECT	Ожидание завершения TCP-соединения (R1 ждёт SYN от соседа)	При успехе → OPEN SENT; при неудаче → ACTIVE

ACTIVE	Активная попытка установить TCP-соединение (отправка SYN)	При успехе → OPEN SENT; при неудаче → IDLE или CONNECT
OPEN SENT	TCP-соединение установлено, отправлено OPEN-сообщение	При получении OPEN от соседа → OPEN CONFIRM
OPEN CONFIRM	Получено OPEN-сообщение от соседа, ждём KEEPALIVE или NOTIFICATION	При успехе → ESTABLISHED; при неудаче → IDLE
ESTABLISHED	Сессия установлена, обмен данными возможен	При разрыве → IDLE

Для обмена маршрутной информацией используется сообщение **UPDATE**.

Для подтверждения при передаче маршрутной информации используются сообщения **KEEPALIVE** (по умолчанию каждые 60 секунд)



Автономная система (autonomous system, AS) – группа маршрутизаторов, обменивающаяся маршрутизирующей информацией с помощью одного протокола маршрутизации.

У каждой AS есть свой номер. Выдачей этих номеров занимаются RIR (Regional Internet Registry) или LIR (Local Internet Registry). Глобально этим занимается IANA (Internet Assigned Numbers Authority), но она делегирует свои задачи на RIR, а она на LIR.

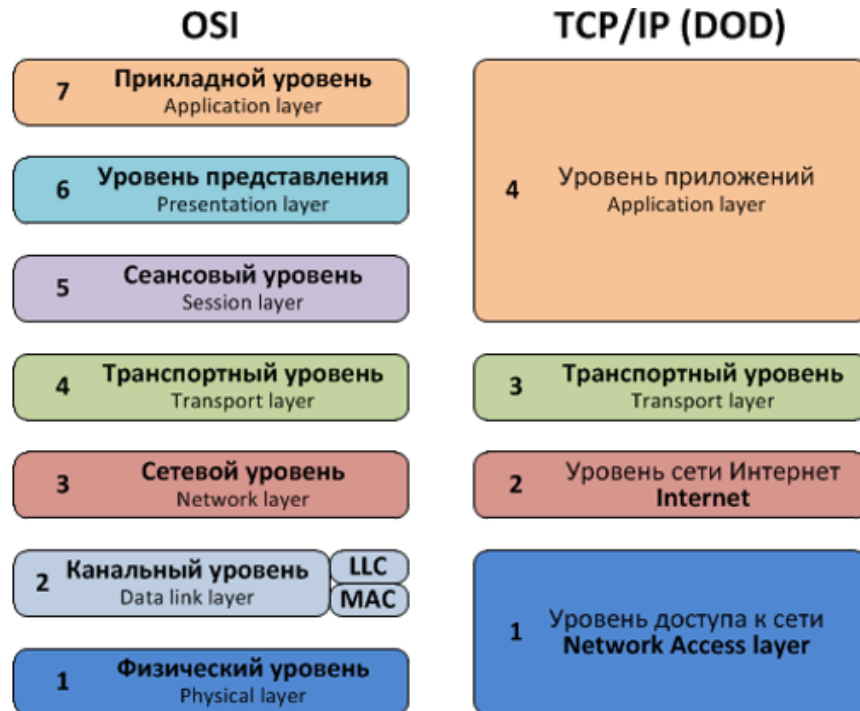
LIR'ом может стать почти любая желающая организация при наличии необходимых документов. Необходимо просто заключить договор с RIPE NCC и пройти процедуру регистрации.

Типы провайдеров:

1. **[Tier 1] Транзитный провайдер высшего уровня.** Имеет **глобальное покрытие**, не платит другим провайдерам за трафик, продает доступ нижестоящим. (Например: NTT, Telia)
2. **[Tier 2] Региональный провайдер.** Покупает транзит у Tier 1, продает доступ клиентам, может обмениваться трафиком через **peering**.
3. **[Tier 3] Локальный провайдер.** Только продает доступ конечным клиентам, покупает трафик у Tier 2 *(это уже наши любимые провайдеры, которым мы платим за интернет)*.

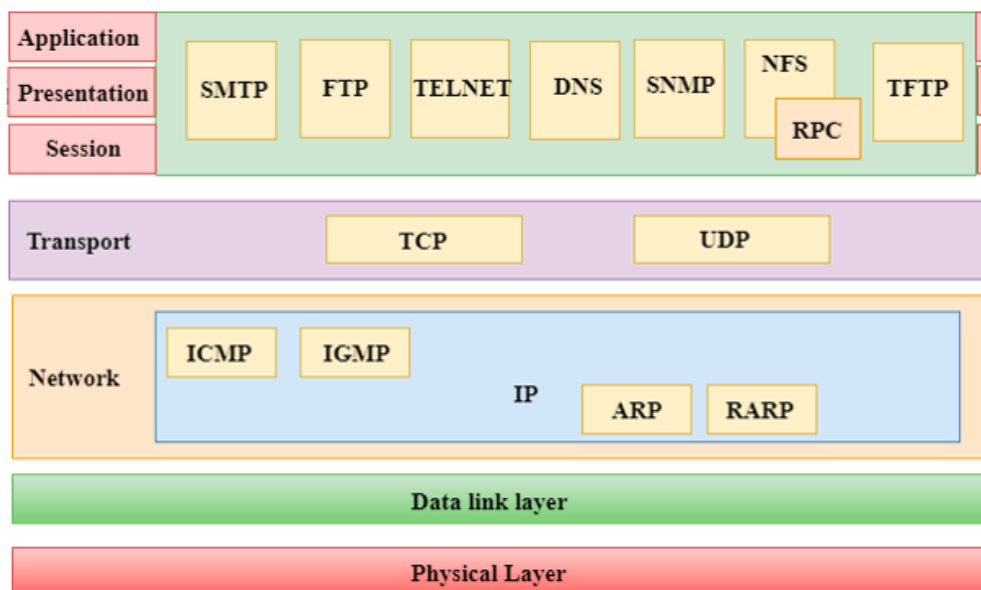
25 - Функции транспортного уровня стека TCP/IP. Порты. UDP.

Напоминаю транспортный уровень в иерархии протоколов:



Справа стек TCP/IP. Он широко используется как в глобальных, так и в локальных сетях. Имеет иерархическую структуру из 4-х уровней (снизу вверх):

1. Уровень доступа к сети (уровень сетевых интерфейсов).
2. Уровень сети Интернет (сетевой): IP, ICMP, RIP, OSPF.
3. Транспортный уровень: TCP, UDP.
4. Уровень приложений (прикладной уровень): FTP, Telnet, HTTP, SMTP, SNMP, TFTP.



Основные функции транспортного уровня стека TCP/IP

1. Передача данных между прикладными процессами, выполняющимися на различных узлах сети.
2. MUX/DMUX (мультиплексирование и демультиплексирование если че) потоков прикладных процессов – *диспетчеризация данных*.
3. Обеспечение требуемого качества передачи.
4. Управление потоком пакетов.

На практике протоколы транспортного уровня часто реализуют некоторые функции сеансового уровня ЭМВОС (это типа модель OSI но на русском).

Транспортный уровень стека TCP/IP может предоставлять вышележащему уровню два типа сервиса:

- **гарантированную доставку** обеспечивает протокол управления передачей (TCP),
- **доставку по возможности**, или доставку с максимальными усилиями, обеспечивает протокол пользовательских дейтаграмм (UDP).

В функции протоколов TCP и UDP также входит исполнение роли *связующего звена* между прилегающим к транспортному уровню **прикладным** и **сетевым** уровнями. От прикладного протокола транспортный уровень принимает задание на передачу данных с тем или иным качеством прикладному уровню-получателю. Т.е. прикладной уровень может потребовать качество передачи (TCP для надежности или UDP для скорости).

Порты

Назначение номеров портов

Порты назначаются тремя способами:

1. **Централизованный способ:** IETF/IANA (RFC 6335) (например, HTTP -> 80/tcp).
2. **Локальный способ:** назначает разработчик приложения (например, кастомный сервис -> 8080).
3. **Динамический способ:** ОС выделяет эфемерные порты для клиентов (например, клиент -> 49152–65535).

Обзор от ИИ 😊: RFC 6335 — это основополагающий технический стандарт IETF Datatracker, который определяет официальные

процедуры Internet Assigned Numbers Authority (IANA) по управлению реестром имен служб и номеров портов транспортных протоколов (TCP и UDP).

Файлы со списком сервисов:

- UNIX – /etc/services
- Windows 7/10 – C:\WINDOWS\system32\drivers\etc\services

N порта — ID прикладного процесса в узле

Прикладной уровень — “клиент-сервер”.

Клиентские порты — Сервисные порты.

- 0-1023 — зарезервированные (сервисные) порты,
- > 1023 — свободные (клиентские) порты,
- 0 — запрещен в TCP, но разрешен в UDP.

Примеры зарезервированных портов:

20-21 → FTP (data/control)

22 → SSH

23 → Telnet

25 → SMTP

53 → DNS (UDP/TCP)

80 → HTTP

110 → POP3

161 → SNMP

443 → HTTPS

2049 → NFS

UDP

User Datagram Protocol

Протокол *End-to-End*: протоколы транспортного уровня работают только в конечных узлах соединения. Тип протокола определяется **прикладным процессом и спецификой сети**.

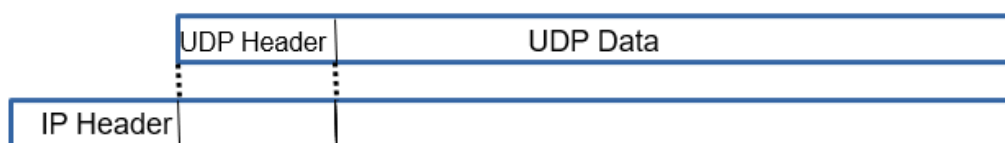
В стеке TCP/IP:

- UDP – connectionless protocol,
- TCP – connection oriented protocol.

То есть UDP — протокол без установления соединения. Каждая дейтаграмма является отдельным сообщением.

При работе на хосте-отправителе данные от приложений поступают по протоколу UDP через порт в виде сообщений. Протокол UDP добавляет к каждому отдельному сообщению свой 8-байтный заголовок, формируя из этих сообщений собственные протокольные единицы, называемые **UDP-дейтаграммами**, и передает их нижележащему протоколу IP. В этом и заключаются его функции по мультиплексированию данных.

Инкапсуляция UDP-дейтаграммы в IP-пакет:



ТОПОВАЯ картинка:

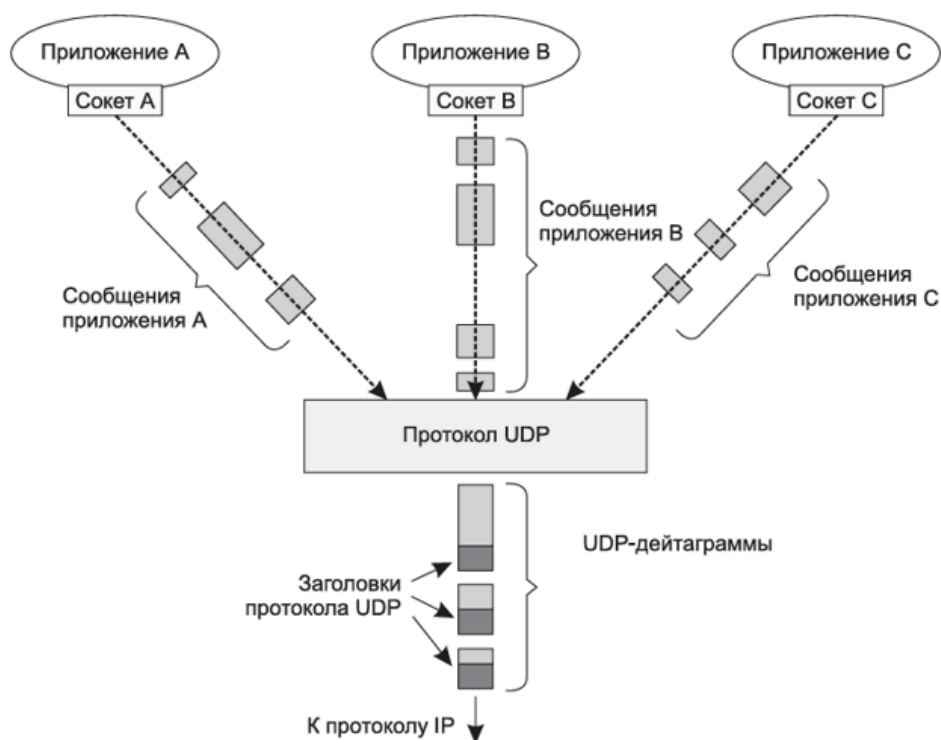


Рис. 15.3. Работа протокола UDP на хосте-отправителе

Каждая дейтаграмма переносит отдельное пользовательское сообщение. Сообщения могут иметь разную длину, не превышающую, однако, длину поля данных протокола IP, которое, в свою очередь, ограничено размером кадра технологии нижнего уровня. Поэтому если буфер UDP переполняется, то сообщение приложения отбрасывается.

Формат UDP-пакета:



Поля заголовка:

- Source Port (номер порта источника-отправителя) — 16 бит — может быть 0;
- Destination Port (номер порта назначения-получателя) — 16 бит;
- Length (длина заголовка + данных в байтах) — 16 бит;
- Checksum (контрольная сумма) — 16 бит — опционально в IPv4, обязательно в IPv6.

По UDP кратко:

- нет установления соединения;
- нет подтверждений, повторных передач, упорядочивания;
- нет управления потоком и перегрузками;
- минимальные задержки, низкие накладные расходы.

Дополнительно: про де- и мультиплексирование

полина привет :33333 это я ^^ знаю))) ты хотела киндер поэтому я пингви) у меня ощущение что мне мозги чешут(членом)±++++++
ня

привет пингвин :33333 я поняла ^-^ ня) мм) читаешь мои мысли я уже с ума сошла тута

Задача: передать данные между любыми прикладными процессами на любых узлах.

Каждый процессор может выполнять несколько процессов, т.е. необходим ID процесса. Прикладной процесс может иметь несколько точек входа (адресов назначения для TCP/UDP) — портов.

Адрес узла и № порта — это ID прикладного процесса в сети, т.е. **сокет**.
Соединение/сеанс:

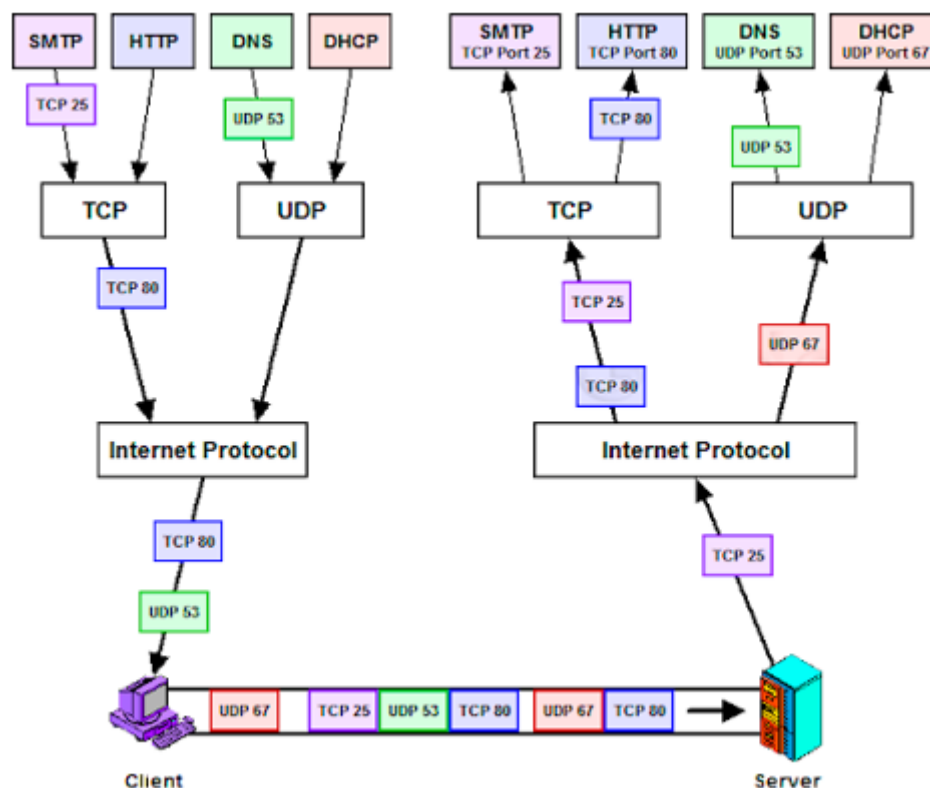
$(SrcIP, SrcPort) \Leftrightarrow (DstIP, DstPort)$

ТО ЕСТЬ:

Каждый компьютер способен выполнять несколько процессов, более того, даже отдельный прикладной процесс может иметь несколько точек входа, выступающих в качестве адресов назначения для пакетов данных. Поэтому доставка данных на сетевой интерфейс компьютера-получателя — это еще не конец пути, так как данные необходимо переправить конкретному процессу-получателю.

Реализуемая протоколами TCP и UDP **процедура распределения между прикладными процессами пакетов**, поступающих от сетевого уровня, называется **демультиплексированием**.

вот такая **ТОПОВАЯ** картинка:



Существует и обратная задача: **данные**, генерируемые разными приложениями, работающими на одном конечном узле, должны быть переданы **общему** для всех них **протольному модулю IP** для последующей отправки в сеть. Эту работу, называемую **мультиплексированием**, тоже выполняют протоколы TCP и UDP.

26 - Протокол TCP. Методы обеспечения надежной доставки.

Протокол TCP предназначен для передачи данных между приложениями. Он основан на **логическом соединении**, что позволяет обеспечивать *гарантированную доставку* данных.

Протокол TCP реализует *надежную доставку по ненадежным линиям* связи: TCP использует “ненадежный” дейтаграммный сервис протокола IP для транспортировки данных.

Основные функции

1. Управление потоком данных.
2. MUX/DMUX потоков данных.
3. Протокол End-to-End (не broadcast и не multicast).
4. Connection-oriented protocol.

При работе на хосте-отправителе протокол TCP рассматривает информацию, поступающую к нему от прикладных процессов, как *неструктурированный поток байтов*. Поступающие данные буферизуются средствами TCP. Для передачи на сетевой уровень из буфера “вырезается” некоторая непрерывная часть данных, которая называется **сегментом**, и снабжается заголовком.

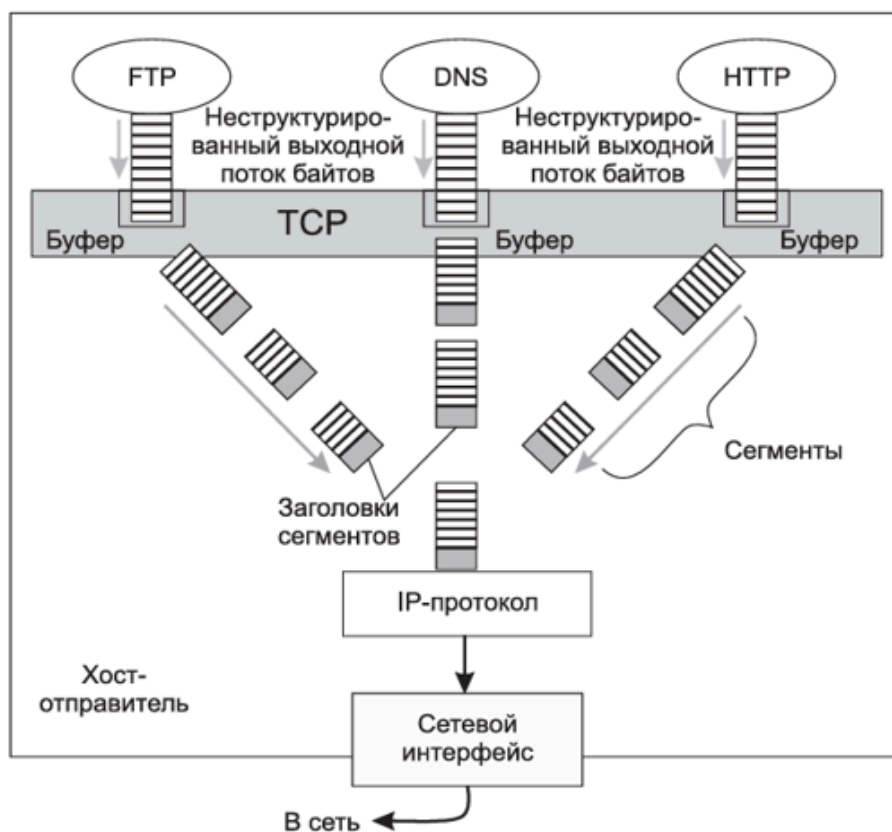


Рис. 15.4. Формирование TCP-сегментов из потока байтов

В отличие от протокола UDP, создающего дейтаграммы на основе логически обособленных единиц данных (сообщений, генерируемых приложениями), протокол TCP делит поток данных на сегменты **без учета их смысла** или внутренней структуры.

Формат TCP-сегмента (20-60 байт)



картинка вообще НЕ ТОПОВАЯ вот пояснение:

- **номер последовательности** (32 бита) — номер 1-го байта данных в сегменте, он определяет смещение сегмента относительно потока отправляемых данных;
- **номер подтверждения** (32 бита) — максимальный номер байта в полученном сегменте, увеличенном на единицу;
- **окно** (16 бит) — количество байтов данных, ожидаемых отправителем данного сегмента, начиная с байта, номер которого указан в поле подтвержденного номера;
- **указатель срочности** (16 бит) — указывает на конец данных, которые необходимо срочно принять, несмотря на переполнение буфера;
- **опции** — необязательное поле, которое используется для решения вспомогательных задач (например, согласование максимального размера сегмента).

Пояснения к однобитным полям (флагам), которые содержат служебную информацию о типе данного сегмента:

- NS – используется для защиты от многократной маркировки пакетов при ECN (RFC 3540),
- CWR (Congestion Window Reduced) – устанавливается отправителем, когда он снизил окно перегрузки;
- ECE (ECN-Echo) – сигнализирует о приеме пакета с пометкой о перегрузке;
- URG – срочное сообщение,
- ACK (Acknowledgment) – квитанция на принятый сегмент,
- PSH (Push) – запрос на отправку сообщения без ожидания заполнения буфера,
- RST (Reset) – запрос на аварийный сброс соединения,
- SYN (Synchronize) – сообщение, используемое для синхронизации счетчиков переданных данных при установлении соединения,
- FIN (Final) – признак достижения передающей стороной последнего байта в потоке передаваемых данных, т.е. признак того, что соединение завершилось нормально. Сначала посылает инициатор разрыва, затем ждёт подтверждения с другой стороны.

ECE и **CWR** используются вместе с механизмом Explicit Congestion Notification (ECN) для сигнализации о перегрузке сети без потери пакетов. ECE помечает, что сегмент получил пометку Congestion Experienced и CWR сообщают о снижении окна перегрузки отправителем

Если в положении этих битов стоит единица, то значение положительное (логично).

Логическое соединение

Идентификатор логического TCP-соединения — это пара сокетов (Port_a, IPa) - (Port_b, IPb), определенных для этого соединения двумя взаимодействующими процессами.

Фазы TCP-соединения:

1. Установление соединения.
2. Передача данных.
3. Завершение соединения.

Логическое соединение дает участникам обмена следить за тем, чтобы данные не были потеряны, искажены или

продублированы, а также чтобы они пришли к получателю в том порядке, в котором были отправлены.

Протокол TCP устанавливает логические соединения между *прикладными* процессами, причем в каждом соединении участвуют только **два** процесса. TCP-соединение является **полнодуплексным**, то есть каждый из участников этого соединения может одновременно получать и отправлять данные.

[TCP — это программный уровень, который поддерживает полнодуплексную передачу, но не гарантирует ее. Если на физическом аппаратном уровне используется только полудуплексная передача Ethernet, то технически мы получаем полудуплексную передачу с временными интервалами на аппаратном уровне, даже если программный уровень TCP не замечает разницы.]

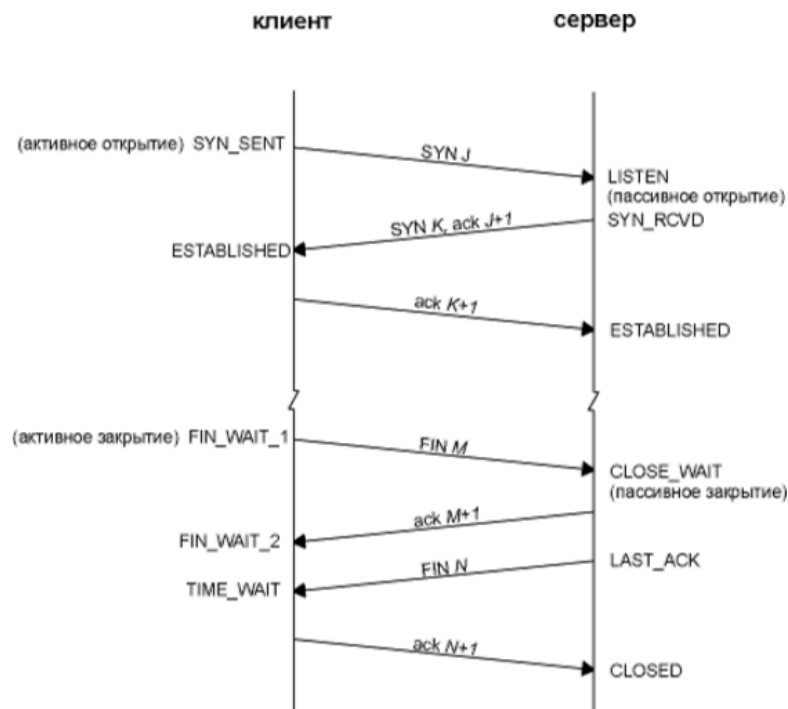
Таким образом, если оборудование поддерживает это, то TCP действительно является полнодуплексным. Если оборудование поддерживает только полудуплексную связь, то программное обеспечение TCP работает как полнодуплексное, поскольку оно обеспечивает «одновременную» двунаправленную связь, но технически оно является только полудуплексным.

Современный Ethernet, начиная со стандарта 10BASE-T [10 Мбит/с], по-видимому, теперь действительно является полнодуплексным]

Протокольные модули TCP устанавливаются на **конечных** узлах. При установлении логического соединения модули TCP договариваются между собой о параметрах процедуры обмена данными. В протоколе TCP каждая сторона соединения посылает противоположной стороне следующие параметры:

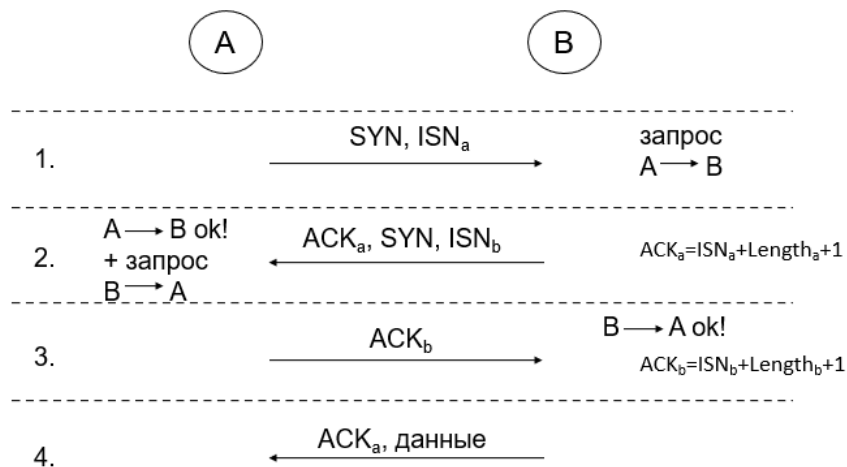
- максимальный размер сегмента, который она готова принимать;
- максимальный объем данных (возможно несколько сегментов), которые она разрешает другой стороне передавать в свою сторону;
- начальный порядковый номер байта, с которого она начинает отсчет потока данных в рамках данного соединения.

Механизмы обеспечения надежности



При установлении соединения определяется maximum segment size (MSS)

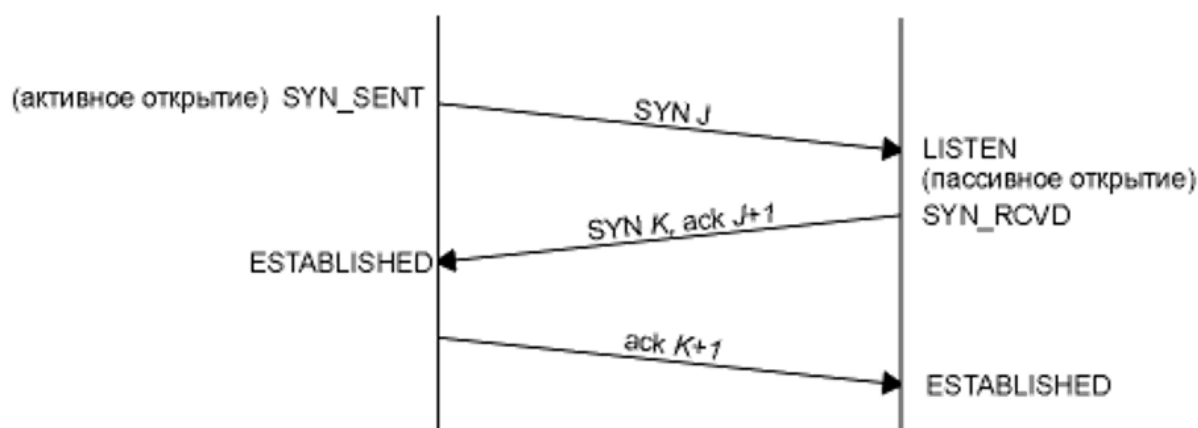
1. Установление соединения (трехстороннее рукопожатие)



ISN - Initial sequence number

- Клиент отправляет на сервер сегмент данных с флагом SYN, который отвечает за установку соединения, а также случайный номер последовательности байт (**ISN** — Initial Sequence Number), с которого будет проходить отсчет байт данных.
- Получив запрос, сервер подтверждает его, отправляя SYN и подтверждение (ACK) и свой ISN.
- Клиент подтверждает получение, отправив подтверждение (ACK).
- Соединение установлено.

На самой первой картинке это вот эта часть:



Клиент в ACK указывает sequence number + 1 сервера, а сервер в сообщениях клиенту указывает в ACK sequence number + 1 клиента. То есть в TCP соединении есть два номера последовательности – у одной и у другой стороны.

TCP A		TCP B
1. CLOSED		LISTEN
2. SYN-SENT	--> <SEQ=100><CTL=SYN>	--> SYN-RECEIVED
3. ESTABLISHED	<-- <SEQ=300><ACK=101><CTL=SYN,ACK>	<-- SYN-RECEIVED
4. ESTABLISHED	--> <SEQ=101><ACK=301><CTL=ACK>	--> ESTABLISHED
5. ESTABLISHED	--> <SEQ=101><ACK=301><CTL=ACK><DATA>	--> ESTABLISHED

Basic 3-Way Handshake for Connection Synchronization

В строке 2 TCP A начинает с отправки сегмента SYN, указывающий, что он будет использовать номера последовательности, начиная с номера 100. В строке 3 TCP B отправляет SYN и подтверждает SYN, который он получил от TCP A. Обратите внимание, что поле подтверждения указывает, что TCP B теперь ожидает получить последовательность 101, подтверждая SYN, который занимал последовательность 100.

В строке 4 TCP A отвечает пустым сегментом, содержащим ACK для SYN TCP B; а в строке 5 TCP A отправляет некоторые данные. Обратите внимание, что номер последовательности сегмента в строке 5 такой же, как и в строке 4, поскольку ACK не занимает пространство номера последовательности (если бы это было так, мы закончили бы ACK).

2. Нумерация байтов и управление порядком доставки

Для контроля байтов используется Sequence Number, который позволяет определить начало всего потока байт, которыми обмениваются сервер и клиент. С каждой новой пересылкой четко определяется байт начала и байт конца сообщений.

Например, SEQ = 100. Получатель в свою очередь отправит в следующем сообщении ACK = 101 — значит, мы приняли 1 байт данных.

Еще пример:

Пакет 4: клиент => сервер

TCP Segment Len: 72

Sequence Number: 1728730490

[Next Sequence Number: 73 (relative sequence number)]

Acknowledgment number (raw): 379626816

Пакет 5: сервер => клиент ([ACK])

ACK=73 – сервер подтвердил получение первых 72 байт.

Acknowledgment Number (raw): 1728730562

Sequence Number (raw): 379626816

Можем увидеть, что ACK в пакете 5 больше SYN пакета 4 ровно на 72 байта, т.е. на его длину ($1728730562 - 1728730490 = 72$)

Отправка подтверждения ACK называется **квитированием**.

Хайповая картиночка из презентации:

В заголовке сегмента (A→B):

N последовательности – N первого байта передаваемого сегмента A→B

N подтверждения - N последнего байта полученного сегмента + 1 (полученного в обратном направлении B→A)

№ посл-ти 39100 38780 38440 36980 35520 34060 32600

1400	320	340	1460	1460	1460	1460	...	...	...
------	-----	-----	------	------	------	------	-----	-----	-----

Направление передачи сегментов

№ подтверждения 38781

Накопительный принцип квитирования:

- подтверждается только непрерывный поток байт,
- возможен один ACK на несколько сегментов.

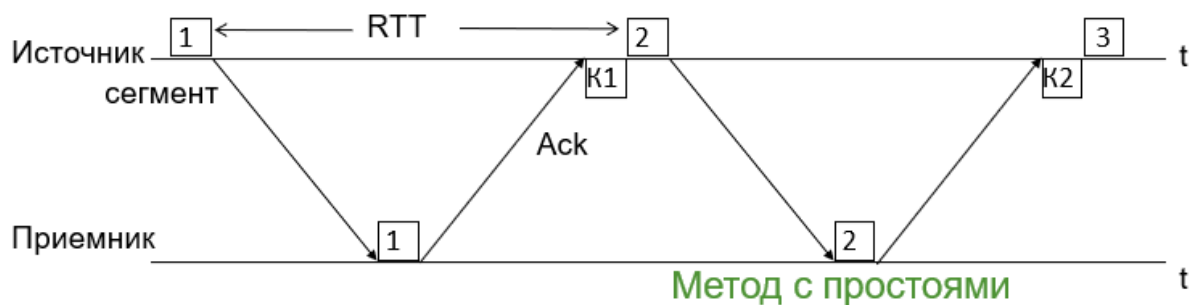
3. Надежность доставки

На стороне передатчика происходит:

- разбиение на блоки (сегменты) и нумерация,
- посылка сегмента – запуск таймера,
- TimeOut – отсутствие Ack – повторная передача.

На стороне приемника:

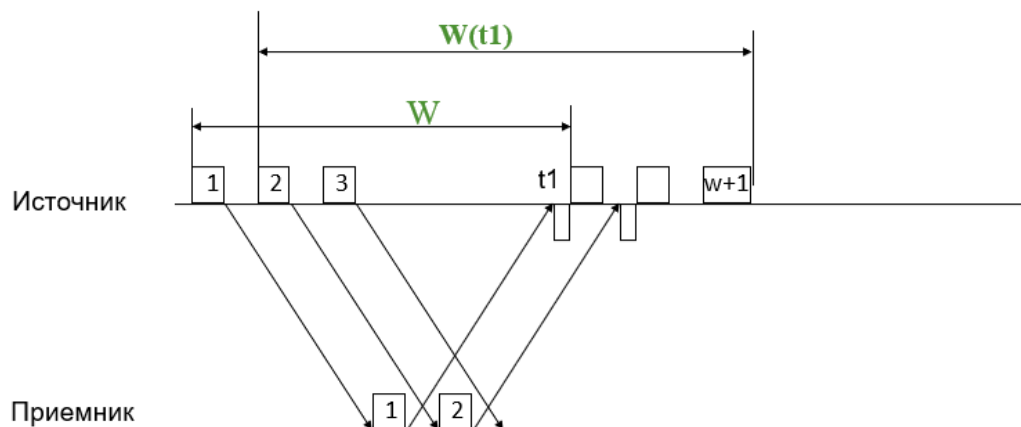
- получение сегмента – проверка контрольной суммы:
 - нет ошибок – отправка Ack,
 - иначе – удаление сегмента (нет Ack).
- упорядочивание сегментов (дубликаты удаляются).



RTT, TimeOut $\sim 2RTT$

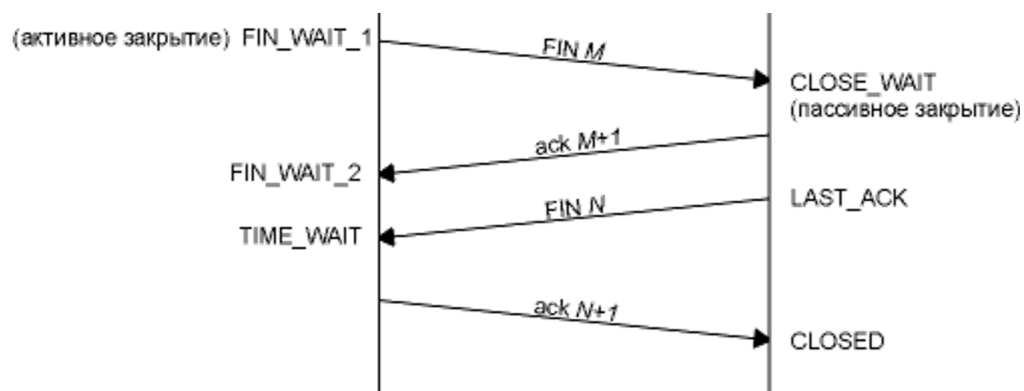
4. Контроль потока

Позволяет адаптировать скорость отправителя к возможностям приемника, предотвращая переполнение буферов: получает сообщается размер доступного окна (WiNdOw SiZZe ой простите немножко заваябилась хотела написать Window Size), а отправитель в свою очередь не может передать больше данных, чем позволяет окно.



- **Окно (W)** – число байтов данных, которые отправитель может отправить без получения подтверждения

5. Корректное завершение соединения (четырёхстороннее рукопожатие)



TCP A		TCP B
1. ESTABLISHED		<u>ESTABLISHED</u>
2. (Close) FIN-WAIT-1	--> <SEQ=100><ACK=300><CTL=FIN,ACK>	--> CLOSE-WAIT
3. FIN-WAIT-2	<-- <SEQ=300><ACK=101><CTL=ACK>	<-- CLOSE-WAIT
4. TIME-WAIT	<-- <SEQ=300><ACK=101><CTL=FIN,ACK>	(Close) <-- LAST-ACK
5. TIME-WAIT	--> <SEQ=101><ACK=301><CTL=ACK>	--> CLOSED
6. (2 MSL) CLOSED		

Normal Close Sequence

Что происходит

1. Клиент отправляет сегмент с флагом FIN, что сигнализирует о прекращении соединения.
2. Сервер отправляет ACK, что подтверждает запрос на завершение соединения.
3. Сервер отправляет сегмент с флагом FIN, чтобы подтвердить завершение с его стороны.
4. Клиент отправляет сегмент с флагом ACK, чтобы подтвердить завершение соединения.

6. Управление перегрузкой сети — Explicit Congestion Notification

ECN – гладкий механизм борьбы с перегрузкой сети, который позволяет маршрутизаторам и конечным узлам координировать

снижение скорости передачи без потерь пакетов и с минимальными задержками.

1. Когда маршрутизатор сталкивается с перегрузкой, вместо сброса пакета он устанавливает **кодвое значение СЕ** в **сегменте Тип обслуживания заголовка IP** (третий с начала сегмент). Выставляем 11 в последние два бита поля (*IP-уровень*).
2. **Сигнал о перегрузке приёму.** Приёмник видит в заголовке IP пакета $CE=1$ и в следующем ACK устанавливает флаг ECE. Это «эхо» доставки пакета с меткой о перегрузке обратно отправителю (*TCP-уровень*).
3. **Реакция отправителя.** Получив ACK с $ECE=1$, отправитель снижает размер окна перегрузки, аналогично тому, как он бы отреагировал на потерю пакета. В том же сегменте выставляет флаг CWR, подтверждая приемнику, что он уменьшил скорость передачи (*TCP-уровень*).

27 - Протокол TCP. Автомат состояния TCP. Ориентация на соединение. Установление и закрытие TCP-соединения.

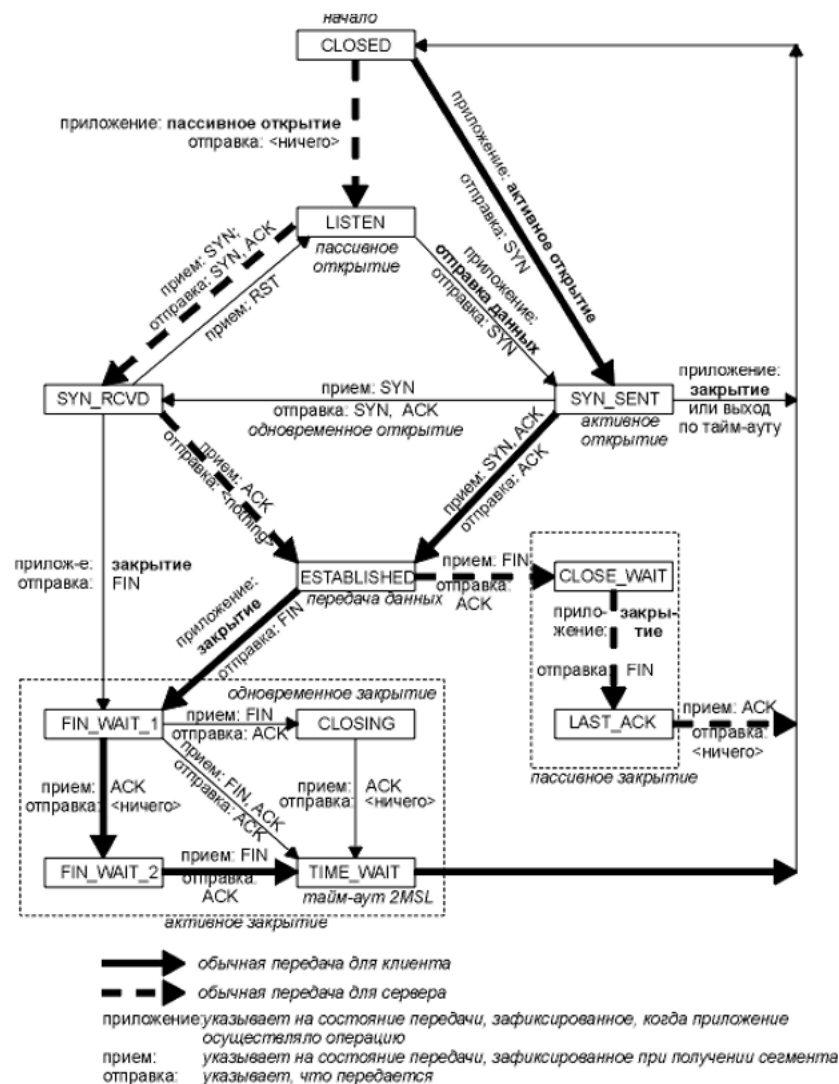
Читать билет [26](#).

TCP — это транспортный протокол, который нужен для надежной передачи данных между двумя компьютерами в интернете. В отличие от UDP, TCP работает по принципу «соединение сначала, потом передача». То есть перед тем как начать передавать данные, стороны сначала договариваются, и только после этого начинается обмен.

- Гарантируется доставка данных.
- Контроль порядка — в каком порядке отправили, в таком и получили.
- Контроль потока и перегрузки.

Про установление и закрытие TCP соединения также в [26 билете](#).

Автомат состояния TCP



хуй знает что добавить к этой няшной картиночке)

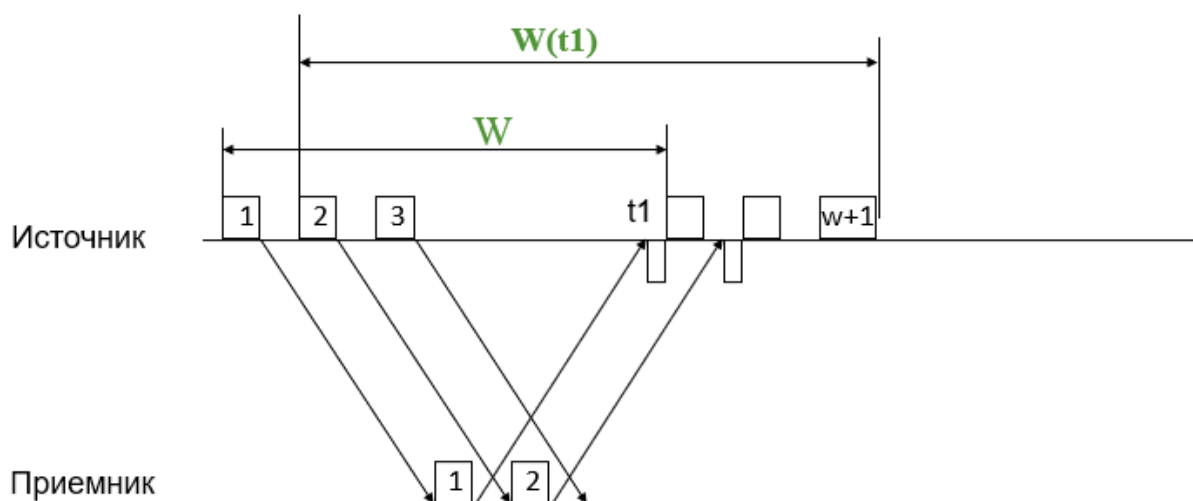
28 - Протокол TCP. Управление потоком. Скользящее окно.

ох полетела пизда по кочкам

Поток данных или просто **поток** — это данные, поступающие от приложений на вход протоколов TCP и UDP.

TCP реализует управление потоком, используя концепцию **окна**, которая применяется к количеству данных, которые могут быть ожидающими подтверждения в любой момент времени. Концепция окна позволяет *принимающему* хосту сообщать *отправителю*, сколько данных он может получить прямо *сейчас*, давая принимающему хосту способ замедлить или ускорить отправляющий хост. Получатель может перемещать размер окна вверх и вниз (*скользящее/динамическое окно*), чтобы изменить объем данных, который может отправить хост-отправитель.

Повторим картиночку:



Окно (W) – число байтов данных, которые отправитель может отправить без получения подтверждения.

Концепция скользящего окна (sliding window) заключается в том, что для повышения скорости передачи данных отправителю разрешается передать некоторое число пакетов, не дожидаясь прихода квитанций на эти пакеты.

Механизмы управления:

- окно приемника W_{rcvr} — управление от приемника: учет места в буфере, защита от перегрузки буфера приемника,

- окно насыщения CWND (Congestion Win) — внутренняя переменная источника, управление от источника: учет загрузки сети,
- порог медленного старта SSTRESH – внутренняя переменная,
- время ожидания ACK TimeOut.



Wrcvr – число байт, которое может принять приемник

$Wrcvr_A$ и $Wrcvr_B$ – разные сущности

$Wrcvr = var, Wrcvr \text{ м.б} = 0$

Изменяя размер окна можно влиять на загрузку сети

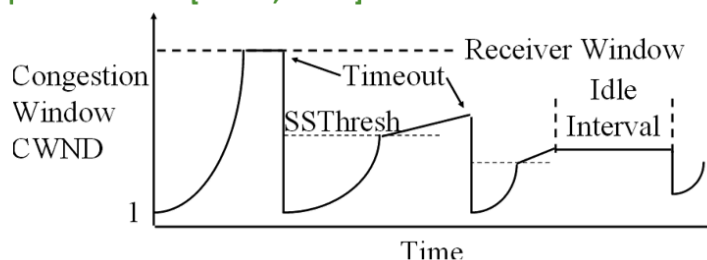
Если сеть загружена, размер окна уменьшается; если сеть свободна, размер окна увеличивается.

Вычисление CWND зависит от состояния TCP:

- SlowStart (SS) – начальная фаза, когда CWND увеличивается экспоненциально для быстрого набора скорости,
- Congestion Avoidance (CA) – более осторожное линейное увеличение CWND, когда сеть уже близка к насыщению,
- Fast Retransmission (FRT) – ускоренная повторная передача сегмента при получении нескольких дубликатов ACK,
- Fast Recovery (FRC) – механизм, позволяющий не сбрасывать CWND до минимума, а восстанавливаться быстрее после потерь.

SlowStart

- $CWND = 1$
- Каждый Ack = $CWND := CWND + 1$
- $Win = \min [CWND, Wrcvr]$
- **ВСЕГДА** $Win = \min [CWND, Wrcvr]$



Congestion Avoidance

- Потеря пакета – сигнал о насыщении
- 2 признака – TimeOut и дубликат Ack

1. SStart
 2. Если TimeOut, то $SSTRESH = 1/2 \text{ Win}$ (текущ.)
- $CWND = 1$ увеличение с кажд. Ack, но по-разному
 - Если $CWND < SSTRESH$, то SStart
 - Если $CWND > SSTRESH$, то CA, т.е. 1 сегмент за RTT

TimeOut

Timeout $\sim > 2RTT$

Сложность определения Таймаута:

$$RTT \sim \alpha * RTT_{old} + (1 - \alpha) * RTT_{new}$$

Каждое соединение – своя переменная RTT

- Постоянно уточняется: $RTT \sim \alpha * RTT_{old} + (1 - \alpha) * RTT_{new}$
- Timeout – мало – лишняя повторная передача
- Timeout – велико – длительный простой

Управление потоком

Для управления потоком TCP в первую очередь определяет количество сегментов данных, которое может принять устройство назначения. Заголовок TCP включает в себя 16-битное поле, которое называется **размером окна** – это количество байтов, которое устройство назначения сеанса TCP способно принять и обработать одновременно.

Исходный размер окна согласовывается во время запуска сеанса через трехстороннее рукопожатие между устройствами источника и назначения. После согласования исходное устройство должно ограничить количество сегментов данных, отправленных устройству назначения, в соответствии с размером окна. Только после того как исходное устройство получит подтверждение того, что сегменты данных получены, оно может продолжить отправку остальных данных в этом сеансе.

Во время задержки в получении подтверждения отправитель не отправляет дополнительных сегментов. В те периоды, когда сеть чрезвычайно загружена или ресурсы получающего узла на пределе, длительность задержки может увеличиться. По мере увеличения длительности задержки эффективная скорость передачи данных для этого сеанса снижается. Замедление передачи данных при каждом сеансе помогает уменьшить конфликт ресурсов на сетевом устройстве и устройстве назначения в случае запуска нескольких сеансов связи.

Протокол TCP использует размер окна, чтобы попытаться управлять скоростью передачи данных в соответствии с максимальным значением потока, которое поддерживается сетью и устройством назначения, одновременно с этим уменьшая потери данных и количество их повторных пересылок.

Сравнение TCP и UDP

	TCP	UDP
Размер заголовка, байт	20-60	8
Форма передачи данных	Поток	Датаграмма
Надежность	Да	Нет
Упорядоченность	Да	Нет
Контроль перегрузок	TCP Congestion Avoidance Algorithm	Нет
Тяжеловесность	Дополнительные 3 пакета для установки соединения	Никаких дополнительных пакетов не нужно
Применение	Там, где нужна надежность и упорядоченность WWW, e-mail, FTP, SSH	Там, где высокая нагрузка на сервер и потеря некоторых пакетов не критична DNS, DHCP, SNMP, голосовой и видео трафик, игры

29 - QUIC: Особенности, преимущества и недостатки.

QUIC (Quick UDP Internet Connections) – новый транспортный протокол, который был разработан компанией Google в 2013 году. Он был принят IETF и является основным транспортным протоколом HTTP3.

В модели OSI QUIC сочетает в себе функции уровней транспортный-прикладной.

Отличительные особенности

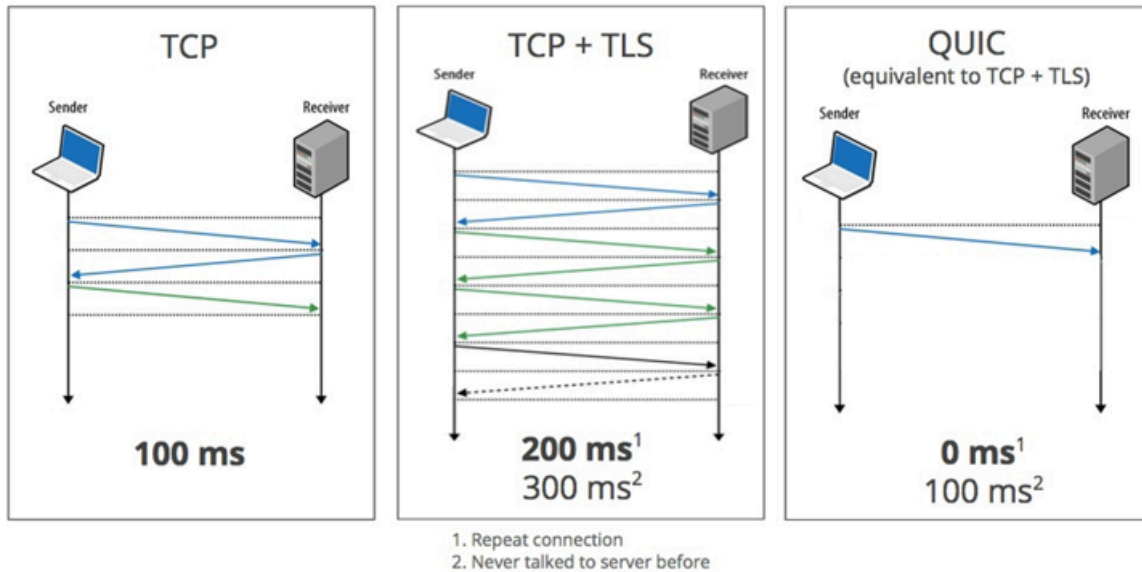
1. **Возможность миграции соединения.** Соединение идентифицируется 64 битным ID, который сохраняется при смене IP.
2. **Поддержка оптимизированных ACK:** поддержка 256 диапазонов NACK – эффективное подтверждение выборочных потерь без повторной отправки всего окна.
3. **Восстановление потерь:** две попытки передачи до TimeOut.
4. **Управление перегрузкой:** может использовать любой алгоритм управления окном и легко менять их без разрыва соединения.
5. **Многопоточность:** передача до 2^{16} независимых потоков данных через одно соединение.
6. **Приоритезация потоков:** приложение может задавать приоритеты потокам.
7. **Шифрование** (TLS 1.3 по умолчанию).
8. **QPACK** – сжатые заголовки, позволяющие использовать ранее полученную информацию.
9. **2 типа заголовков** – длинный и короткий. Длинные используются для установления соединения, в процессе передачи используются в основном короткие.
10. **Stream Frame** – возможность передачи нескольких потоков по одному соединению.
11. **Избыточная передача данных для восстановления** (может быть реализована).

Недостатки:

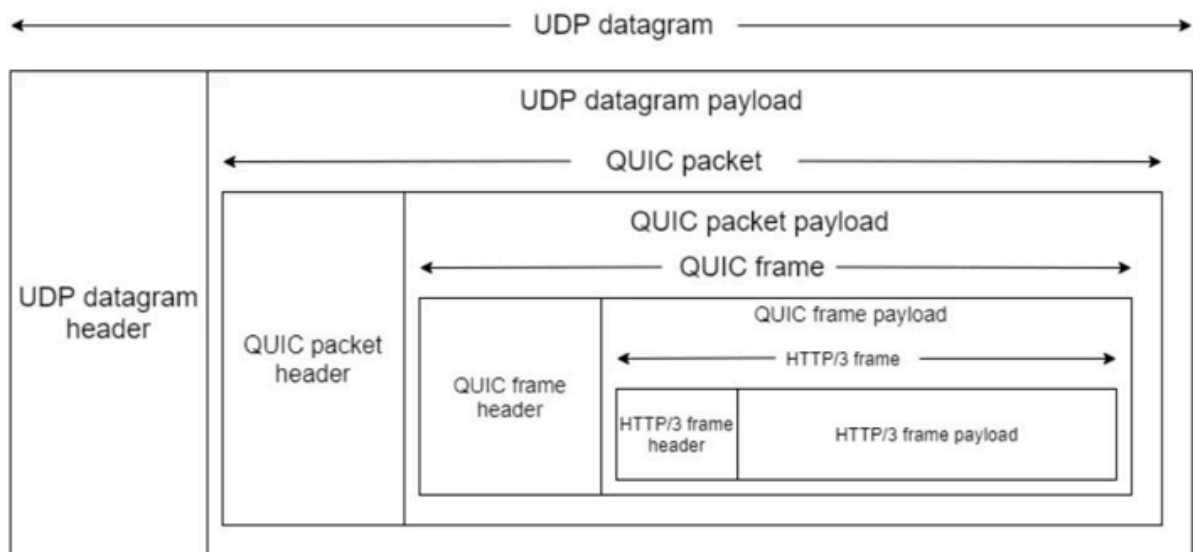
- сложность отладки из-за шифрования заголовков;
- сложность с оценкой качества сети для сторонних наблюдателей, включая администраторов и операторов сети;
- потребление большего объема оперативной памяти, чем TCP;

- высокая нагрузка на процессор на сервере, выполняется в контексте пользователя;
- не поддерживает фрагментацию на уровне IP – максимальный размер пакета ограничен MTU, большие данные дробятся на уровне приложения;
- блокируется почти всеми средствами контроля в России 😞

Разница во времени установления соединения Zero RTT Connection Establishment



30 - QUIC: Заголовки.



Long Header

Header Form	Fixed Bit	Long Packet Type	Type Specific bits	Version ID	DCID Len	DCID	SCID Len	SCID
1 bit	1 bit	2 bits	4 bits	32 bits	8 bits	0-160 bits	8 bits	0-160 bits

Short Header

Header Form	Fixed Bit	Spin bits	Reserved	Key Phase	P	DCID	Packet Number	Protected payload
1 bit	1 bit	1 bit	2bits	1 bit	2 bits	160 bits	P+8 bits	

Header Form (HF) – 1 бит – идентификатор типа заголовка.

- 1 => Long Header
- 0 => Short Header

Для длинного Long Header заголовка:

Fixed Bit (FB) – бит, указывающий на действительность пакета (1 – действительный, 0 – нет).

Long Packet Type (T) – 2 бита – тип длинного заголовка:

- 0x00 (Initial) – первый пакет клиента;
- 0x01 (0-RTT) – передача данных до завершения рукопожатия;
- 0x02 (Handshake) – пакеты для завершения установки защищенного соединения;
- 0x03 (Retry) – ответ сервера с токеном валидации.

Type-Specific Bits (S) – 4 бита – биты зависящие от типа длинного заголовка. Зависят от пакета.

Version ID (VID) – 32 бита для версии QUIC.

Destination Connection ID Length (DCID Len) – 8 бит – длина ID приемника.

Destination Connection ID (DCID) – 0-160 бит – сам ID приемника.

Source Connection ID Length (SCID Len) – 8 бит – длина ID источника.

Source Connection ID (SCID) – 0-160 бит – ID источника.

Для короткого Short Header заголовка:

Fixed Bit – всегда 1.

Spin Bit – бит мониторинга задержки. Копируется в ответах, позволяет измерять RTT без расшифровки.

Key Phase – бит, указывающий на то, какой набор ключей используется (установка бита – смена ключей).

Reserved – 2 бита – должны быть 0.

Packet Number – 8-32 бита – номер пакета.

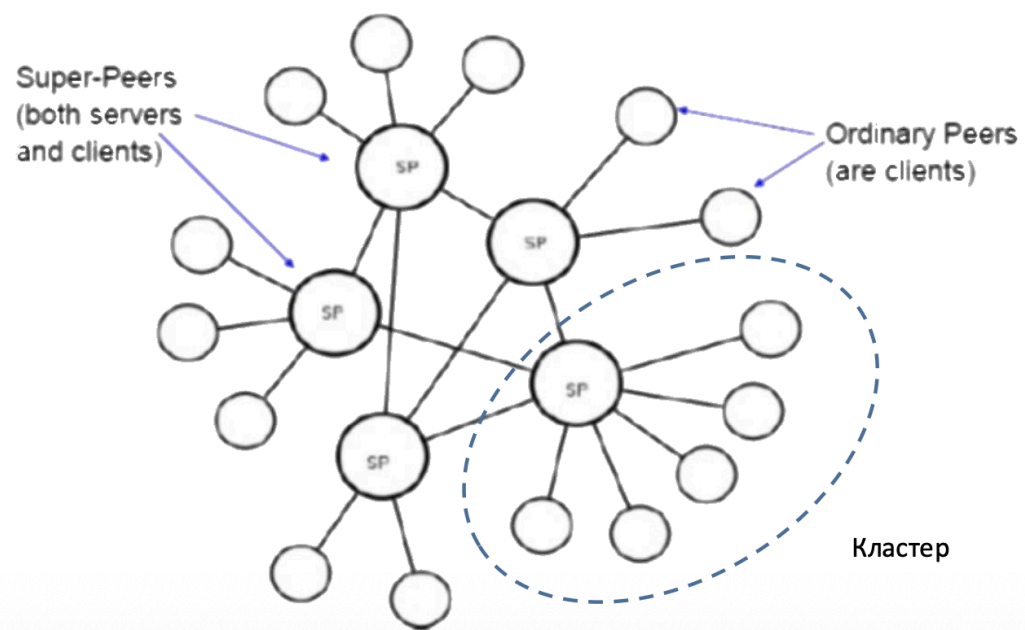
31 - Протоколы прикладного уровня. Модель “клиент-сервер”, Модель P2P. Примеры протоколов прикладного уровня.

Прикладной уровень (Application layer) является верхним уровнем в сетевых иерархиях моделей OSI и TCP/IP. Из особенностей прикладного уровня выделяют:

- Большое число протоколов
- Связь с конкретным приложением
- Модель “клиент-сервер” / одноранговые сети
- Протоколы (примеры):
 - HTTP — протокол передачи гипертекста
 - FTP — протокол передачи файлов
 - NFS (network file system) — протокол сетевой файловой системы для организации удаленного доступа к файловым ресурсам
 - SMTP (Simple Mail Transfer Protocol)
 - SNMP (Simple Network Management Protocol)
 - Telnet — текстовый терминальный интерфейс по сети
 - SSH (Secure Shell) — защищенный протокол безопасного удаленного соединения с обязательным шифрованием всего передаваемого трафика.
 - BitTorrent — всемирно известный прикладной P2P-протокол («битовый поток»), оптимизированный для однорангового обмена файлами.
- Служебные: DNS, LDAP, DHCP и др.
 - DNS (Domain Name System) — критически важная система доменных имен, осуществляющая трансляцию символьных имен сайтов в числовые IP-адреса.
 - DHCP (Dynamic Host Configuration Protocol) — протокол автоматической динамической конфигурации узлов (выдачи IP-адресов, масок подсетей и адресов шлюзов).
 - LDAP (Lightweight Directory Access Protocol) — облегченный протокол доступа к сетевым службам каталогов.
 - SNMP (Simple Network Management Protocol) — протокол, используемый администраторами для мониторинга и управления сетевыми устройствами.

Выделяют 2 типа архитектуры сетевых приложений:

1. **Централизованная модель «клиент-сервер» (Client-Server)** — это классическая архитектура построения сетевых приложений, основанная на строгом разграничении ролей между узлами.
 - a. **Клиент (Client)** — узел или процесс, который формирует и отправляет запрос на получение определенного сервиса или данных.
 - b. **Сервер (Server)** — выделенный узел, который принимает запросы, обрабатывает их и предоставляет клиентам соответствующий сервис.
 - c. Сервер обязан функционировать непрерывно (всегда находиться в режиме онлайн) и постоянно ожидать поступающие запросы.
 - d. Конечные клиенты не имеют возможности общаться друг с другом напрямую; все коммуникации осуществляются исключительно через центральный сервер.
2. **Одноранговая модель P2P (Peer-to-Peer).** В отличие от клиент-серверной схемы, P2P-архитектура предполагает децентрализацию и равноправие участников сети:
 - a. **Узлы (Пир / Peers)** общаются между собой непосредственно и напрямую, совмещая в себе функции как клиента, так и сервера.
 - b. В сети отсутствует жесткое требование к узлам постоянно находиться онлайн (пиры могут произвольно подключаться и отключаться).
 - c. Модель обладает превосходной масштабируемостью (capacity).
 - d. Характеризуется симметричным распределением трафика (узлы запрашивают и отдают данные в одинаковой степени).
 - e. В одноранговых сетях значительно усложнены механизмы обеспечения безопасности и аутентификации пользователей.
 - f. Существуют также гибридные, или суперпиринговые сети (например, Skype), в которых для координации обычных клиентов выделяются специальные узлы-кластеры — суперпиры (Super-Peers).



32 - Протокол HTTP. История от 0.9 до 3.

HTTP (HyperText Transfer Protocol) — это протокол прикладного уровня, предназначенный для передачи гипертекста и любых других структурированных данных в распределенных информационных системах. Он функционирует по классической схеме «запрос-ответ» в рамках модели «клиент-сервер».

HTTP/0.9 (1991 год)

Это первоначальный вариант протокола, разработанный Тимом Бернерсом-Ли. Его возможности были крайне ограничены:

- **Синтаксис:** Поддерживался всего один метод — **GET**. Запрос состоял из одной строки, содержащей метод и путь к ресурсу (например: **GET /index.html**).
- **Формат данных:** Сервер мог возвращать исключительно текст в формате HTML (plain text/HTML).
- **Отсутствие метаданных:** Не существовало понятия HTTP-заголовков, кодов состояния (Status Codes) и версионирования. Если страница была найдена, она передавалась, если нет — передавался текст ошибки внутри HTML.
- **Управление соединением:** После отправки ответа TCP-соединение немедленно закрывалось сервером.

HTTP/1.0 (1996 год, RFC 1945)

Протокол зафиксировал стихийное развитие веба первой половины 90-х годов:

- **Версионирование:** В строке запроса стала явно указываться версия (например, **GET /index.html HTTP/1.0**).
- **HTTP-заголовки:** Введены заголовки запросов и ответов. Это позволило передавать метаданные: тип кодировки, параметры авторизации, информацию о клиенте и сервере.
- **Коды состояния:** Появилась строка статуса ответа (например, **200 OK, 404 Not Found**), что позволило браузерам корректно обрабатывать логику ошибок.
- **Мультимедийность:** Благодаря внедрению заголовка **Content-Type** (на основе стандартов MIME) веб-страницы получили возможность передавать не только HTML-текст, но и графику, аудио, видео и бинарные файлы.

- **Методы:** Добавлены методы **POST** (передача данных на сервер) и **HEAD** (запрос только заголовков без тела ответа).
- **Главная проблема:** Для загрузки каждого отдельного элемента (картинки, скрипта) требовалось открывать новое TCP-соединение, что приводило к огромным накладным расходам на фазе трехэтапного рукопожатия TCP (Three-way handshake) и механизма медленного старта (Slow Start). Неофициально использовался заголовок **Connection: Keep-Alive** для удержания соединения, но он не был стандартом.

HTTP/1.1 (1997/1999 годы, RFC 2068, RFC 2616)

Эта версия оставалась доминирующей на протяжении более чем 15 лет (в 2022 году перевыпущена в RFC 9112). Ключевые нововведения:

- **Постоянные соединения (Persistent Connections):** По умолчанию соединения стали долгоживущими (**Connection: keep-alive**). На одном TCP-соединении теперь можно было отправлять цепочку запросов без его разрыва.
- **Конвейеризация (Pipelining):** Возможность отправлять следующий запрос, не дожидаясь ответа на предыдущий.
- **Проблема HOL (Head-of-Line Blocking) на прикладном уровне:** Конвейеризация не решила проблему полностью — сервер обязан возвращать ответы строго в порядке поступления запросов. Если первый «тяжелый» запрос (например, генерация сложного скрипта) зависал, все последующие ответы блокировались. На практике браузеры начали открывать до 6 параллельных TCP-соединений к одному домену.
- **Обязательный заголовок Host:** Позволил организовывать виртуальный хостинг (размещение множества сайтов с разными доменными именами на одном IP-адресе).
- **Дополнительные методы:** **PUT, DELETE, OPTIONS, PATCH.**
- **Потоковая передача данных:** Механизм **Transfer-Encoding: chunked** позволил серверу отдавать данные по частям, когда их итоговый размер заранее неизвестен.

HTTP/2 (2015 год, RFC 7540 / обновление RFC 9113)

Создан на базе разработанного Google экспериментального протокола **SPDY**. Архитектура прикладных семантик (методы,

заголовки, коды ответов) осталась прежней, но радикально изменился способ передачи данных по сети:

- **Бинарный фрейминг (Binary Framing):** Вместо текстового формата данные разбиваются на мелкие бинарные кадры (фреймы).
- **Полноценное мультиплексирование (Multiplexing):** В рамках **одного** TCP-соединения клиент и сервер могут одновременно (чередую фреймы) передавать множество запросов и ответов (поток). Это полностью ликвидировало проблему Head-of-Line blocking на прикладном уровне.
- **Сжатие заголовков (HPACK):** Заголовки HTTP часто дублируются. HTTP/2 использует алгоритм HPACK, создающий статическую и динамическую таблицы на обеих сторонах, передавая вместо громоздких текстовых строк короткие индексы. Это экономит до 85% объема метаданных.
- **Приоритезация потоков:** Клиент может указать, какие ресурсы (например, CSS-стили) критически важны для рендеринга страницы и должны передаваться в первую очередь.
- **Server Push:** Сервер может проактивно отправить клиенту ресурсы в кэш (например, картинки к статье), которые клиент еще не успел запросить.
- **Главная проблема HTTP/2:** Протокол по-прежнему работает поверх **TCP**. Если на физическом или канальном уровне теряется хотя бы один IP-пакет, механизм TCP останавливает передачу *всего* буфера до тех пор, пока потерянный пакет не будет передан повторно. Таким образом, **проблема HOL-блокировки сместилась на транспортный уровень**, парализуя все мультиплексированные потоки сразу.

HTTP/3 (2022 год, RFC 9114)

Разработан для устранения транспортных ограничений TCP на нестабильных (мобильных) сетях. Главная революция — полный отказ от TCP.

- **Переход на UDP и QUIC:** Данные HTTP/3 инкапсулируются в пакеты протокола **QUIC**, который, в свою очередь, работает поверх датаграмм **UDP**.
- **Устранение HOL-блокировки на транспортном уровне:** QUIC реализует независимые потоки данных прямо на уровне

транспорта. Если один пакет в рамках одного потока теряется, все остальные потоки продолжают передаваться без задержек.

- **Быстрое установление соединения (0-RTT / 1-RTT Handshake):** За счет интеграции криптографического протокола TLS 1.3 непосредственно в транспортный заголовок QUIC, установление защищенного соединения занимает всего 1 раунд (1 Round-Trip Time) вместо 2-3 раундов в связке TCP+TLS. При повторном соединении возможна отправка данных вообще без задержки (0-RTT).
- **Миграция соединения (Connection Migration):** Идентификация сессии происходит не по классическому сокету (IP-отправителя, Порт-отправителя, IP-получателя, Порт-получателя), а по уникальному ID соединения (**Connection ID**). Если пользователь переключается с мобильного LTE на домашний Wi-Fi (IP-адрес меняется), скачивание файлов или видеозвонок не прерываются.

33 - Протокол HTTP. Структура, методы, коды состояний.

HTTP (HyperText Transfer Protocol) — это протокол прикладного уровня, предназначенный для передачи гипертекста и любых других структурированных данных в распределенных информационных системах. Он функционирует по классической схеме «запрос-ответ» в рамках модели «клиент-сервер».

Структура сообщений:

1. **Стартовая строка (Start-line):** Живет на первой позиции; всегда обязательна.
 - a. *В запросе (Request Line):* Состоит из трех элементов: **Метод** (тип запроса заглавными буквами), запрашиваемый **URI** (путь к документу) и **Версия протокола**. Пример: **GET /wiki/HTTP HTTP/1.1**.
 - b. *В ответе (Status Line):* Состоит из трех элементов: **Версия протокола**, числовой **Код состояния** (Status Code) и текстовое **Пояснение** (Reason Phrase), которое носит справочный характер для пользователя и не влияет на логику обработки. Пример: **HTTP/1.1 200 OK**.
2. **Заголовки (Headers):** Идут сразу за стартовой строкой. Представляют собой гибкий набор метаданных в формате текстовых пар **Параметр: значение**, разделенных двоеточием. Заголовки разделяют на 4 типа:
 - a. *General Headers (Общие):* Могут включаться в любые сообщения (и в запросы, и в ответы).
 - b. *Request Headers (Запроса):* Передаются исключительно в пакетах от клиента к серверу.
 - c. *Response Headers (Ответа):* Присутствуют строго в пакетах ответов от сервера.
 - d. *Entity Headers (Сущности):* Описывают свойства конкретного контента (сущности) внутри сообщения.
 - e. *Пользовательские заголовки:* При нехватке стандартных полей разработчики могут вводить свои метаданные, традиционно используя префикс **X-** или любой другой индивидуальный префикс (например, **MS-Echo-Request**).
3. **Пустая строка:** Обязательный технический разделитель (символы перевода строки), сигнализирующий о завершении блока заголовков.
4. **Тело сообщения (Message Body):** Используется для непосредственной передачи данных (тела объекта), тип которого

указан в метаданных. Сообщения со статусными кодами **1xx**, **204** и **304** принципиально **не содержат тела**, во всех остальных случаях тело присутствует, хотя его длина может быть нулевой.

Методы запросов:

- **GET** — Запрашивает содержимое (тело) указанного ресурса. Также может инициировать запуск какого-либо процесса на сервере (тогда в ответе возвращается статус процесса).
- **HEAD** — Полностью аналогичен методу **GET**, но сервер возвращает исключительно стартовую строку и заголовки метаданных, полностью **без тела** сообщения. Используется для проверки доступности ресурса и валидации кэша.
- **POST** — Передает пользовательские или системные данные на сервер, инкапсулируя их внутри тела запроса. Часто применяется для отправки веб-форм или создания новых сущностей.
- **PUT** — Загружает (замещает) данные на сервер по указанному целевому URI.
- **PATCH** — По аналогии с PUT отправляет данные на сервер, но предназначен не для полной замены, а для частичного обновления (модификации) ресурса.
- **DELETE** — Удаляет указанный в URI целевой ресурс на стороне сервера.
- **TRACE** — Запускает эхо-тестирование. Сервер возвращает клиенту обратно тот же самый запрос, который получил, что позволяет увидеть изменения и заголовки, добавленные промежуточными узлами (прокси-серверами, шлюзами).
- **CONNECT** — Предписывает прокси-серверу перевести установленное соединение в режим прозрачного TCP-туннеля (активно применяется для организации HTTPS-сессий через прокси).
- **OPTIONS** — Запрашивает у сервера параметры конфигурации конкретного ресурса или перечень поддерживаемых сервером методов (возможностей).

Коды состояний

Код состояния возвращается сервером в стартовой строке ответа. Это трехзначное число, по которому клиент (браузер) определяет логику дальнейшего поведения и формат содержимого пакета. Все коды делятся на 5 классов по первой цифре:

1. **1xx — Informational (Информационные):** Уведомляют о промежуточных этапах выполнения запроса.
2. **2xx — Success (Успешно):** Запрос принят и корректно обработан сервером.
 - a. **200 OK** — Действие успешно выполнено, запрошенный ресурс передан.
 - b. **201 Created** — Запрос выполнен успешно, в результате чего на сервере создан новый ресурс.
 - c. **206 Partial Content** — Запрос успешно обработан, но сервер выслал только частичные данные (используется при докачке файлов или потоковом видео).
3. **3xx — Redirection (Перенаправление):** Клиенту необходимо предпринять дополнительные действия (сменить адрес).
 - a. **301 Moved Permanently** — Запрашиваемый ресурс окончательно перенесен на новый постоянный URI.
 - b. **302 Found (Moved Temporarily)** — Ресурс временно перенесен на другой адрес. Часто применяется для балансировки нагрузки, но иногда может обрабатываться некорректно.
 - c. **304 Not Modified** — Ответ на запрос GET с условием; сообщает клиенту, что запрашиваемый ресурс не менялся с указанного момента, и его можно взять из локального кэша клиента.
4. **4xx — Client Error (Ошибка клиента):** Запрос содержит синтаксические ошибки или не может быть выполнен по вине пользователя.
 - a. **401 Unauthorized** — Для доступа к ресурсу требуется обязательная аутентификация (ввод логина/пароля).
 - b. **403 Forbidden** — Запрос понятен, но у клиента категорически нет прав на совершение действий с данным ресурсом.
 - c. **404 Not Found** — Сервер не может найти запрашиваемый ресурс по указанному пути.
5. **5xx — Server Error (Ошибка сервера):** Ошибка возникла на стороне сервера при корректно сформированном запросе.
 - a. **500 Internal Server Error** — Любая непредвиденная внутренняя ошибка сервера (например, падение бэкенд-скрипта).

- b. **502 Bad Gateway** — Промежуточный сервер (шлюз или прокси) получил некорректный (ошибочный) ответ от вышележащего сервера, к которому обращался.
- c. **503 Service Unavailable** — Сервер временно не имеет технической возможности обрабатывать входящие запросы по техническим причинам (перегрузка трафиком, проведение регламентных работ).

34 - Протоколы Telnet (SSH) и FTP.

Telnet

Текстовый терминальный интерфейс, предназначенный для организации удаленного управления узлами сети и работы с командной строкой компьютеров под управлением различных операционных систем.

Клиент выступает в роли виртуального «удаленного терминала» строчного типа (ANSI). Над Telnet либо программа терминала и пользователь, либо процесс на сервере.

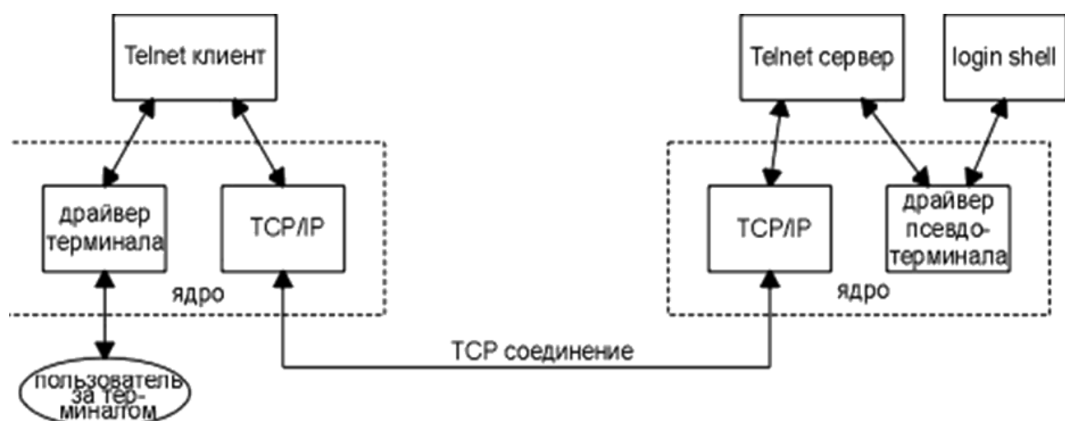
Использует одно постоянное TCP-соединение; серверный процесс по умолчанию ожидает входящие запросы на порту **23**.

При подключении пользователь получает очень большие права в системе. Однако протокол Telnet передает данные в открытом текстовом виде без шифрации (включая логины и пароли). Из-за критической уязвимости к перехвату трафика в подавляющем большинстве современных систем Telnet принудительно отключен.

Протокол SSH (Secure Shell) — современная и полностью защищенная альтернатива Telnet для удаленного администрирования.

Является протоколом безопасного соединения и выполняет **обязательное шифрование** всей передаваемой информации (включая аутентификационные данные).

Транспортный уровень: Работает поверх TCP через выделенный стандартный порт 22.



Взаимодействие Telnet-клиента и Telnet-сервера

FTP

Специализированный протокол прикладного уровня, оптимизированный для обмена файлами между клиентом и сервером.

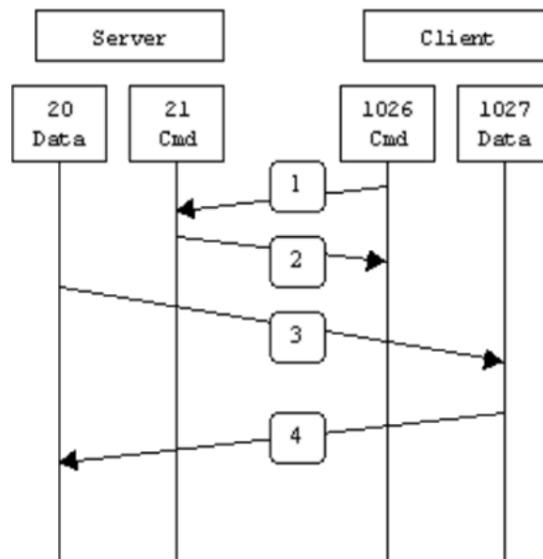
Архитектура соединений (двухканальная модель): Главная техническая особенность FTP заключается в строгом разделении каналов управления и каналов данных. В рамках одной сессии параллельно открываются **два отдельных TCP-соединения**:

1. **Управляющее соединение (Control Connection):**
Устанавливается на порт **21** сервера и остается открытым (постоянным) на протяжении всего времени работы пользователя. Оно функционирует по принципу протокола Telnet — через него клиент передает текстовые команды, а сервер осуществляет контроль состояния сессии (проверка имени пользователя, пароля, фиксация текущего каталога).
2. **Соединение для передачи данных (Data Connection):**
Краткосрочное динамическое соединение. Оно создается заново для выполнения каждого отдельного действия — будь то непосредственное скачивание/загрузка файла или просто получение списка файлов в директории (команда `dir / ls`). Как только передача контента завершена, это соединение закрывается.

Режим работы выбирается и устанавливается клиентом. Управляющий канал в обоих случаях всегда привязан к 21-му порту сервера.

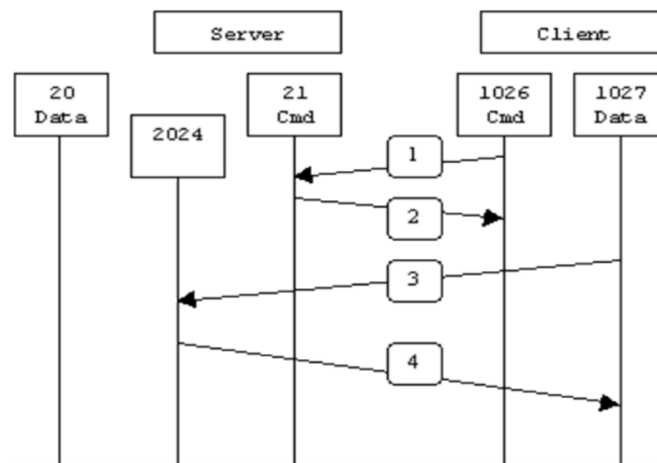
Активный режим (Active FTP):

- Сервер жестко использует закрепленные порты: порт **21** — для команд, порт **20** — для передачи данных.
- Клиент открывает на своей стороне случайный динамический порт, начинает его слушать и по управляющему каналу сообщает серверу номер этого порта.
- Сервер со своего порта 20 самостоятельно **инициирует встречное TCP-соединение** на указанный клиентом порт.
- *Проблема:* Если клиент защищен сетевым брандмауэром (Firewall) или находится за NAT, входящее встречное соединение от сервера будет заблокировано.



Пассивный режим (Passive FTP):

- Сервер использует порт **21** для команд, а для передачи данных выделяет на своей стороне случайный динамический порт из верхнего диапазона (> 1024).
- Сервер сообщает клиенту номер этого динамического порта через управляющий канал.
- Клиент самостоятельно инициирует установление **обоих TCP-соединений** (отправляя пакет с флагом SYN): как управляющего (на порт 21), так и соединения данных (на динамический порт сервера).
- *Преимущество:* Так как все запросы исходят изнутри клиентской сети наружу, пассивный режим беспрепятственно работает через Firewall и NAT на стороне клиента.



35 - Служба DNS, Пространство символьных имен.

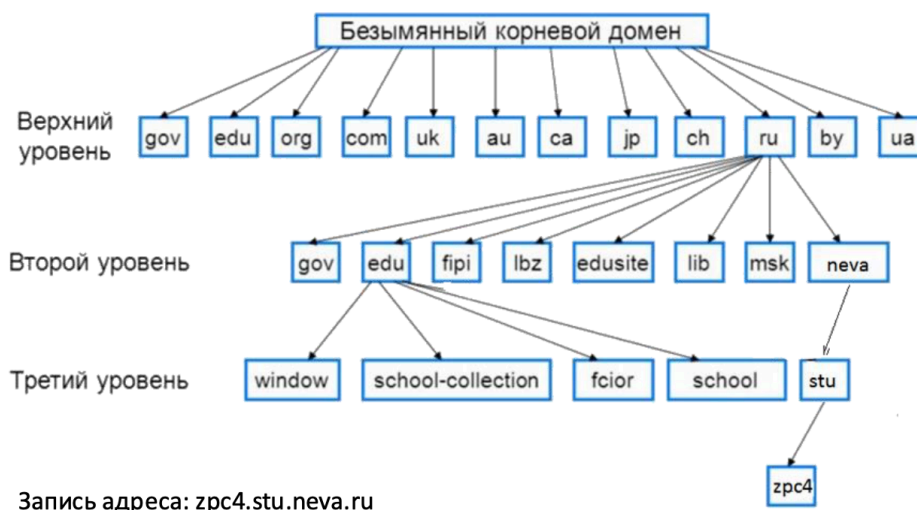
DNS (Domain Name System) — это служебная распределенная база данных и одноименный протокол прикладного уровня, предназначенные для автоматического перевода (разрешения) символьных доменных имен хостов в числовые IP-адреса и наоборот.

Сетевому оборудованию на сетевом уровне необходимы фиксированные двоичные IP-адреса для маршрутизации пакетов, однако человеку намного удобнее оперировать легко запоминающимися символьными строками. Эту трансляцию и обеспечивает DNS.

Ключевые свойства архитектуры DNS:

- Иерархическая система имен.
- **Разделение административной ответственности:** управление разными сегментами пространства имен распределено между множеством независимых организаций и администраторов.
- **Независимость имени от местоположения:** при физическом переносе сервера на другую площадку и смене его IP-адреса символьное имя хоста может оставаться неизменным.

Пространство символьных имен (Доменная иерархия)



Иерархическая система имен и протокол DNS

Пространство имен DNS представляет собой глобальную распределенную древовидную структуру.

- **Корень дерева (Root):** Находится на вершине иерархии, обозначается символом точки (.). В повседневных адресах сайтов в браузере конечная точка опускается, но синтаксически она подразумевается (например, `example.com.`). Корень обслуживается корневыми DNS-серверами.
- **Домены верхнего (первого) уровня (TLD - Top-Level Domains):** Ветви, идущие непосредственно от корня. Они делятся на две основные группы:
 - *Национальные домены (ccTLD):* жестко привязаны к государствам (например, `.ru`, `.su`).
 - *Общие тематические домены (gTLD):* отражают тип организации (например, `.com` — коммерческие, `.org` — некоммерческие, `.edu` — образовательные).
- **Домены второго и последующих уровней:** Регистрируются и настраиваются внутри родительских зон (например, `spbstu.ru`, `yandex.ru`).
- **Полное доменное имя хоста (FQDN - Fully Qualified Domain Name):** Полный путь от конкретного узла вверх до корня дерева, где имена всех уровней разделяются точками. Чтение иерархии в имени хоста всегда происходит **справа налево**.
 - *Пример:* В записи `zpc4.stu.neva.ru`: `.ru` — домен 1-го уровня, `neva.ru` — домен 2-го уровня, `stu.neva.ru` — домен 3-го уровня, а `zpc4` — имя конкретного физического компьютера (хоста) в этой подсети.

Принцип хранения данных на DNS-серверах

Единого центра хранения всей базы данных интернета не существует. Вместо этого за каждый домен отвечает свой выделенный DNS-сервер, который хранит:

1. **Прямую зону:** отображение «символьное имя – IP-адрес» для узлов, находящихся на его уровне ответственности.
2. **Обратную зону (`in-addr.arpa`):** специальную зону, осуществляющую обратное отображение «числовой IP-адрес — символьное имя хоста».
3. **Ссылки на дочерние поддомены:** указатели (делегирующие) на DNS-серверы нижележащих уровней, которым передана административная ответственность за их ветки.

Процедура разрешения имен (Выполнение запроса)

Когда приложению требуется узнать IP-адрес по символическому имени хоста, процесс проходит следующие стадии:

1. **DNS-клиент (Resolver)** на стороне операционной системы пользователя формирует пакет-запрос и отправляет его на локальный DNS-сервер (как правило, это DNS-сервер интернет-провайдера или корпоративной сети).
2. Алгоритм обработки запроса может выполняться в двух режимах:
 - **Рекурсивный режим:** Локальный DNS-сервер берет всю работу на себя. Если в его локальном кэше нет нужной записи, он самостоятельно последовательно опрашивает другие серверы вплоть до получения финального ответа и возвращает клиенту готовый IP-адрес.
 - **Итеративный (нерекурсивный) режим:** Сервер, к которому обратились, если не знает точного ответа, возвращает клиенту ссылку (рекомендацию) на другой, более авторитетный DNS-сервер, стоящий выше или ниже по иерархии (например, корневой сервер отправляет к серверу зоны **.ru**, а тот — к серверу конкретного домена).

Ресурсные записи DNS

База данных DNS оперирует структурированными кортежами данных, называемыми ресурсными записями, имеющими следующий формат: **(NAME, TYPE, VALUE, PRIORITY, TTL)**

- **NAME:** Доменное имя хоста, к которому относится запись.
- **TYPE:** Тип ресурсной записи, определяющий назначение и формат поля VALUE.
- **VALUE:** Непосредственно полезные данные (IP-адрес, имя сервера и т.д.).
- **PRIORITY:** Параметр приоритета (используется преимущественно для балансировки или резервирования в почтовых записях).
- **TTL (Time to Live):** Время жизни записи в секундах. Определяет, на какой срок промежуточные прокси-серверы и клиенты могут заносить данную запись в свой локальный кэш без выполнения повторных запросов к авторитетному источнику.

Основные типы записей:

- **A (Address):** Предоставляет IPv4-адрес, соответствующий указанному доменному имени.
- **AAAA (Quad-A):** Предоставляет IPv6-адрес для данного доменного имени.
- **MX (Mail Exchange):** Содержит имя почтового сервера, уполномоченного принимать и обрабатывать электронную почту для этого домена.
- **CNAME (Canonical Name):** Задаёт текстовый синоним (алиас) описываемого домена. Любые запросы по этому синониму автоматически перенаправляются на адрес основного (канонического) доменного имени хоста.
- **NS (Name Server):** Хранит доменные имена DNS-серверов, которые являются авторитетными (первоисточниками) и обслуживают данную доменную зону.

Структура сообщений DNS и флаги заголовка

Трафик DNS по умолчанию инкапсулируется в дейтаграммы транспортного протокола UDP через порт 53. В заголовке пакета передается уникальный идентификатор запроса (целое число, которое дублируется в ответе для сопоставления пакетов) и счетчики количества записей в четырех полях пакета (Запросы, Ответы, Авторитетные серверы, Дополнительные ответы).

Критически важным является поле флагов заголовка, управляющее логикой работы службы:

- **QR (Query/Response):** Бит типа операции. 0 — запрос, 1 — ответ.
- **OPCODE:** 4 бита, кодирующие тип запроса (в повседневной практике всегда равен 0 — стандартный запрос).
- **AA (Authoritative Answer):** Указывается сервером в ответе. 1 означает, что ответивший DNS-сервер сам является авторитетным первоисточником для запрашиваемой зоны.
- **TC (Truncated):** Бит обрезания пакета. Если ответ превысил максимальный объем стандартной UDP-датаграммы, флаг выставляется в 1, заставляя клиента переключиться на протокол TCP для повторного полноценного запроса.
- **RD (Recursion Desired):** Передается клиентом в запросе. Выставленный в 1 флаг означает требование к серверу выполнить поиск в рекурсивном режиме.

- **RA (Recursion Available):** Передается сервером в ответе. **1** подтверждает, что данный сервер технически умеет работать в рекурсивном режиме.
- **Z:** Зарезервированный неиспользуемый бит.
- **RCODE (Response Code):** Последние 4 бита — статус выполнения операции. Значение **0** указывает на успешное завершение без ошибок, любые другие коды сообщают о характере проблемы (например, **NXDOMAIN** — домен не существует).

Для ручного тестирования, проверки ресурсных записей и отправки диагностических запросов к конкретным серверам используется стандартная консольная системная утилита **nslookup**.

```
C:\Program Files\Far>nslookup www.neva.ru
Server: ns.stu.neva.ru
Address: 194.85.96.53

Name: www.neva.ru
Address: 194.85.4.145

C:\Program Files\Far>nslookup 195.208.113.137
Server: ns.stu.neva.ru
Address: 194.85.96.53

Name: lpc7.stu.neva.ru
Address: 195.208.113.137

C:\Program Files\Far>nslookup www.google.com
Server: ns.stu.neva.ru
Address: 194.85.96.53

Non-authoritative answer:
Name: www.l.google.com
Addresses: 216.239.59.99, 216.239.59.103, 216.239.59.104, 216.239.59.147
Aliases: www.google.com
```


36 - Протоколы электронной почты, программы Ping, Traceroute/Tracert.

Email address: почтовый_ящик@почтовый_домен

SMTP (Simple Mail Transfer Protocol)

SMTP — протокол отправки электронной почты.

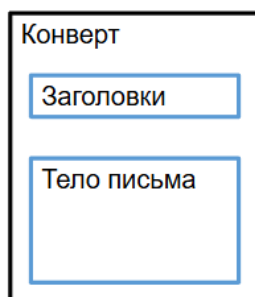
Основные функции:

- передача писем от клиента к почтовому серверу;
- передача писем между почтовыми серверами;
- работа по принципу «отправитель → сервер → сервер → получатель».

Стандартные порты:

- 25 — передача между серверами;
- 587 — отправка почты клиентом;
- 465 — защищённое соединение SMTP over SSL/TLS.

Формат письма



Команда	Назначение	Пример
HELO	Установка соединения	HELO example.com
MAIL	Адрес отправителя	MAIL FROM sender@example.com
RCPT	Адрес получателя	RCPT TO recipient@mail.ru
DATA	Передача письма	DATA
QUIT	Выход	QUIT

POP3 (Post Office Protocol v3)

POP3 — протокол получения электронной почты.

Особенности:

- загружает письма с сервера на компьютер пользователя;
- после загрузки письма обычно удаляются с сервера;
- удобен при работе с одного устройства.

Стандартный порт:

- 110;
- 995 — защищённое соединение.

IMAP (Internet Message Access Protocol)

IMAP — протокол доступа к электронной почте на сервере.

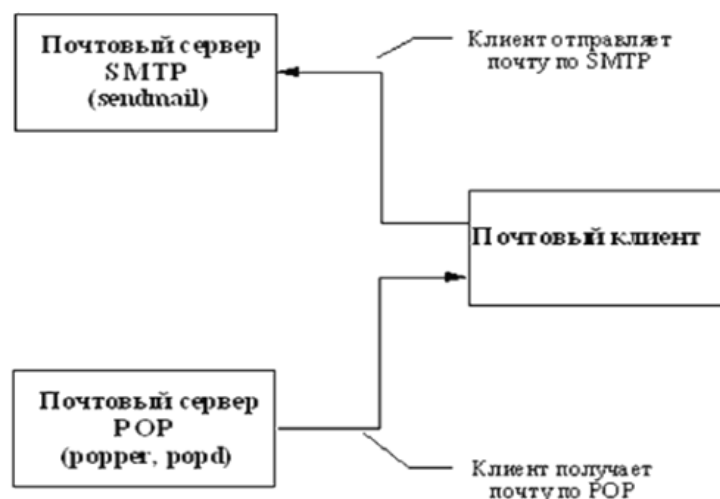
Особенности:

- письма хранятся на сервере;
- возможна работа с нескольких устройств;
- синхронизируются папки, статусы сообщений и структура почтового ящика.

Стандартный порт:

- 143;
- 993 — защищённое соединение.

Сервер исходящей почты и сервер входящей почты могут находиться на разных узлах.



Программа Ping

Ping — утилита диагностики сети, предназначенная для проверки доступности узла и оценки качества соединения.

Работает на основе протокола **ICMP**.

Принцип работы

1. Отправляется сообщение **ICMP Echo Request**.
2. Узел-получатель отвечает сообщением **ICMP Echo Reply**.
3. Измеряется время прохождения пакета.

Возможности Ping

- проверка доступности узла;

- измерение времени отклика (RTT);
- определение потерь пакетов;
- диагностика сетевых проблем.

Пример команды:

ping 8.8.8.8

Основные параметры

- время отклика (мс);
- количество отправленных и полученных пакетов;
- процент потерь пакетов.



Программа Traceroute (Linux) / Tracert (Windows)

Traceroute (tracert) — утилита для определения маршрута прохождения пакетов от отправителя до получателя.

Работа основана на поле **TTL (Time To Live)** IP-пакета.

Принцип работы

1. Отправляется пакет с TTL = 1.
2. Первый маршрутизатор уменьшает TTL до нуля и отправляет сообщение ICMP Time Exceeded.
3. Затем отправляется пакет с TTL = 2.
4. Процесс повторяется до достижения конечного узла.

В результате определяется последовательность всех маршрутизаторов на пути следования пакета.

Возможности Traceroute

- определение маршрута передачи данных;
- обнаружение проблемного участка сети;
- измерение задержек на каждом промежуточном узле;

- диагностика сетевых неисправностей.

Пример команды:

tracert google.com

37 - Технологии работы с трафиком. Отличие VPN от проху.

Сетевой трафик — это поток данных, передаваемых между устройствами сети.

Основные технологии работы с трафиком

1. Маршрутизация

Маршрутизация — процесс выбора пути передачи пакетов между сетями.

Выполняется маршрутизаторами на основе таблиц маршрутизации и специальных протоколов.

Назначение:

- доставка пакетов к получателю;
- выбор оптимального маршрута;
- объединение различных сетей.

2. Фильтрация трафика

Фильтрация позволяет разрешать или запрещать передачу данных по определённым правилам.

Используется для:

- защиты сети;
- ограничения доступа;
- блокировки нежелательного трафика.

Обычно реализуется межсетевыми экранами (Firewall).

3. Трансляция сетевых адресов (NAT)

NAT (Network Address Translation) — технология преобразования IP-адресов.

Позволяет:

- использовать один внешний IP-адрес для нескольких устройств;
- экономить адресное пространство IPv4;
- скрывать внутреннюю структуру сети.

4. QoS (Quality of Service)

QoS — механизм управления качеством обслуживания.

Позволяет:

- назначать приоритеты различным видам трафика;
- уменьшать задержки;
- обеспечивать стабильную работу голосовой и видеосвязи.

5. Proxy-сервер

Proxy (прокси-сервер) — промежуточный сервер между клиентом и внешними ресурсами.

Принцип работы:

1. Клиент отправляет запрос прокси-серверу.
2. Прокси обращается к нужному ресурсу.
3. Полученный ответ передаётся клиенту.

Функции:

- сокрытие IP-адреса пользователя;
- фильтрация запросов;
- кэширование данных;
- контроль доступа.

Существует три основных типа прокси-серверов:

- HTTP-прокси – они обслуживают только веб-страницы. Весь трафик браузера с настроенным прокси этого типа будет проходить через HTTP-прокси-сервер.
- Прокси SOCKS – они уже обрабатывают все виды трафика (а не исключительно веб), но все же работают только на уровне приложений.
- Прозрачные прокси – это различные прокси, о существовании которых пользователи обычно не знают. Отели и кафе используют их для аутентификации пользователей общедоступного Wi-Fi, а компании и частные пользователи – для экономии трафика.

6. VPN

VPN (Virtual Private Network) — технология создания защищённого логического канала связи поверх публичной сети.

Принцип работы:

1. Между клиентом и VPN-сервером создаётся защищённый туннель.

2. Передаваемые данные шифруются.
3. Весь сетевой трафик проходит через VPN-сервер.

Преимущества:

- защита передаваемых данных;
- скрытие реального IP-адреса;
- безопасная работа через публичные сети.

Сравнение проху и vpn

vpn	проху
Передаёт весь сетевой трафик	Обычно работает только для отдельных приложений
Использует шифрование данных	Чаще всего не шифрует трафик
Обеспечивает высокий уровень безопасности	Основная цель – посредничество и фильтрация
Создает защищенный туннель	Перенаправляет запросы через промежуточный сервер
Работает на сетевом уровне	Работает на прикладном уровне

VPN так же, как и прокси-сервер, перенаправляет трафик через удаленный сервер, скрывая от веб-сайтов IP-адрес пользователя и его местоположение

Однако VPN работает на уровне операционной системы, то есть он перенаправляет весь трафик, поступающий как из браузера, так и из приложений.

VPN также шифрует трафик между интернетом и устройством. Наблюдатель знает только то, что вы подключены к VPN-серверу.

VPN может работать медленнее, чем прокси, поскольку ему нужно шифровать данные.

38 - История развития VPN.

Предпосылки появления VPN (Virtual Private Network)

В конце 1980-х и начале 1990-х годов корпорациям требовалось объединять свои удаленные офисы и филиалы в единую безопасную сеть. До появления VPN для этого использовались два пути:

1. Физическая прокладка собственных кабелей (невероятно дорого).
2. Аренда выделенных линий у провайдеров (технологии Frame Relay, ATM).

С бурным развитием глобального публичного Интернета возникла идея: вместо дорогостоящих выделенных каналов использовать общую инфраструктуру сети Интернет, но «нарезать» внутри нее изолированные, защищенные виртуальные туннели. Так родилась концепция VPN.

сука как это сделать красиво +- похуй пойдет

Эволюция поколений VPN-технологий

Первое поколение (Середина 1990-х)

PPTP (Point-to-Point Tunneling Protocol — 1996 год): Создан консорциумом компаний во главе с Microsoft. Протокол инкапсулировал кадры PPP (Point-to-Point Protocol) внутри IP-пакетов.

Архитектура: Использует TCP-порт 1723 для управления и протокол GRE (Generic Routing Encapsulation, IP-протокол номер 47) для передачи данных.

Итог: Был встроен во все версии Windows, что обеспечило ему огромную популярность. С точки зрения современной криптографии — критически **уязвим**, шифрование MPPE легко взламывается.

Второе поколение (Конец 1990-х)

L2TP (Layer 2 Tunneling Protocol — 1999 год, RFC 2661): Стал результатом объединения протокола PPTP от Microsoft и L2F от Cisco.

Особенность: Сам по себе L2TP не имеет встроенных механизмов шифрования трафика (он только создает туннель на канальном уровне). Поэтому его стали повсеместно использовать в связке с тяжелым архитектурным решением сетевого уровня — IPSec.

IPSec (Internet Protocol Security): Набор протоколов для защиты данных на сетевом уровне. Как указано в *Лекции 4*, его интеграция заложена непосредственно в архитектуру IPv6 через заголовки расширения:

АН (Authentication Header): Гарантирует целостность и аутентичность пакета.

ESP (Encapsulating Security Payload): Обеспечивает непосредственное шифрование полезной нагрузки.

Итог: Связка L2TP/IPSec стала «золотым стандартом» корпоративных сетей на долгие годы. Главный минус — сложность настройки и легкое обнаружение/блокировка сетевыми администраторами

Третье поколение (2000-е годы).

OpenVPN (2001 год): Революционный открытый протокол, который перенес логику создания VPN с сетевого уровня на прикладной. Вместо сложных конструкций IPSec он использует стандартную и проверенную библиотеку OpenSSL / TLS.

Плюсы: Может работать поверх как UDP, так и TCP на абсолютно любом порту (стандартный — UDP 1194). Это позволило маскировать VPN-трафик под обычные веб-запросы (например, пуская его через порт TCP 443).

Минусы: Работает в пространстве пользователя (User Space) ОС. Пересылка данных между ядром операционной системы и процессом OpenVPN требует постоянного контекстного переключения процессора, что сильно снижает скорость на высоконагруженных каналах.

SSTP (Secure Socket Tunneling Protocol): Закрытый аналог от Microsoft, полностью инкапсулирующий трафик в стандартную HTTPS-сессию (TCP 443).

Четвертое поколение (2015–2020 годы)

WireGuard (Появился в 2015, массовое внедрение к 2020 году): Протокол, совершивший прорыв в производительности за счет интеграции непосредственно в ядро операционной системы (Kernel Space).

Особенности: Обладает крайне лаконичным кодом (всего около 4 000 строк против сотен тысяч у OpenVPN/IPSec), использует современные быстрые криптографические примитивы (ChaCha20, Poly1305).

Итог: Обладает минимальным пингом и максимальной пропускной способностью, мгновенно переподключается при смене сети (мобильный роуминг). Однако WireGuard разрабатывался не для скрытности, а для скорости — его пакеты имеют четкую структуру.

НАСТОЯЩИЕ РЕАЛИИ. РКН, DPI, Обфускация и Антицензура (2012 — 2026 гг.)

Современные DPI-комплексы (включая российские ТСПУ) без труда распознают математические сигнатуры и структуру пакетов OpenVPN и WireGuard, мгновенно блокируя их, даже если они работают через стандартные веб-порты. Из-за этого классические VPN уступили место протоколам обфускации трафика:

Shadowsocks (2012 год): Изначально созданный в Китае протокол. Формально это не классический VPN (не создает виртуальный сетевой интерфейс TUN/TAP), а защищенный шифрованный прокси-сервер на базе SOCKS5. Его задача — сделать поток данных похожим на случайный, неструктурированный TCP-трафик, лишенный явных сигнатур.

Протоколы на базе бесшовного **TLS** (VLESS + Reality, Trojan, Xray) — актуальное настоящее:

Принцип работы (Reality): Протокол полностью отказывается от создания собственных легко вычисляемых криптографических отпечатков. Вместо этого при установлении соединения (Handshake) прокси-сервер «крадет» TLS-сертификат у легитимного разрешенного сайта (например, microsoft.com или apple.com) и выдает себя за него. Для систем DPI такой трафик выглядит как абсолютно законное обращение к серверам обновлений крупных технологических компаний.

Исторический инцидент (или как РКН нашалил): В ноябре 2025 года российские операторы связи провели масштабное обновление сигнатур ТСПУ, что временно привело к перебоям даже продвинутого протокола VLESS+Reality. Обходные пути со стороны разработчиков

потребовали тонкой настройки параметров отпечатка браузера (fingerprint) и манипуляций со строкой SNI (Server Name Indication).

39 - Принципы работы TOR браузеров, преимущества и недостатки.

TOR (The Onion Router) — технология анонимной передачи данных в сети, основанная на маршрутизации трафика через несколько промежуточных узлов с многослойным шифрованием.

Основная цель TOR — обеспечение анонимности пользователя и сокрытие его реального IP-адреса.

Для работы с сетью TOR обычно используется специальный браузер — Tor Browser.

Мосты Tor (Tor Bridges) — это ретрансляторы, которые помогают пользователям обойти блокировку Tor. Реализовано это путём выделения альтернативной точки входа в сеть Onion.

Snowflake — это система подключения однорангового сетевого трафика, созданная проектом Tor. Она сочетает прокси-сервер с протоколом WebRTC для автоматического назначения эфемерных мостов Tor. Конфиденциальность и анонимность здесь — опции по умолчанию.

TOR использует технологию **луковой маршрутизации (Onion Routing)**.

Перед отправкой данные несколько раз шифруются, после чего проходят через цепочку случайно выбранных узлов сети.

Этапы передачи данных

1. Пользователь отправляет запрос.
2. TOR формирует маршрут из нескольких узлов:
 - входной узел (Entry Node);
 - промежуточный узел (Relay Node);
 - выходной узел (Exit Node).
3. Данные многократно шифруются.
4. Каждый узел снимает только один слой шифрования и знает адрес лишь предыдущего и следующего узла.
5. Выходной узел отправляет запрос конечному серверу.
6. Ответ возвращается по аналогичному маршруту.

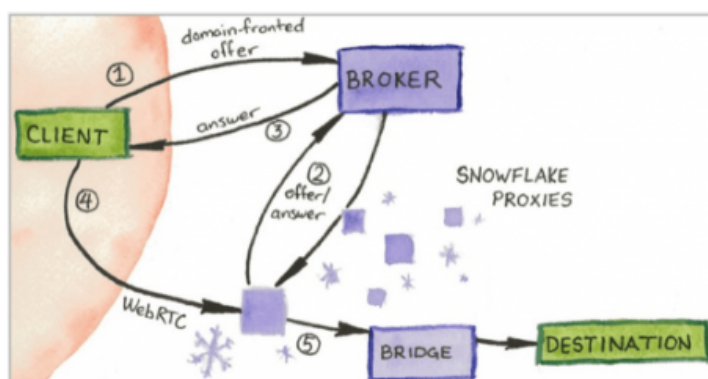
Таким образом, ни один узел сети не обладает полной информацией одновременно о пользователе и посещаемом ресурсе.

Преимущества TOR

1. *Анонимность пользователя*
Скрывается реальный IP-адрес и местоположение пользователя.
2. *Многослойное шифрование*
Каждый участок маршрута защищён дополнительным уровнем шифрования.
3. *Защита от отслеживания*
Усложняется анализ сетевого трафика и сбор информации о действиях пользователя.
4. *Децентрализованная структура*
Сеть состоит из большого числа независимых узлов по всему миру.
5. *Доступ к скрытым сервисам*
TOR позволяет работать с ресурсами в доменной зоне *.onion*.

Недостатки TOR

1. *Низкая скорость работы*
Трафик проходит через несколько узлов, что увеличивает задержки и снижает скорость передачи данных.
2. *Высокая нагрузка на сеть*
Дополнительное шифрование требует вычислительных ресурсов.
3. *Возможность анализа трафика*
При определённых условиях возможны атаки на анонимность пользователей с помощью корреляции трафика.
4. *Опасность выходных узлов*
Если соединение с сайтом не использует HTTPS, оператор выходного узла может просматривать передаваемые данные.
5. *Блокировка TOR-трафика*
Некоторые сайты и организации ограничивают или блокируют подключения через сеть TOR.



TOR BROWSER: ИНСТРУМЕНТ ПРИВАТНОСТИ, А НЕ НЕВИДИМОСТИ

Скрывает IP, усложняет слежку, но требует дисциплины и понимания границ защиты.



ИТОГ: Tor снижает риски, но не обнуляет их. Аккуратность и здравый смысл важнее «секретных настроек».

картинка от мулюхи вдруг тут понятнее

40 - Гонки процессов (состязания). Критические секции.

Гонки (Race Condition) – ситуация, когда конечный результат выполнения зависит от того, в какой конкретно последовательности произойдут операции в независимых процессах.

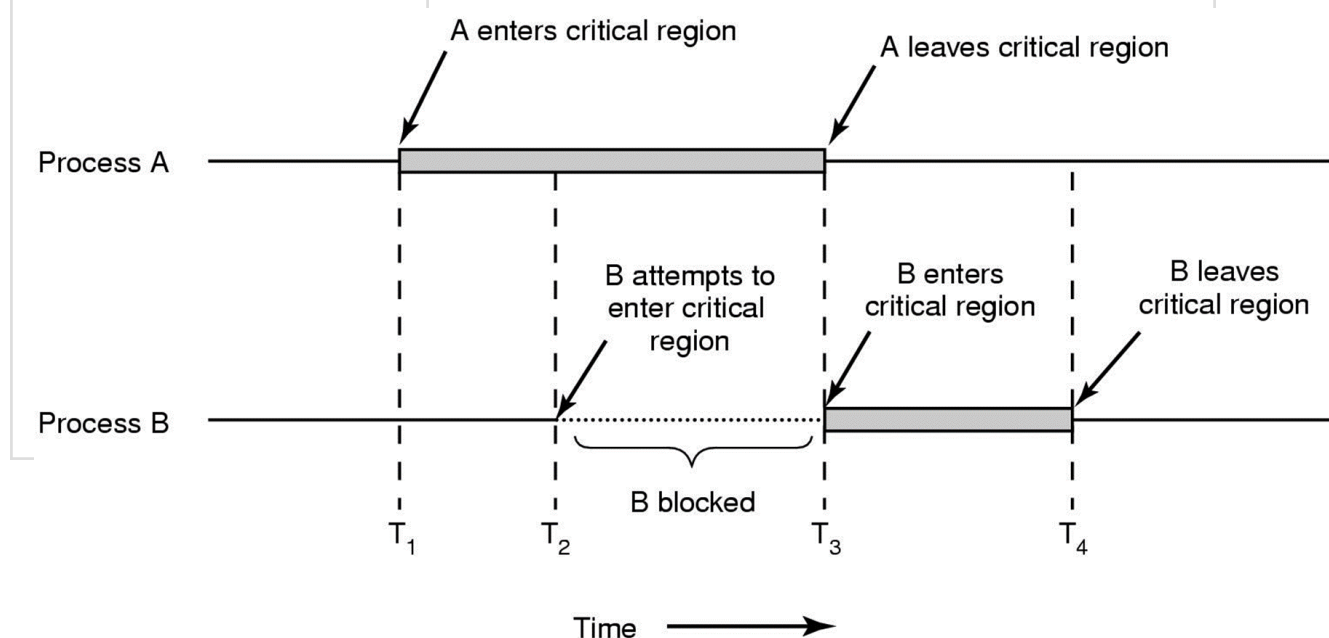
Если прям очень нужен пример, я думаю вы придумайте. Можете про $i++$ рассказать.

Критическая секция – это часть программы, результат выполнения которой может непредсказуемо меняться, если переменные, относящиеся к этой части программы, изменяются другими потоками в то время, когда выполнение этой части еще не завершено.

Если в критической секции состоит более одного процесса → возникает состояние «гонки».

Требования к критической секции:

Требование	Описание
1. Взаимное исключение (Mutual Exclusion)	В любой момент времени только один процесс может находиться в своей критической секции
2. Отсутствие предположений о скорости	Процессы не должны ждать много времени; нельзя полагаться на скорость выполнения процессов или количество процессоров
3. Ограниченное время в секции	Ни один процесс не может находиться в критической секции долгое время
4. Отсутствие блокировки внешних процессов	Ни один процесс вне критической секции не должен замедлять или блокировать другие процессы
5. Отсутствие вечного ожидания	Невозможна ситуация, в которой поток вечно ждёт попадания в критическую область



41 - Средства синхронизации процессов. Семафоры, мьютексы.

Средства синхронизации – механизмы, которые позволяют контролировать доступ к разделяемым ресурсам и предотвращают ошибки многопоточности, когда процессы пытаются обратиться к одним и тем же данным.

Мьютекс (Mutual Exclusion – взаимное исключение) – объект синхронизации, который устанавливается в сигнальное состояние, когда не занят каким-либо потоком.

- Только один поток владеет мьютексом в любой момент времени.
- Находится в адресном пространстве процесса, который его создал.

Принцип работы: Поток ждёт освобождения мьютекса (WaitOne), становится владельцем, пишет/читает общий ресурс, затем освобождает мьютекс (ReleaseMutex).

Мьютекс между процессами: Два или более процесса могут создать мьютекс с одним именем. Первый создает мьютекс, следующие получают хэндл существующего объекта. Это позволяет разным процессам взаимодействовать через один мьютекс.

Семафор – примитив синхронизации, в основе которого лежит счетчик с двумя атомарными операциями:

- **P (wait)** — если счетчик > 0 , уменьшает на 1 и продолжает процесс; если счетчик $= 0$, процесс ждёт
- **V (signal)** — увеличивает счетчик на 1 и будит ожидающие процессы

Ограничение: Ограничивает количество потоков, которые могут получить доступ к ресурсу одновременно. Например, семафор со значением 3 позволяет трём потокам одновременно работать с ресурсом.

В общем мьютекс это одноместный семафор, только у мьютекса есть “владелец” – его должен освободить тот, кто занял. А семафор просто неотрицательная величина – счетчик.

42 - Средства синхронизации процессов. События, таймеры, сигналы.

Средства синхронизации – механизмы, которые позволяют контролировать доступ к разделяемым ресурсам и предотвращают ошибки многопоточности, когда процессы пытаются обратиться к одним и тем же данным.

События

Событие (Event) — объект синхронизации, предназначенный для уведомления процессов или потоков о наступлении некоторого состояния или выполнении определённого действия.

События содержат счетчик числа пользователей (как и все объекты ядра) и две булевы переменные: одна сообщает тип данного объекта-события, другая – его состояние (свободен или занят).

Объекты-события бывают двух типов:

- *со сбросом вручную (manual-reset events),*
- *с автосбросом (auto-reset events).*

Разница в том, что первый вид события нужно применять если событие ждут несколько потоков. Только сброс вручную позволяет это сделать. Иначе первый же обработчик сбросит событие и другие потоки об этом не узнают.

Объекты-события обычно используют в том случае, когда какой-то поток выполняет инициализацию, а затем сигнализирует другому потоку, что тот может продолжить работу. Инициализирующий поток переводит объект "событие" в занятое состояние и приступает к своим операциям. Закончив, он сбрасывает событие в свободное состояние. Тогда другой поток, который ждал перехода события в свободное состояние, пробуждается и вновь становится планируемым.

Принцип работы событий

1. Один процесс ожидает наступления события.
2. Другой процесс устанавливает событие.
3. Ожидающий процесс получает уведомление и продолжает выполнение.

События используются для:

- синхронизации процессов;
- запуска операций по готовности данных;
- организации взаимодействия между потоками.

Таймеры

Ожидающие таймеры (waitable timers) – это объекты ядра, которые самостоятельно переходят в свободное состояние в определенное время или через регулярные промежутки времени.

Принцип работы таймеров

1. Устанавливается время ожидания.
2. Начинается отсчёт.
3. После истечения времени формируется уведомление или сигнал.

Применение таймеров

- выполнение периодических задач;
- контроль времени выполнения операций;
- организация задержек и расписаний;
- контроль тайм-аутов сетевых соединений.

Сигналы

Сигнал — асинхронное уведомление процесса о наступлении некоторого события.

Принцип работы сигналов

1. Операционная система или другой процесс отправляет сигнал.
2. Процесс получает уведомление.
3. Выполняется обработчик сигнала либо действие по умолчанию.

Применение сигналов

- завершение процессов;
- обработка ошибок;
- межпроцессное взаимодействие;
- уведомление о системных событиях.

Примеры сигналов

- *SIGINT* — прерывание программы пользователем (Ctrl+C);
- *SIGTERM* — запрос на завершение процесса;
- *SIGKILL* — принудительное завершение процесса;
- *SIGALRM* — сигнал от таймера;
- *SIGSTOP* — остановка процесса.

43 - Средства передачи информации между процессами. Каналы, сокеты.

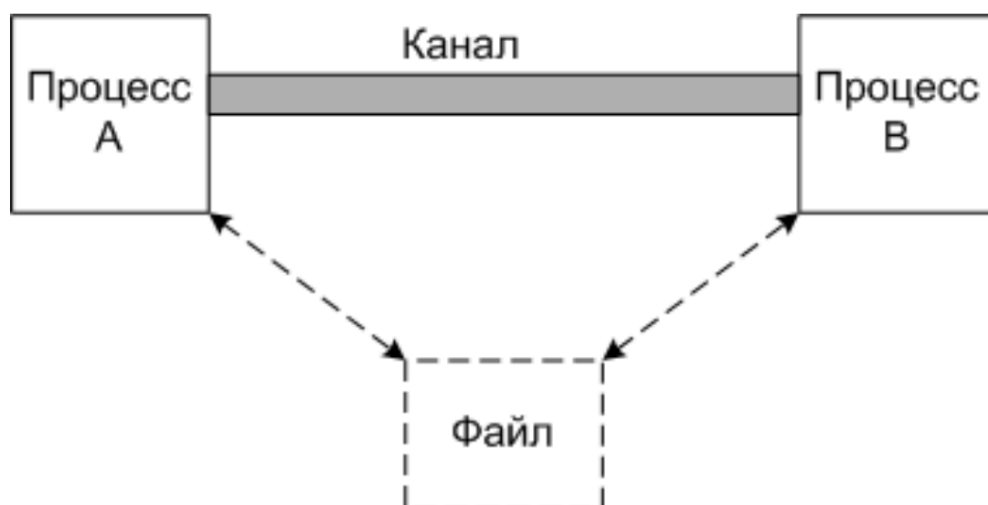
В современных операционных системах каждый процесс выполняется в своем изолированном виртуальном адресном пространстве. Прямой доступ одного процесса к памяти другого заблокирован на уровне ядра ради безопасности и стабильности системы.

IPC (Inter-Process Communication) — это набор механизмов, предоставляемых операционной системой, которые позволяют разделенным процессам безопасно взаимодействовать друг с другом. Согласно лекционному материалу, выделяют три главные задачи IPC:

- Передача информации от одного процесса другому.
- Синхронизация работы процессов.
- Предотвращение критических ситуаций (например, взаимных блокировок / Deadlocks и состояний гонки / Race Conditions).

Каналы

Канал представляет собой однонаправленный (полудуплексный) буфер данных в оперативной памяти, контролируемый ядром ОС. Один процесс пишет данные в дескриптор записи канала, а второй процесс считывает их из дескриптора чтения по принципу FIFO (First In, First Out — первым пришел, первым ушел). Все операционные системы, совместимые со стандартом POSIX, поддерживают механизмы каналов.



Каналы делятся на 2 типа:

1. Анонимные каналы

Используются исключительно для взаимодействия между связанными процессами — как правило, между родительским (Parent) и дочерним (Child) процессами.

Существуют только в оперативной памяти и только до тех пор, пока запущены создавшие их процессы. При их закрытии буфер уничтожается.

Активно применяются в CLI-оболочках (например, Bash в Linux) для перенаправления вывода одной утилиты на вход другой.

Пример синтаксиса: `cat log.txt | grep "Error"`. Символ вертикальной черты (|) указывает ядру создать неименованный канал, связав стандартный вывод (`stdout`) команды `cat` со стандартом ввода (`stdin`) команды `grep`.

2. NPFS (Named Pipe File System) или же Именованные каналы

Доступны для общения между абсолютно любыми (неродственными) процессами, запущенными в системе.

Имеют собственное имя и создаются в виде специального файла на диске (в Linux тип файла обозначается как `p`). Этот файл служит лишь точкой входа (адресом) для процессов, а сам обмен данными все так же происходит через буфер в оперативной памяти.

Именованные каналы (в отличие от анонимных) способны обеспечивать передачу информации между процессами, запущенными на разных компьютерах в рамках одной сети (механизм удаленного вызова процедур).

Сокеты

Если каналы изначально проектировались для работы внутри одной операционной системы, то сокеты стали универсальной революцией, объединившей локальное IPC и сетевое взаимодействие.

Сокет — это абстрактный конечный пункт (Endpoint) сетевых коммуникаций, предоставляемый операционной системой для

прикладных процессов. По сути, сокет — это управляемый операционной системой интерфейс, через который приложение передает данные в сеть и забирает их оттуда.

Свойства сокетов:

- Поддерживаются подавляющим большинством современных операционных систем.
- Назначение сокета в коде — связать конкретный процесс приложения с сетевой картой, определенным IP-адресом и портом транспортного уровня.
- **Идентификация (Адрес сокета):** На транспортном уровне сокет однозначно определяется кортежем [IP-адрес, Номер порта].

Классификация сетевых сокетов

При создании сокета процесс обязан явным образом выбрать тип транспортного протокола для доставки данных:

1. **Потоковые сокет (SOCK_STREAM):** Работают поверх протокола **TCP**. Ориентированы на предварительное установление соединения, гарантируют надежную доставку, сохранение порядка байтов и автоматическую сборку дубликатов пакетов.
2. **Датаграммные сокет (SOCK_DGRAM):** Работают поверх протокола **UDP**. Работают без установления соединения, отправляют данные независимыми пакетами (датаграммами). Не гарантируют доставку и порядок прихода, но обеспечивают минимальную задержку трафика.

Unix Domain Sockets (Локальные сокет)

Существует особый подвид сокетов (семейство **AF_UNIX**), который используется процессами на одном компьютере. Они полностью обходят сетевой стек (не вычисляют контрольные суммы, не используют IP и порты), за счет чего работают в несколько раз быстрее сетевых сокетов и классических именованных каналов.

44 - Средства передачи информации между процессами. Атомы, сообщения, почтовые ящики.

Межпроцессное взаимодействие (IPC, Interprocess Communication)

— это обмен данными между процессами, выполняющимися в одной или разных вычислительных системах.

Основные задачи IPC:

- передача данных;
- координация работы процессов;
- синхронизация выполнения;
- организация распределённых вычислений.

Атомы

Атом (Atom) — минимальная неделимая единица данных, передаваемая между процессами.

Атомы - это очень простой и доступный путь IPC. Идея состоит в том, что процесс может поместить строку в таблицу атомов и эта строка будет видна другим процессам. Когда процесс помещает строку в таблицу атомов, он получает 32-х битное значение (атом), и это значение используется для доступа к строке. Система не различает регистр строки

Набор атомов собирается в таблицу (atom table). Система обеспечивает несколько таблиц атомов для разных задач. По типу доступа их два типа:

- Локальные (доступны только из приложения)
- Глобальные (доступны из всех приложений)

Особенности атомов

- передаются как единый объект;
- обеспечивается целостность данных;
- используются для передачи простых команд, идентификаторов и небольших сообщений.

Сообщения

Сообщение — структурированный блок данных, передаваемый от одного процесса другому.

Сообщение обычно содержит:

- заголовок;
- служебную информацию;

- полезные данные.

Принцип работы

1. Процесс-отправитель формирует сообщение.
2. Сообщение передаётся через механизм IPC.
3. Процесс-получатель принимает и обрабатывает сообщение.

Почтовые ящики

Почтовый ящик (Mailslot) — специальный объект операционной системы, предназначенный для хранения и передачи сообщений между процессами.

Почтовый ящик выступает посредником между отправителем и получателем.

Mailslot является одним из механизмов, предназначенных для осуществления обмена данными между процессами (IPC). При этом процессы могут быть запущены как локально, так и разных ПЭВМ, включённых в одну ЛВС (удалённо).

Приложение-сервер открывает почтовый ящик, а клиенты могут писать в него. Ящик сохраняет сообщения до тех пор, пока сервер их не прочтет. Разумеется, одно приложение может одновременно быть сервером и клиентом, обеспечивая двунаправленную связь. При этом приложения могут находиться даже на разных компьютерах в сети.

Mailslot представляет собой псевдофайл, хранящийся в памяти. Для доступа к данным, содержащимся в этом псевдофайле, используются стандартные файловые функции ОС.

Объект Mailslot является временным объектом. После того, как будет закрыт последний дескриптор, ссылающийся на объект Mailslot, сам объект с данными будет уничтожен.

Обмен данными посредством Mailslot осуществляется в большинстве случаев между двумя процессами – клиентом и сервером.

45 - Средства передачи информации между процессами. Брокер RabbitMQ.

RabbitMQ – брокер сообщений с открытым исходным кодом, который маршрутизирует сообщения, отправленные приложением-продюсером (producer), к приложению-консьюмеру (consumer) через очередь сообщений. Основан на **TCP** соединении.

Брокер – центральная часть системы, которая принимает сообщения от производителей и маршрутизирует их к соответствующим очередям.

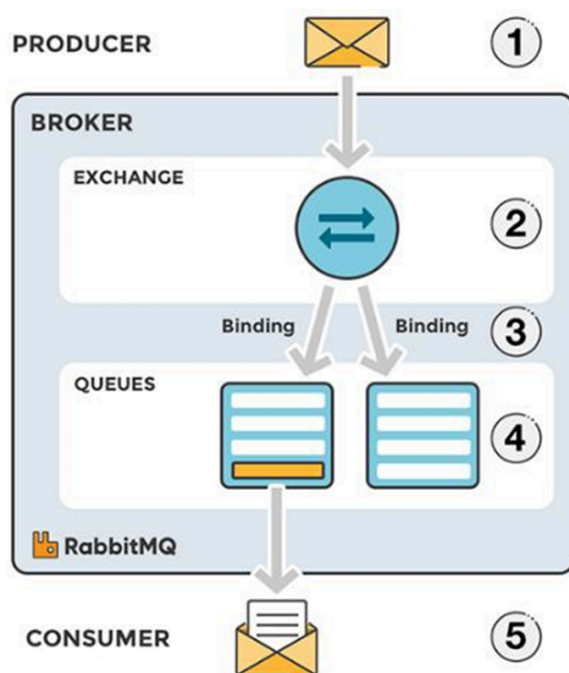
Producer (Продюсер) – приложение, которое отправляет сообщения в RabbitMQ.

Exchange (Обменник) – отвечает за маршрутизацию сообщений. Сообщения сначала отправляются в обменник, а не напрямую в очередь.

Queue (Очередь) – очередь для хранения сообщений до их получения консьюмером.

Binding (Привязка) – связь между обменником и очередью, определяющая, какие сообщения куда попадают.

Routing Key (Ключ маршрутизации) – метка сообщения, которая используется обменником для определения нужной очереди.

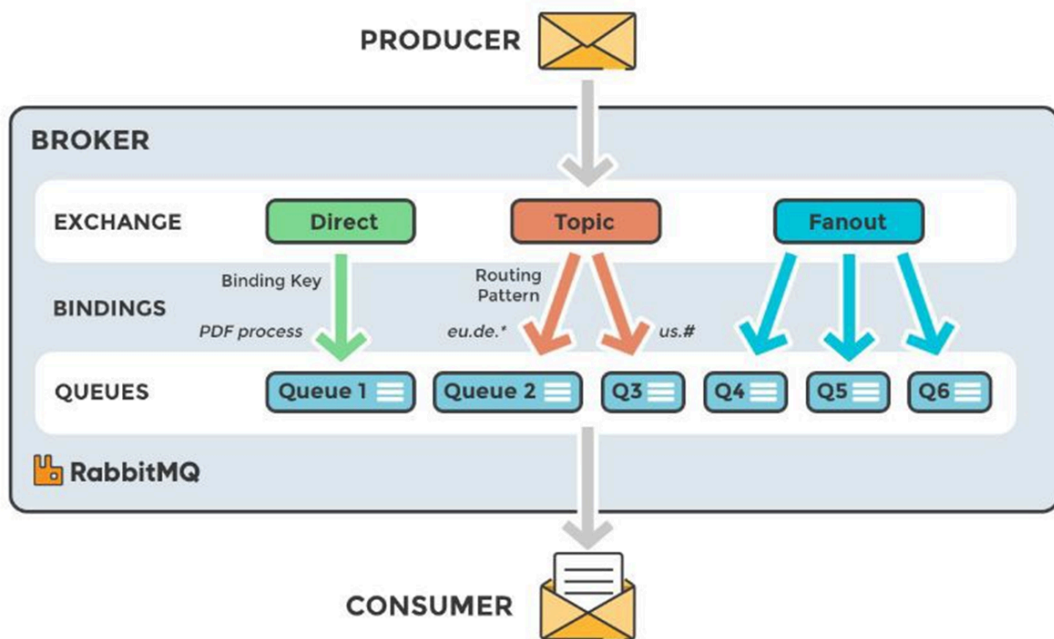


Характеристики:

- гарантия доставки сообщений (at least once delivery, at most once delivery)
- Гарантирует порядок ПЕРЕДАЧИ сообщений (FIFO)
- Возможность сохранения на диск
- Подтверждение отправки, подтверждение получения
- Ограничение кол-ва отправляемых сообщений
- Управление поведением полученных сообщений
- Кластеризация
- Зеркалирование

RabbitMQ поддерживает несколько типов Exchange:

- **Default**
 - **Не нужно** настраивать связь (Binding) между Exchange и очередями
 - Имя Exchange указывать не нужно; нужно указать имя очереди в "Routing Key"
 - Можно отправить сообщение в любую очередь в данном пространстве
- **Direct** (сообщение попадает в очередь, если ключ маршрутизации точно совпадает)
 - Необходимо настроить связи между очередью и обменником
 - Для отправки сообщения в очередь нужно указать имя очереди в "Routing Key"
- **Topic** (сообщение попадает в очередь, если ключ маршрутизации соответствует шаблону)
 - Поддерживает Binding с шаблонами
 - # – Wildcard (ноль или более слов)
 - * – одно слово в заданной позиции
 - Пример: agreements.*.headstore.#
 - Работает медленнее Direct
- **Fanout** (сообщение отправляется во все привязанные очереди (широковещание))
 - Необходимо настроить связи между очередью и обменником
- **Headers** (маршрутизация по заголовкам сообщения)
 - Поддерживает Binding по заголовкам, пары key = value
 - Самый медленный вариант
 - Поддерживает x-match (задает стратегию соответствия)
 - All – все пары должны совпасть
 - Any – любая пара
 - Пример: format = pdf, type = log, x-match = any



Флаги для очередей:

1. Параметры
 - durable (true) – постоянная очередь
 - exclusive (false) – только один клиент
 - auto-delete (false) – после отключения клиента
2. Аргументы:
 - x-message-ttl – время жизни сообщения
 - x-expires – время жизни очереди
 - x-max-length – максимальная длина очереди
 - x-overflow – стратегия переполнения
 - x-dead-letter-exchange – exchange для недоставленных соо
 - x-queue-mode – default \ lazy – режим работы очереди, когда сообщения сразу сбрасываются на диск

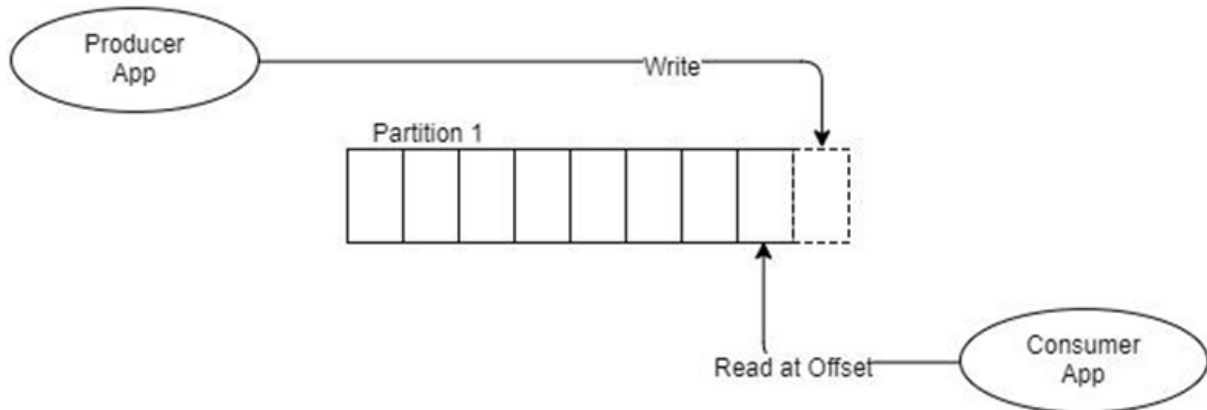
Кластер RabbitMQ

1. Publisher отправляет сообщение на любую ноду кластера RabbitMQ
2. Сервера RabbitMQ синхронизируют состояние сообщения в очереди
3. Сообщение, отправленное в очередь хранится на всех серверах кластера

По-умолчанию, если RabbitMQ начинает использовать больше 40% от общего объема памяти, то все соединения блокируются. После высвобождения процесс работы возобновляется. Порог можно переопределить.

46 - Средства передачи информации между процессами. Брокер Apache Kafka.

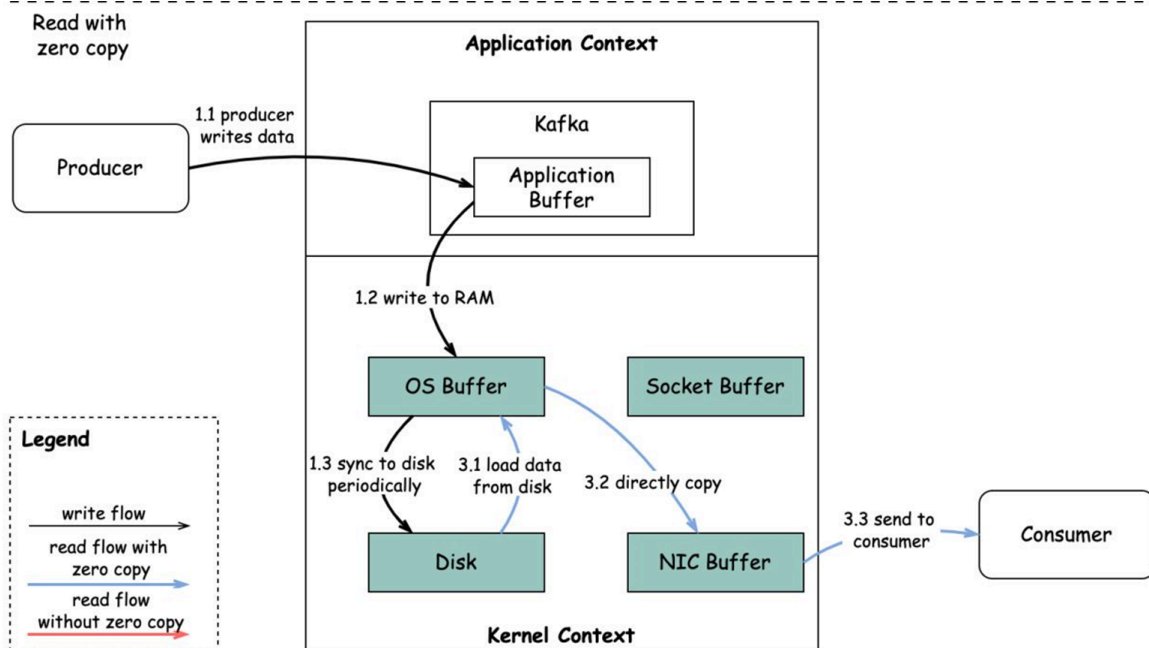
Kafka – это распределенный реплицированный журнал фиксации изменений (commit log).



Важное замечание: тут вообще нет очереди по сути, потому что в *partition* можно только записывать (доставать нельзя) и читать. То есть мы пишем лог, а консьюмер читает его, никто не “забирает” ничего из очереди. Консьюмер просто хранит указатель (место где читать).

Ключевая задача — сбор данных у приложений и их хранение в распределенном виде, организация потока данных из одной точки в другую с опциональной обработкой.

Хер знает зачем это (+я не очень понимаю), но пусть будет (из лекции)

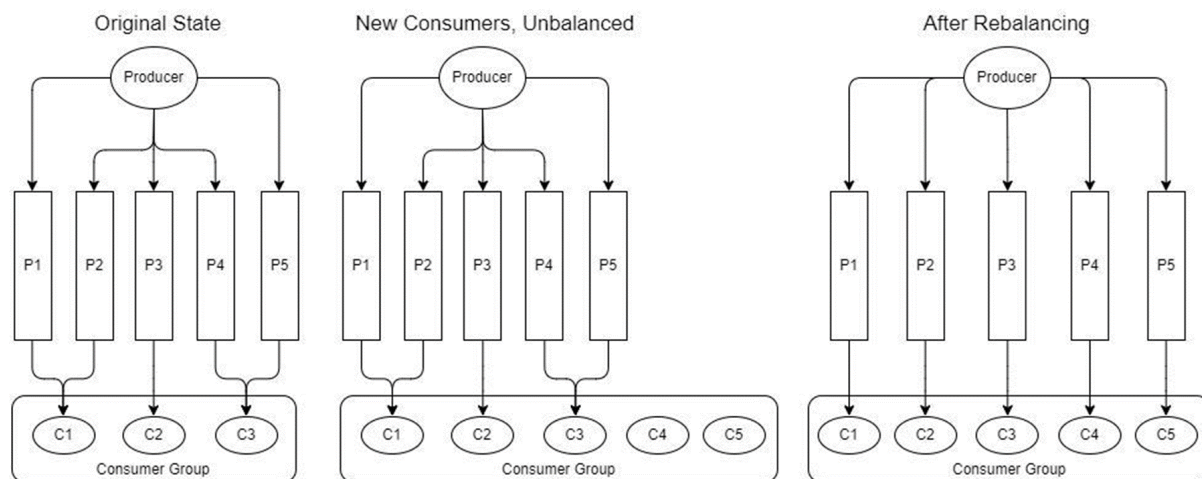


Характеристики:

- **Гарантия доставки** (если настроить acks)
- **Репликация журнала и синхронизация реплик** (каждый топик делится на партиции, каждая партиция имеет лидера и несколько реплик-фолловеров. Лидер принимает запись, фолловеры копируют данные для отказоустойчивости. Если лидер падает, фолловер становится новым лидером)
- **Уведомление о смещении получателя** (После прочтения сообщения консьюмер подтверждает смещение, чтобы Kafka знала, какие сообщения уже прочитаны)
- Уведомление о получении сообщения для продюсера
- **Передаёт данные с диска в сеть минуя большую часть работы с памятью** (Данные читаются с диска в page cache ОС и сразу отправляются в сеть, минуя копирование в пользовательское пространство)
- Активно использует память для кеширования
- **Последовательное чтение и запись на диск** (последовательные операции на диске намного быстрее случайных)
- API для producers/consumers
- **Поддержка kSQL** (ksqlDB для обработки потоков данных с SQL-подобным синтаксисом (фильтрация, агрегация, join))
- Consumer через pull запрашивает о появлении новых сообщений
- Журнал очищается на основе настроек времени и размера
- Есть возможность сжатия журнала
- Есть возможность сдвига смещения для переигрывания приема сообщений (консьюмер может вручную сдвинуть offset, чтобы перечитать сообщения заново или пропустить их)
- Для хранения метаданных используется zookeeper, в последних версиях отдельные топики kafka

Topic – категория или имя потока куда публикуются записи; **Partition** – шард топика.

Прикольная картинка (ребалансировка при добавлении консьюмеров в группу)



Zookeeper – это распределенное хранилище ключей и значений. Оно сильно оптимизировано для считывания, но записи в нем происходят медленнее.